HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

TIỂU LUẬN FINAL

AI, HỌC MÁY VÀ HỆ ĐA AGENT

PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Giảng viên:	PGS. TS. Trần Đình Quế
Sinh viên:	Kiều Trường Giang
Mã sinh viên:	B23DCCN254
Nhóm học phần:	06
Nhóm thực hiện:	02

Hà Nội, tháng 11 năm 2026

Mục lục

Mở đầu **1**

I Nền tảng và tổng hợp **2**

1 LỊCH SỬ VÀ PHÁT TRIỂN AI **3**

1.1. AI là gì? Bản chất của Trí tuệ Nhân tạo 3
1.2. Biên niên sử phát triển AI qua các kỷ nguyên 4
1.3. Bản đồ phân loại toàn cảnh các kỹ thuật AI 8
1.4. Ba trụ cột tạo nên sự bùng nổ của Deep Learning sau 2012 9
1.5. Phân loại AI theo mức độ năng lực và Trường phái tiếp cận 10
1.6. Xu hướng AI tại Việt Nam và Những thách thức đương đại 10
1.7. Mối liên hệ giữa các giai đoạn phát triển 11
1.8. Tiểu kết chương 1 12

2 CÁC KỸ THUẬT MACHINE LEARNING CƠ BẢN **14**

2.1. Cơ sở lý thuyết Học máy (Machine Learning Foundations) 14
2.2. Phân tích và chuẩn bị dữ liệu thực nghiệm 21
2.3. So sánh và lựa chọn mô hình 24
2.4. Kết quả test độc lập và phân tích sai số 25
2.5. Kết luận chương 28

3 MẠNG NƠ-RON TÍCH CHẬP CNN **29**

3.1. Lý thuyết CNN 29
3.2. Dữ liệu ảnh và kiến trúc thực nghiệm 31
3.3. Kết quả và phân tích dự đoán 33
3.4. Suy luận, triển khai và phạm vi mở rộng 37

4 MẠNG NƠ-RON HỒI TIẾP RNN **38**

4.1. Lý thuyết RNN 38
4.2. Dữ liệu, phép chia và cấu hình 40
4.3. So sánh RNN, LSTM, GRU và Persistence 42

4.4. Hạn chế và khả năng mở rộng 45

5 KNOWLEDGE GRAPH VÀ RAG 47
5.1. MỞ ĐẦU 47
5.2. KNOWLEDGE GRAPH - KNOWLEDGE BASE 49
5.3. RAG - RETRIEVAL-AUGMENTED GENERATION 52
5.4. KB VÀ RAG CHO BỆNH TIỂU ĐƯỜNG 54
5.5. KB VÀ RAG CHO CHỨNG KHOÁN 57
5.6. KB VÀ RAG CHO GIÁ VÀNG 60
5.7. DEPLOYMENT 63
5.8. Đánh giá tổng hợp 65
5.9. Kết luận chương 71

6 Học sâu trên đồ thị 72
Từ bảng dữ liệu đến cấu trúc liên kết 72
Cơ chế truyền thông điệp 72
Các biến thể và lựa chọn phù hợp 73
Áp dụng vào kho tri thức của project 73
Thiết kế một thí nghiệm mở rộng 73
Hạn chế cần kiểm soát 74

7 Agent, hệ đa agent và học tăng cường 75
Khái niệm agent 75
Phối hợp và giao thức 75
Mô hình học tăng cường 75
Kết hợp học với luật 76
Liên hệ các phần tiểu luận 76

II Project tích hợp 77

1 Giới thiệu 78
1.1 Bài toán và sản phẩm 78
1.2 Phạm vi đầu ra 78
1.3 Mục tiêu đánh giá 78

2 Agent và cơ chế phối hợp 79
2.1 Vai trò xử lý 79
2.2 Reinforcement Learning kết hợp luật 79
2.3 Giới hạn của kiến trúc 80

3 Kiến trúc tổng quan **81**
3.1 Nhánh dữ liệu . . . 81
3.2 Nhánh tri thức . . . 81
3.3 Luồng người dùng . . . 82

4 Các thành phần và thực nghiệm **83**
4.1 Phân loại . . . 83
4.2 Hồi quy . . . 83
4.3 Kết quả kiểm tra . . . 84
4.4 Truy xuất và đồ thị . . . 84
4.5 Phân tích sai số theo lớp . . . 84
4.6 Phân tích phần dư hồi quy . . . 86
4.7 Hợp đồng dữ liệu và API . . . 86
4.8 Kết quả kiểm thử tích hợp . . . 86
4.9 Đối chiếu với sơ đồ yêu cầu . . . 86

5 Code và demo **88**
5.1 Địa chỉ và cách chạy . . . 88
5.2 Kịch bản trình diễn . . . 88
5.3 Hạn chế và hướng hoàn thiện . . . 89
5.4 Ảnh chụp sản phẩm đang chạy . . . 91

A Mã nguồn tái lập **95**
A.1 scripts/ingest_graph.py . . . 95
A.2 scripts/train_models.py . . . 95
A.3 scripts/train_policy.py . . . 96
A.4 app/agents.py . . . 97
A.5 app/main.py . . . 97
A.6 scripts/test_scenarios.py . . . 99

Mở đầu

Báo cáo tổng hợp nền tảng trí tuệ nhân tạo, học máy, mạng nơ-ron, đồ thị tri thức và cơ chế phối hợp đa agent. Phần thứ nhất kế thừa nội dung của tiểu luận 02 và 03 của Kiều Trường Giang, đồng thời bổ sung học sâu trên đồ thị và hệ đa agent. Tiểu luận 01 của Phạm Mai Linh được dùng làm tài liệu tham khảo nền tảng theo lựa chọn của sinh viên; các số liệu không được nhận là kết quả mới nếu chỉ xuất hiện trong tài liệu tham khảo.

Phần thứ hai triển khai một sản phẩm mới theo sơ đồ trên lớp: dữ liệu có sẵn đi qua mô hình Deep Learning để tạo kết quả phân loại và dự đoán; nguồn tri thức đi qua KB và RAG; các thành phần được điều phối bằng agent, chính sách học và luật để đưa ra phản hồi. Báo cáo project riêng trình bày chi tiết kiến trúc, thực nghiệm, API và kịch bản demo. Các số liệu mới đều có tệp kết quả trong thư mục results.

Bố cục tham khảo hai báo cáo của Nguyễn Tự Kiên do sinh viên cung cấp. Bản hiện thực không nhận các bộ dữ liệu, số agent hoặc kết quả của bài tham khảo là thành phần đã được triển khai. Cơ chế sinh phản hồi hiện là tổng hợp trích đoạn có nguồn; GNN trong phần lý thuyết chưa được huấn luyện trong demo.

Phần I

Nền tảng và tổng hợp

Chương 1

LỊCH SỬ VÀ PHÁT TRIỂN AI

Một hệ thống có thể dự đoán khá chính xác nhưng vẫn không giải thích được vì sao nó đưa ra kết quả đó. Ngược lại, một bộ luật có lời giải thích rõ ràng lại dễ bỏ sót những tình huống không được người thiết kế dự kiến. Sự đánh đổi giữa khả năng biểu diễn tri thức, học từ dữ liệu và giải thích quyết định là vấn đề xuyên suốt quá trình phát triển trí tuệ nhân tạo.

Chương này xem xét các giai đoạn phát triển từ góc độ bài toán mà mỗi hướng tiếp cận muốn giải quyết: làm thế nào biểu diễn được hiểu biết, học được quy luật trong dữ liệu và sử dụng kiến thức ngoài mô hình. Những giai đoạn này không tách rời nhau. Hệ thống trong báo cáo sử dụng mô hình thống kê cho dữ liệu bảng, mạng nơ-ron cho ảnh và chuỗi, đồng thời dùng đồ thị và truy xuất để tổ chức bằng chứng.

Trình tự trình bày đi từ khái niệm và các cột mốc đến điều kiện tạo nên bước tiến của học máy, học sâu và mô hình sinh. Phần cuối liên hệ những hướng tiếp cận đó với lựa chọn thiết kế trong các chương thực nghiệm, làm rõ vì sao không nên dùng cùng một mô hình cho mọi dạng đầu vào.

1.1. AI là gì? Bản chất của Trí tuệ Nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence – AI) là một phân ngành khoa học máy tính chuyên sâu, nghiên cứu và xây dựng các hệ thống tính toán có khả năng thực thi những tác vụ vốn dĩ đòi hỏi năng lực nhận thức và trí tuệ của con người: nhận thức không gian thị giác, phân tích quy nạp, suy luận logic, hiểu ngôn ngữ tự nhiên, học hỏi kinh nghiệm từ quá khứ và giải quyết vấn đề trong môi trường bất định.

Hội thảo Dartmouth năm 1956 đánh dấu sự hình thành của AI như một lĩnh vực nghiên cứu. John McCarthy dùng tên gọi Artificial Intelligence cho hướng xây dựng các hệ thống máy tính thông minh.

Trong giáo trình kinh điển thế giới Artificial Intelligence: A Modern Approach, hai tác giả Stuart Russell và Peter Norvig đã tổng hợp và phân loại các cách tiếp cận AI trong lịch sử theo hai chiều kích: Tư duy vs. Hành động và Mô phỏng con người vs. Tính duy lý (Hợp lý):

Bảng 1.1. Bốn hướng tiếp cận trí tuệ nhân tạo

Chiều kích	Tiếp cận dựa trên con người	Tiếp cận dựa trên tính hợp lý
Tư duy	Tư duy như con người (Thinking Humanly). Khoa học nhận thức; mô phỏng quá trình nơ-ron não bộ xử lý thông tin.	Tư duy duy lý (Thinking Rationally). Dựa trên các quy luật logic hình thức (Boole, Frege); rút ra kết luận đúng đắn từ các tiền đề.
Hành động	Hành động như con người (Acting Humanly). Tiêu biểu là phép thử Turing (1950); hành vi máy khiến con người tin rằng nó là người.	Hành động duy lý (Acting Rationally). Tác tử hợp lý quan sát môi trường và hành động tối đa hóa kỳ vọng đạt mục tiêu.

Cách tiếp cận hiện đại chủ đạo: Ngày nay, cộng đồng AI toàn cầu chọn cách tiếp cận Tác tử hợp lý (Rational Agent) làm nền tảng. Thay vì cố gắng sao chép từng hành vi

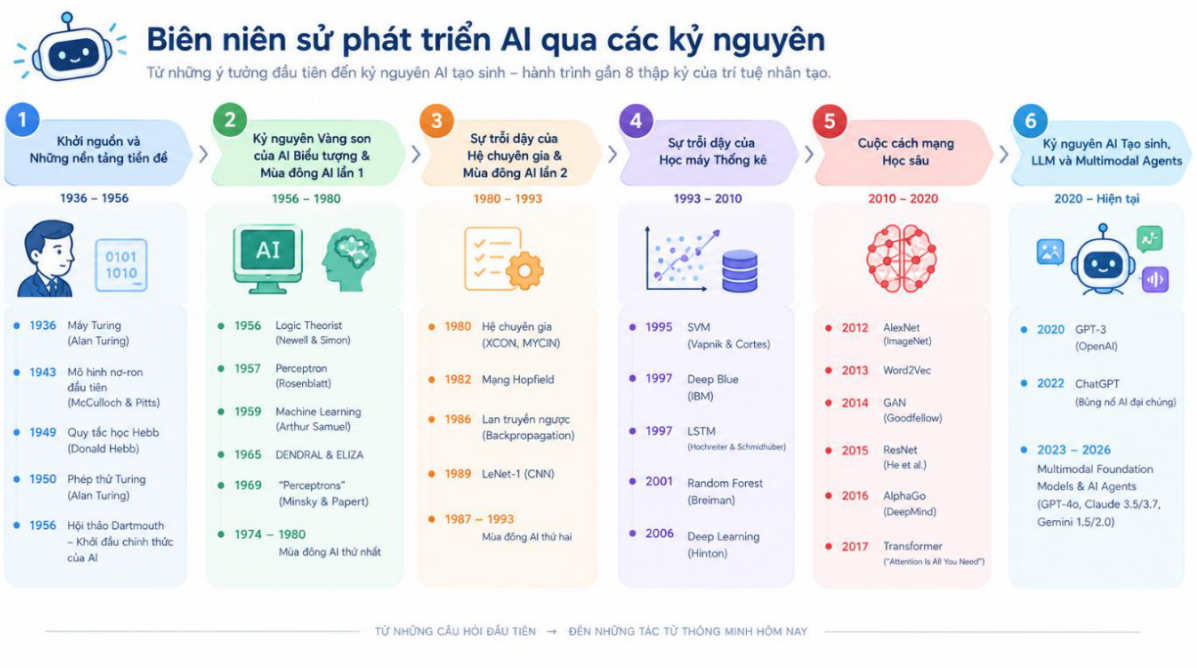
cảm tính của con người, hệ thống AI được thiết kế để nhận tri giác (Percepts) thông qua cảm biến, xử lý thông tin bằng thuật toán toán học và đưa ra hành động (Actions) thông qua cơ cấu chấp hành nhằm tối đa hóa thước đo hiệu năng (Performance Measure).

Phân biệt mối quan hệ giữa các khái niệm cốt lõi:

- AI (Trí tuệ nhân tạo): Chiếc ô rộng nhất, bao gồm cả AI dựa trên luật (Rule-based), logic hình thức, hệ chuyên gia và học máy.
- Machine Learning (Học máy): Nhánh con trọng yếu của AI, tập trung vào việc tự động rút ra quy luật thống kê từ dữ liệu.
- Deep Learning (Học sâu): Nhánh con của học máy, sử dụng mạng nơ-ron nhiều lớp (Deep Neural Networks) để tự động học biểu diễn đặc trưng phân cấp từ dữ liệu thô.

1.2. Biên niên sử phát triển AI qua các kỷ nguyên

Lịch sử phát triển AI bao gồm các giai đoạn kỳ vọng cao và những đợt suy giảm đầu tư. Nhiều thay đổi về phương pháp xuất phát từ giới hạn kỹ thuật của giai đoạn trước, cùng sự xuất hiện của dữ liệu và phần cứng mới.



Hình 1.1. Các giai đoạn và quan hệ giữa các kỹ thuật AI

Giai đoạn 1: Khởi nguồn và Những nền tảng tiền đề (1936 – 1956)

• 1936 — Máy Turing (Alan Turing): Alan Turing công bố công trình lịch sử "On Computable Numbers", hình thức hóa khái niệm "thuật toán" và "tính toán" thông qua cỗ máy trừu tượng Turing Machine. Đây là nền móng toán học khẳng định khái niệm tính được bằng thuật toán có thể được mô hình hóa chặt chẽ; điều này không chứng minh mọi hoạt động tư duy đều tính được.

• 1943 — Mô hình nơ-ron nhân tạo đầu tiên (Warren McCulloch & Walter Pitts): Đề xuất mô hình toán học nhị phân mô phỏng tế bào thần kinh sinh học, chứng minh rằng một mạng lưới các nơ-ron logic có thể tính toán bất kỳ hàm logic nào.

• 1949 — Quy tắc học Hebb (Donald Hebb): Cuốn sách "The Organization of Behavior" chỉ ra rằng khi hai tế bào thần kinh được kích hoạt đồng thời, liên kết giữa chúng sẽ mạnh lên ("Cells that fire together, wire together"). Đây là nguồn gốc của việc điều chỉnh trọng số liên kết trong mạng nơ-ron.

• 1950 — Phép thử Turing & Câu hỏi lịch sử: Alan Turing công bố bài báo "Computing Machinery and Intelligence", đặt ra câu hỏi bất hủ: "Máy móc có thể suy nghĩ được không?" và giới thiệu phép thử Turing (Imitation Game).

• 1956 — Hội thảo Dartmouth — Điểm khởi đầu chính thức: John McCarthy, Marvin Minsky, Nathaniel Rochester và Claude Shannon đồng tổ chức hội thảo 2 tháng tại Đại học Dartmouth. Thuật ngữ "Artificial Intelligence" chính thức ra đời, đánh dấu sự hình thành của một ngành khoa học độc lập.

Giai đoạn 2: Kỷ nguyên Vàng son của AI Biểu tượng & Mùa đông AI lần 1 (1956 – 1980)

• 1956 — Logic Theorist (Newell & Simon): Chương trình AI đầu tiên có khả năng chứng minh 38 trong 52 định lý của chương 2 cuốn sách Principia Mathematica, thậm chí tìm ra cách chứng minh ngắn gọn hơn cả các tác giả gốc.

• 1957 — Perceptron (Frank Rosenblatt): Phát minh ra thuật toán Perceptron – mô hình học máy đầu tiên có thể học trọng số từ dữ liệu để phân loại nhị phân trên phần cứng Mark I Perceptron. Rosenblatt tuyên bố máy sẽ sớm có thể nhìn, nói và tự sinh sản.

• 1959 — Khái niệm ”Machine Learning” (Arthur Samuel): Viết chương trình chơi cờ đam (checkers) tự cải thiện năng lực qua từng ván đấu và định nghĩa Machine Learning: ”Khoa học giúp máy tính có khả năng tự học mà không cần được lập trình chi tiết từng bước”.

• 1965 — DENDRAL & ELIZA (1966): Hệ thống DENDRAL phân tích cấu trúc phân tử hóa học (tiền thân của hệ chuyên gia); Joseph Weizenbaum phát triển chat-bot ELIZA mô phỏng nhà tâm lý học giao tiếp qua khớp mẫu từ khóa.

• 1969 — Cuốn sách ”Perceptrons” (Minsky & Papert): Cuốn sách toán học chỉ ra giới hạn chết người của Perceptron đơn lớp: hoàn toàn không thể giải quyết bài toán XOR (bài toán phân tách phi tuyến). Kết luận bi quan này khiến làn sóng nghiên cứu mạng nơ-ron bị đình trệ nghiêm trọng suốt hơn một thập kỷ.

• 1974–1980 — Mùa đông AI thứ nhất (The First AI Winter): Báo cáo Lighthill (1973) tại Anh và các đánh giá của DARPA tại Mỹ kết luận AI đã hứa hẹn quá mức so với thực tế (”Bùng nổ tổ hợp”, máy tính không đủ bộ nhớ để giải bài toán ngoài ”thế giới đồ chơi”). Tài trợ bị cắt giảm ồ ạt, các dự án AI bước vào giai đoạn thoái trào sâu rộng.

Giai đoạn 3: Sự trỗi dậy của Hệ chuyên gia & Mùa đông AI lần 2 (1980 – 1993)

• 1980 — Sự bùng nổ của Hệ chuyên gia (Expert Systems): AI chuyển hướng sang AI tri thức biểu tượng. Hệ thống mã hóa tri thức chuyên gia dưới dạng hàng nghìn luật ”NẾU... THÌ...”. Hệ thống XCON của Digital Equipment Corporation (DEC) giúp tiết kiệm 40 triệu USD/năm; MYCIN chẩn đoán nhiễm trùng máu chính xác ngang chuyên gia hàng đầu.

• 1982 — Mạng Hopfield (John Hopfield): Giới thiệu mạng nơ-ron hồi quy có khả năng lưu trữ thông tin như bộ nhớ liên kết (Associative Memory), làm bừng tỉnh lại cộng đồng nghiên cứu mạng nơ-ron.

• 1986 — Thuật toán Lan truyền ngược (Backpropagation): David Rumelhart, Geoffrey Hinton và Ronald Williams xuất bản bài báo kinh điển trên tạp chí Nature, tái khám phá và phổ biến giải thuật Lan truyền ngược lỗi cho mạng nơ- ron nhiều lớp. Giới hạn hàm XOR của Minsky chính thức bị đập tan.

• 1989 — LeNet-1 (Yann LeCun): Yann LeCun sáng tạo ra mạng nơ-ron tích chập (CNN) kết hợp lan truyền ngược, ứng dụng thành công trong việc nhận dạng chữ số viết tay trên séc ngân hàng bưu điện Mỹ.

• 1987–1993 — Mùa đông AI thứ hai (The Second AI Winter): Các hệ chuyên gia bộc lộ điểm nghẽn nghiêm trọng: chi phí bảo trì khổng lồ, ”nút thắt thu thập tri thức” (Knowledge Acquisition Bottleneck), hệ thống giòn giã (brittle) – thất bại hoàn toàn khi gặp tình huống ngoại lệ. Thị trường phần cứng chuyên biệt LISP sụp đổ trước sự vươn lên của PC thương mại. AI một lần nữa rơi vào khủng hoảng lòng tin.

Giai đoạn 4: Sự trỗi dậy của Học máy Thống kê (1993 – 2010)

Từ bỏ tham vọng giải quyết trí tuệ tổng quát bằng các luật cứng nhắc, AI chuyển dịch trọng tâm sang phương pháp thống kê, xác suất và tối ưu hóa lồi:

• 1995 — Máy vector hỗ trợ (Support Vector Machine – SVM): Vladimir Vapnik và Corinna Cortes công bố thuật toán SVM với nguyên lý tối đa hóa biên phân cách (Margin Maximization) và kỹ thuật Kernel Trick. SVM nhanh chóng trở thành công cụ phân loại thống trị nhờ cơ sở lý thuyết toán học vững chắc và tránh được cực tiểu địa phương.

• 1997 — Deep Blue đánh bại Garry Kasparov: Siêu máy tính Deep Blue của IBM đánh bại nhà vô địch cờ vua thế giới Garry Kasparov, thu hút sự chú ý về năng lực tính toán của máy tính.

• 1997 — Mạng LSTM ra đời (Hochreiter & Schmidhuber): Sepp Hochreiter và Jürgen Schmidhuber phát minh kiến trúc Long Short-Term Memory (LSTM), giảm khó khăn học phụ thuộc dài hạn do biến mất gradient trong mạng RNN, đặt nền móng cho xử lý chuỗi thời gian và NLP hiện đại.

• 2001 — Random Forest (Leo Breiman): Đề xuất thuật toán Rừng ngẫu nhiên kết hợp Bagging và lựa chọn ngẫu nhiên tập đặc trưng, trở thành thuật toán mạnh mẽ và bền bỉ hàng đầu cho dữ liệu dạng bảng.

• 2006 — Geoffrey Hinton và ”Deep Learning”: Hinton cùng các cộng sự công bố kỹ thuật tiền huấn luyện từng lớp (Layer-wise Pre-training) cho mạng Deep Belief Networks, góp phần khôi phục quan tâm đến huấn luyện mạng sâu và nhen nhóm sự phát triển của học sâu.

Giai đoạn 5: Cuộc cách mạng Học sâu (Deep Learning Revolution) (2010 – 2020)

• 2012 — AlexNet và bước ngoặt ImageNet: Alex Krizhevsky, Ilya Sutskever và Geoffrey Hinton giành chiến thắng áp đảo tại cuộc thi ILSVRC với mạng CNN 8 lớp mang tên AlexNet, huấn luyện song song trên 2 GPU NVIDIA GTX 580. Tỉ

lệ lỗi Top-5 giảm từ 26% xuống 15.3%. Khoảnh khắc này đánh dấu sự ưu thế của mạng sâu trên benchmark này so với các phương pháp thiết kế đặc trưng thủ công.

• 2013 — Word2Vec (Tomas Mikolov / Google): Ánh xạ từ ngữ thành vector ngữ nghĩa trong không gian liên tục (Vector Embedding), nắm bắt quan hệ ngữ nghĩa tinh tế.

• 2014 — Mạng sinh đối kháng GAN (Ian Goodfellow): Đề xuất kiến trúc hai mạng Generator và Discriminator thi đấu đối kháng, khai sinh lĩnh vực tạo sinh hình ảnh chân thực.

• 2015 — Mạng ResNet (Kaiming He et al.): Sáng tạo cơ chế kết nối tắt (Residual Skip Connections), cho phép huấn luyện thành công các mạng nơ-ron sâu tới 152 lớp mà không bị suy thoái gradient, vượt qua độ chính xác của con người trên ImageNet.

• 2016 — AlphaGo đánh bại Lee Sedol: Mô hình AlphaGo (DeepMind) kết hợp Deep RL và tìm kiếm cây Monte Carlo thắng kỳ thủ cờ vây hàng đầu thế giới 4- 1, phá vỡ định kiến rằng cờ vây nằm ngoài khả năng của AI.

• 2017 — Kiến trúc Transformer (”Attention Is All You Need”): Đội ngũ nghiên cứu Google đề xuất kiến trúc Transformer dựa thuần túy trên cơ chế tự chú ý (Self-Attention), loại bỏ hoàn toàn cấu trúc tuần tự của RNN, tạo bước ngoặt khai sinh các mô hình nền tảng như BERT, GPT.

Giai đoạn 6: Kỷ nguyên AI Tạo sinh, LLM và Multimodal Agents (2020 – Hiện tại)

• 2020 — GPT-3 (OpenAI): Mô hình ngôn ngữ với 175 tỉ tham số thể hiện năng lực học vài ví dụ (Few-shot In-context Learning) phi thường mà không cần tái huấn luyện trọng số.

• 2022 — ChatGPT & Bùng nổ AI Đại chúng: Kết hợp tinh chỉnh chỉ dẫn (Instruction Tuning) và Học tăng cường từ phản hồi của con người (RLHF), ChatGPT cán mốc 100 triệu người dùng chỉ sau 2 tháng, mở màn cuộc cách mạng AI tạo sinh thương mại hóa toàn cầu.

• Từ 2023: Mô hình nền tảng đa phương thức kết hợp văn bản, hình ảnh và âm thanh. Các tác tử dùng mô hình ngôn ngữ để lựa chọn công cụ và thực hiện chuỗi thao tác, đồng thời cần kiểm soát quyền, đánh giá lỗi và giám sát con người.

1.3. Bản đồ phân loại toàn cảnh các kỹ thuật AI

Dưới đây là sơ đồ kiến trúc phân cấp toàn cảnh, thể hiện mối quan hệ hữu cơ giữa các trường phái Trí tuệ Nhân tạo từ cổ điển đến hiện đại:



Hình 1.2. Các giai đoạn và quan hệ giữa các kỹ thuật AI

1.4. Ba trụ cột tạo nên sự bùng nổ của Deep Learning sau 2012

Tại sao các ý tưởng mạng nơ-ron đã xuất hiện từ thập niên 1980 nhưng phải đợi tới sau năm 2012 mới tạo nên cuộc cách mạng công nghệ bùng nổ? Câu trả lời nằm ở sự hội tụ đồng thời của ba trụ cột nền tảng:

Bảng 1.2. Ba trụ cột phát triển của học sâu

Trụ cột	Bản chất kỹ thuật	Cột mốc minh họa
Dữ liệu số khổng lồ	Mạng nơ-ron sâu chứa hàng triệu đến hàng tỷ tham số, do đó đòi hỏi tập dữ liệu khổng lồ để tránh overfitting và đạt khả năng tổng quát hóa cao.	Dự án ImageNet gán nhãn hơn 14 triệu hình ảnh, cung cấp dữ liệu tiêu chuẩn cho mô hình học sâu.
Tính toán GPU	Mạng nơ-ron là chuỗi phép nhân ma trận song song; kiến trúc nhiều nhân của GPU phù hợp hơn CPU truyền thống.	CUDA xuất hiện năm 2007; AlexNet năm 2012 huấn luyện trên hai GPU GTX 580.
Thuật toán và kiến trúc	ReLU, Dropout, BatchNorm, Adam và kết nối tắt hỗ trợ tối ưu mạng sâu và hạn chế quá khớp.	Dropout (2012), BatchNorm (2015), Adam (2014), ResNet (2015).

1.5. Phân loại AI theo mức độ năng lực và Trường phái tiếp cận

1.5.1. Phân loại theo cấp độ năng lực

• AI Hẹp (Artificial Narrow AI – ANI): Hệ thống AI được tối ưu hóa xuất sắc cho một nhiệm vụ duy nhất cụ thể (chơi cờ vây, nhận dạng khuôn mặt, dịch máy, phân loại ảnh). Mặc dù thể hiện sự vượt trội so với con người trong miền hẹp, ANI hoàn toàn không có nhận thức hay khả năng tự thích ứng ngoài phạm vi được huấn luyện. Toàn bộ các hệ thống AI ứng dụng hiện nay đều thuộc cấp độ này.

• AI Tổng quát (Artificial General Intelligence – AGI): Hệ thống trí tuệ nhân tạo giả định có năng lực tư duy, suy luận, tự học và giải quyết vấn đề linh hoạt tương đương trí tuệ con người trên mọi lĩnh vực tri thức. Hiện nay, các mô hình nền tảng đa phương thức (Foundation Models) đang từng bước tiệm cận các khía cạnh nhận thức của AGI nhưng vẫn chưa đạt tới mục tiêu này.

• Siêu trí tuệ nhân tạo (Artificial Superintelligence – ASI): Khái niệm giả thuyết về một hệ thống AI sở hữu năng lực trí tuệ vượt xa những bộ não xuất sắc nhất của nhân loại ở mọi lĩnh vực khoa học, sáng tạo nghệ thuật và kỹ năng xã hội.

1.5.2. Hai trường phái tư tưởng lớn trong AI

• Trường phái Biểu tượng (Symbolic AI / Good Old-Fashioned AI – GOFAI): Tiếp cận theo hướng ”từ trên xuống” (Top-down). Trí tuệ được mô hình hóa bằng các biểu tượng rõ ràng và các phép toán logic hình thức. Ưu điểm là tính minh

bạch và suy luận chặt chẽ; nhược điểm là không thể xử lý tốt sự nhiễu loạn và tính liên tục của thế giới thực.

• Trường phái Kết nối (Connectionist AI / Neural Networks): Tiếp cận theo hướng ”từ dưới lên” (Bottom-up). Trí tuệ nảy sinh từ sự tương tác phức tạp của hàng triệu nơ-ron đơn giản kết nối với nhau, học hỏi trực tiếp từ mẫu dữ liệu số. Ưu điểm là khả năng thích ứng với dữ liệu phi cấu trúc; nhược điểm là tính chất ”hộp đen” khó diễn giải tường minh.

1.6. Xu hướng AI tại Việt Nam và Những thách thức đương đại

1.6.1. Bức tranh phát triển AI tại Việt Nam

Việt Nam đang có những bước phát triển trong nghiên cứu và ứng dụng trí tuệ nhân tạo:

• Khung chính sách chiến lược: Ban hành Quyết định 127/QĐ-TTg ngày 26/01/2021 về Chiến lược quốc gia về nghiên cứu, phát triển và ứng dụng AI đến năm 2030, định vị AI là công nghệ nền tảng thúc đẩy chuyển đổi số quốc gia.

• Trung tâm nghiên cứu & Doanh nghiệp tiên phong: Sự ra đời của các viện nghiên cứu đẳng cấp quốc tế như VinAI Research, Viettel AI, FPT AI Center, Zalo AI Lab... đã đóng góp nhiều công trình công bố tại các hội nghị đỉnh cao (NeurIPS, CVPR, ICML) và thương mại hóa các giải pháp trợ lý ảo tiếng Việt, xe tự hành, camera thông minh.

• Ứng dụng thực tiễn: Tích hợp AI vào hành chính công, chẩn đoán y tế từ xa, nông nghiệp công nghệ cao và phòng chống thiên tai.

1.6.2. Những thách thức cốt lõi đặt ra

1. Tính chất ”Hộp đen” & Khả năng giải thích (Explainable AI – XAI): Các mạng nơ-ron sâu đưa ra dự báo chính xác nhưng con người khó truy xuất lý do tại sao, gây rủi ro lớn trong các lĩnh vực nhạy cảm như chẩn đoán ung thư, phê duyệt tín dụng hay tư pháp.

2. Thiên lệch dữ liệu & Tính công bằng (Data Bias & Fairness): Mô hình học từ dữ liệu lịch sử của con người nên có nguy cơ phản ánh và khuếch đại các định kiến xã hội về giới tính, chủng tộc hoặc địa lý.

3. Độ tin cậy & Hiện tượng ”Ảo giác” (Hallucination): Các mô hình ngôn ngữ lớn có xu hướng sinh ra thông tin sai lệch nhưng với giọng điệu vô cùng tự tin, gây nguy hại cho việc tiếp nhận tri thức chính xác.

4. Chi phí tính toán & Tác động môi trường (Compute Cost & Green AI): Huấn luyện một mô hình nền tảng hiện đại tiêu tốn hàng chục triệu USD và năng lượng đáng kể, phụ thuộc quy mô và phần cứng, đặt ra bài toán về tính bền vững và sự bất bình đẳng công nghệ giữa các quốc gia.

5. Quyền riêng tư dữ liệu & An toàn bảo mật (Privacy & Adversarial Robustness): Nguy cơ rò rỉ dữ liệu cá nhân khi huấn luyện mô hình và khả năng mô hình bị đánh lừa bởi các cuộc tấn công đối nghịch (Adversarial Attacks).

1.7. Mối liên hệ giữa các giai đoạn phát triển

AI biểu tượng cung cấp cách biểu diễn tri thức và suy luận có thể kiểm tra. Học máy thống kê chuyển trọng tâm sang việc học quy luật từ dữ liệu; học sâu tiếp tục học biểu diễn thay cho nhiều bước thiết kế đặc trưng thủ công. Transformer cho phép mô hình hóa quan hệ giữa các vị trí qua attention và huấn luyện song song hơn RNN. KG và RAG kết nối mô hình dữ liệu với tri thức tường minh; chúng kế thừa các ý tưởng cũ thay vì thay thế toàn bộ AI biểu tượng.

Trong báo cáo này, mối liên hệ đó được thể hiện bằng ba loại đầu vào: bảng thuộc tính, ảnh và chuỗi thời gian. Các mô hình tạo ra dự đoán định lượng; Chương 5 bổ sung bằng chứng và ngữ cảnh để người dùng hiểu phạm vi của các kết quả.

1.7.1. Từ biểu diễn tri thức đến biểu diễn học được

Với một bộ luật, người thiết kế xác định trước thuộc tính và điều kiện suy luận. Cách làm này thuận lợi khi quy tắc rõ và miền tri thức ổn định, nhưng số trường hợp cần mô tả tăng nhanh khi đầu vào đa dạng. Học máy chuyển một phần công việc sang dữ liệu: mô hình ước lượng quan hệ giữa các đặc trưng và nhãn. Học sâu tiếp tục thay đổi ở cấp biểu diễn, chẳng hạn tự học kernel từ pixel thay vì yêu cầu liệt kê thủ công mọi nét đặc trưng của chữ số.

Sự chuyển dịch không có nghĩa tri thức tường minh mất vai trò. Trong bài toán sức khỏe, quy định về đơn vị, phạm vi sử dụng và cảnh báo không được suy ra chỉ từ nhãn của một tập dữ liệu tổng hợp. Trong bài toán thị trường, quan hệ giữa mã tài sản, loại dữ liệu và chỉ báo cần được ghi rõ để tránh gọi vàng futures thành vàng giao ngay. Những ràng buộc này nằm ngoài một giá trị dự đoán đơn lẻ và phù hợp với biểu diễn có cấu trúc.

1.7.2. Tiêu chí lựa chọn công nghệ trong hệ thống

Bảng 1.3. Liên hệ giữa đặc điểm đầu vào và lựa chọn kỹ thuật

Dạng đầu vào	Vấn đề cần giải quyết	Lựa chọn trong project
Bảng thuộc tính	Giá trị thiếu, quan hệ phi tuyến, lớp hiếm	Pipeline tiền xử lý và Random Forest
Ảnh xám	Tương quan cục bộ, nét xuất hiện ở nhiều vị trí	CNN có kernel dùng chung
Chuỗi theo phiên	Thông tin quá khứ và phụ thuộc thứ tự	RNN/LSTM/GRU; đối chiếu Persistence
Câu hỏi và tri thức	Tìm bằng chứng phù hợp, giữ quan hệ thực thể	TF-IDF/BM25 và Knowledge Graph

Các chương sau dùng cùng nguyên tắc kiểm tra: xác định đúng bài toán trước, xây mốc so sánh đơn giản, khóa phần dữ liệu dùng đánh giá và chỉ sau đó mới tăng độ phức tạp. Một mạng sâu không được ưu tiên chỉ vì có nhiều tham số; một câu trả lời không được coi là đúng chỉ vì có điểm truy xuất cao. Giá trị của hệ thống được xem xét đồng thời ở sai số, khả năng kiểm tra và sự phù hợp với nhu cầu sử dụng.

1.8. Tiểu kết chương 1

Lịch sử AI thể hiện sự chuyển đổi giữa nhiều hướng tiếp cận: từ các nền tảng lý thuyết của Alan Turing và Perceptron, qua những giai đoạn suy giảm đầu tư, đến học máy thống kê và học sâu. Các bước tiến gắn với thay đổi về dữ liệu, năng lực tính toán và phương pháp học.

Không có một mô hình đơn lẻ phù hợp với mọi bài toán. Việc lựa chọn cần dựa trên cấu trúc dữ liệu, mục tiêu đánh giá và khả năng kiểm tra kết quả, từ các kỹ thuật học máy cho bảng thuộc tính đến kiến trúc chuyên biệt cho ảnh và chuỗi.

Chương tiếp theo cụ thể hóa các nguyên tắc trên bằng hai bài toán dữ liệu bảng, trong đó phép chia dữ liệu và lựa chọn thước đo được xác định trước khi so sánh mô hình.

Chương 2

CÁC KỸ THUẬT MACHINE LEARNING CƠ BẢN

Trong nhiều ứng dụng, thông tin đầu vào không phải ảnh hay văn bản dài mà là một bảng thuộc tính. Vấn đề đặt ra là chọn những biến nào, xử lý giá trị thiếu ra sao và đánh giá thế nào để kết quả không chỉ đúng trên các bản ghi đã thấy. Một quy trình học máy thiếu kiểm soát phép chia dữ liệu có thể tạo điểm số cao nhưng không sử dụng được cho mẫu mới.

Hai bài toán được triển khai trong chương là phân loại nhãn nguy cơ tổng hợp và ước lượng giá nhà từ thông tin tin đăng. Bài toán thứ nhất có lớp hiếm nên cần xem đồng thời precision, recall và macro-F1; bài toán thứ hai có nhiều bản ghi cùng địa chỉ nên cần chia theo nhóm. Đây là hai tình huống giúp làm rõ quan hệ giữa đặc điểm dữ liệu và thiết kế thực nghiệm.

Sau phần nền tảng, chương lần lượt xác định đầu vào và đầu ra, kiểm tra dữ liệu, xây pipeline tiền xử lý, so sánh mô hình bằng kiểm định chéo và phân tích lỗi trên tập test giữ riêng. Cuối chương trình bày cách sử dụng từng mô hình trên web và giới hạn của kết quả, thay vì coi giao diện dự đoán là bằng chứng cho độ chính xác.

2.1. Cơ sở lý thuyết Học máy (Machine Learning Foundations)

2.1.1. Khái niệm và Các dạng học máy

Năm 1997, Tom Mitchell đã đưa ra định nghĩa toán học chuẩn tắc về học máy:

”Một chương trình máy tính được coi là học từ kinh nghiệm E đối với một lớp tác vụ T và thước đo hiệu năng P, nếu hiệu năng của nó đối với các tác vụ trong T, được đo bằng P, cải thiện theo kinh nghiệm E.”

Dựa vào bản chất của kinh nghiệm E (dữ liệu đầu vào), học máy được chia thành 4 dạng thức chính:

1. Học có giám sát (Supervised Learning):

   • Dữ liệu huấn luyện bao gồm cặp mẫu đầu vào và nhãn mục tiêu tương ứng:

$$D = \{(x_i, y_i)\}_{i=1}^{N} \quad (2.1)$$

   • Mục tiêu: Tìm hàm ánh xạ từ không gian đầu vào sang không gian đầu ra sao cho dự báo xấp xỉ nhãn thực tế.

$f : \mathcal{X} \to \mathcal{Y}, \qquad \hat{y} = f(x)$ (2.2)

• Hồi quy (Regression): Khi nhãn y thuộc tập số thực, đầu ra là giá trị liên tục (ví dụ: dự báo giá nhà, nhiệt độ, doanh thu).

• Phân loại (Classification): Khi nhãn là các nhãn lớp rời rạc (ví dụ: chẩn đoán bệnh tật, phân loại email rác).

2. Học không giám sát (Unsupervised Learning):

• Dữ liệu huấn luyện chỉ bao gồm các mẫu đầu vào mà không có nhãn đích:

$D = \{x_i\}_{i=1}^{N}$ (2.3)

• Mục tiêu: Tự động khám phá cấu trúc ẩn sâu bên trong dữ liệu.

• Phân cụm (Clustering): Gom các điểm dữ liệu có đặc trưng tương đồng vào cùng nhóm (K-Means, DBSCAN).

• Giảm chiều dữ liệu (Dimensionality Reduction): Nén không gian dữ liệu nhiều chiều xuống không gian ít chiều hơn mà vẫn bảo toàn tối đa thông tin quan trọng (PCA, t-SNE).

3. Học bán giám sát & Tự giám sát (Semi-supervised & Self-supervised Learning):

• Học bán giám sát: Kết hợp một lượng nhỏ dữ liệu có gán nhãn với một khối lượng khổng lồ dữ liệu không gán nhãn để tiết kiệm chi phí dán nhãn thủ công.

• Học tự giám sát: Mô hình tự động tạo ra nhãn từ chính cấu trúc nội tại của dữ liệu (ví dụ: che giấu một từ trong câu và bắt mô hình dự đoán từ bị che, tạo tiền đề cho các mô hình ngôn ngữ lớn LLM).

4. Học tăng cường (Reinforcement Learning – RL):

• Tác tử (Agent) tương tác liên tục với môi trường (Environment) qua các trạng thái st, thực hiện hành động at và nhận lại tín hiệu phần thưởng rt

(Reward). Mục tiêu là tìm chính sách hành động π(a|s) tối đa hóa tổng phần thưởng tích lũy dài hạn.

2.1.2. Quy trình chuẩn xây dựng một mô hình Học máy

Một quy trình khoa học dữ liệu hoàn chỉnh bao gồm 5 giai đoạn liên hoàn khép kín:

[1. Thu thập dữ liệu] → [2. Tiền xử lý (EDA, Clean, Scale)] → [3. Phân chia Train/Val/Test] → [4. Huấn luyện & Tối ưu Siêu tham số] → [5. Đánh giá & Giám sát]

1. Thu thập và Khám phá Dữ liệu (EDA): Phân tích phân phối thống kê, phát hiện ngoại lai (Outliers), kiểm tra độ tương quan giữa các biến và nhận diện hiện tượng rò rỉ dữ liệu (Data Leakage).

2. Tiền xử lý Dữ liệu:

• Xử lý dữ liệu khuyết: Điền khuyết (Imputation) bằng giá trị trung bình (mean), trung vị (median) hoặc mô hình học máy.

• Mã hóa biến phân loại: One-Hot Encoding cho biến định danh không thứ bậc, Label Encoding cho biến có thứ bậc.

• Chuẩn hóa thang đo: Đưa các thuộc tính về cùng một miền giá trị bằng Standard Scaling hoặc Min-Max Scaling nhằm giúp các thuật toán hạ gradient hội tụ nhanh và ổn định.

$z = \frac{x-\mu}{\sigma}, \qquad x' = \frac{x-x_{\min}}{x_{\max}-x_{\min}}$ (2.4)

x là giá trị gốc; μ, σ, x_min và x_max lần lượt là trung bình, độ lệch chuẩn và hai biên của đặc trưng. Các đại lượng này chỉ được ước lượng từ train, rồi giữ cố định khi biến đổi validation/test.

3. Phân chia tập dữ liệu: Chia tách độc lập thành tập Huấn luyện (Train set), tập Kiểm định (Validation set) và tập Kiểm thử (Test set). Áp dụng kỹ thuật Kiểm định chéo K-lần (K-Fold Cross-Validation) để đánh giá độ ổn định của mô hình.

4. Huấn luyện và Tối ưu hóa: Lựa chọn kiến trúc thuật toán phù hợp, định nghĩa hàm mục tiêu (Loss function) và tinh chỉnh siêu tham số (Hyperparameter Tuning).

5. Đánh giá và Kiểm định độc lập: Đo lường hiệu năng trên tập Test chưa từng nhìn thấy bằng các thước đo chuẩn mực.

2.1.3. Phương pháp Tối ưu hóa bằng Hạ Gradient (Gradient Descent)

Hạ gradient là thuật toán tối ưu hóa nền tảng trong toàn bộ học máy và học sâu, nhằm tìm bộ tham số w làm cực tiểu hóa hàm mất mát L(w).

Nguyên lý toán học: Vector gradient ∇L(w) chỉ hướng tăng nhanh nhất của hàm số tại điểm khảo sát. Do đó, để làm giảm hàm mất mát nhanh nhất, ta cập nhật tham số ngược hướng với vector gradient:

$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$ (2.5)

Trong đó η> 0 là tốc độ học (Learning Rate). Nếu quá nhỏ, mô hình hội tụ rất chậm chạp; nếu η quá lớn, mô hình sẽ dao động phân kỳ và không thể đạt tới điểm cực tiểu.

Các biến thể của Gradient Descent:

• Batch Gradient Descent: Tính toán gradient trên toàn bộ mẫu dữ liệu của tập huấn luyện. Hướng đi rất ổn định nhưng rất tốn bộ nhớ và chậm chạp khi dữ liệu lớn.

• Stochastic Gradient Descent (SGD): Tính toán gradient và cập nhật tham số trên từng mẫu đơn lẻ (N = 1). Rất nhanh và có khả năng thoát khỏi các điểm cực tiểu địa phương cạn, nhưng dao động gradient rất mạnh (nhiễu).

• Mini-batch Gradient Descent: Cân bằng giữa hai thái cực trên bằng cách chia dữ liệu thành từng nhóm nhỏ kích thước B (thường là 32, 64, 128 mẫu). Đây là cách làm phổ biến trong huấn luyện mạng nơ-ron hiện đại.

2.1.4. Hồi quy tuyến tính (Linear Regression)

Mô hình hồi quy tuyến tính giả thiết mối quan hệ giữa biến đầu vào x= [x1, x2,..., xd]T và biến mục tiêu liên tục y là tuyến tính:

$\hat{y} = w^T x + b = \sum_{j=1}^{d} w_j x_j + b$ (2.6)

Hàm mất mát — Sai số bình phương trung bình (Mean Squared Error – MSE):

$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$ (2.7)

Đạo hàm và Cập nhật trọng số: Đạo hàm riêng theo trọng số w:

$\frac{\partial \mathcal{L}}{\partial w} = \frac{2}{N} X^T (\hat{y} - y)$ (2.8)

Cập nhật tham số qua gradient descent:

$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$ (2.9)

2.1.5. Hồi quy Logistic (Logistic Regression & Softmax)

Dành cho bài toán phân loại. Dù mang tên ”hồi quy”, mô hình này thực chất là một bộ phân loại tuyến tính xác suất.

Phân loại nhị phân (Binary Classification): Để ép giá trị đầu ra tuyến tính z= wTx+ b về khoảng xác suất, mô hình sử dụng hàm kích hoạt Sigmoid:

$\sigma(z) = \frac{1}{1+e^{-z}}$ (2.10)

Hàm mất mát — Binary Cross-Entropy (BCE / Log Loss):

$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$ (2.11)

Phân loại đa lớp (Multiclass Classification — Softmax Regression): Khi có K lớp phân loại rời rạc, đầu ra tại lớp thứ k được chuẩn hóa qua hàm Softmax:

$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$ (2.12)

Hàm mất mát tương ứng là Categorical Cross-Entropy:

$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} y_{ik} \log p_{ik}$ (2.13)

2.1.6. Tổng quan các thuật toán Học máy kinh điển tiêu biểu

Bên cạnh các mô hình tuyến tính, học máy cổ điển sở hữu các thuật toán phi tuyến xuất sắc:

1. Cây quyết định (Decision Tree):

• Phân chia không gian dữ liệu bằng các câu hỏi điều kiện phân nhánh nhị phân trực quan.

• Tiêu chuẩn phân chia tối ưu: sử dụng độ lợi thông tin (Information Gain) dựa trên Entropy hoặc chỉ số Gini Impurity.

$$H(S) = -\sum_{k=1}^{K} p_k \log_2 p_k, \qquad G(S) = 1 - \sum_{k=1}^{K} p_k^2 \quad (2.14)$$

pk là tỷ lệ mẫu thuộc lớp k ở nút đang xét. Độ lợi thông tin là độ giảm entropy có trọng số sau khi chia. Một phép chia có độ lợi cao tạo các nút con đồng nhất hơn về nhãn.

• Ưu điểm: Rất dễ giải thích, không yêu cầu chuẩn hóa thang đo; Nhược điểm: Dễ bị quá khớp (Overfitting) nếu cây phát triển quá sâu.

2. Phương pháp Tổ hợp (Ensemble Methods):

• Rừng ngẫu nhiên (Random Forest): Kỹ thuật đóng gói (Bagging) kết hợp dự báo của hàng trăm cây quyết định độc lập được huấn luyện trên các tập mẫu con lấy mẫu có hoàn lại (Bootstrap) và tập thuộc tính ngẫu nhiên. Giảm phương sai, rất bền bỉ và chính xác trên dữ liệu dạng bảng.

• Gradient Boosting (GBM / XGBoost / LightGBM): Kỹ thuật tăng cường (Boosting) xây dựng các cây quyết định tuần tự, trong đó mỗi cây sau tập trung sửa chữa sai số (Residuals) của các cây trước đó. Thường đạt độ chính xác cao nhất trong các cuộc thi dữ liệu bảng.

3. Máy vector hỗ trợ (Support Vector Machine – SVM):

• Tìm kiếm siêu phẳng phân cách tối ưu sao cho khoảng cách (lề - margin) từ siêu phẳng tới các điểm dữ liệu gần nhất của hai lớp (gọi là các Support Vectors) là cực đại.

• Kernel Trick: Ánh xạ dữ liệu không phân tách tuyến tính từ không gian gốc lên không gian nhiều chiều hơn thông qua các hàm nhân (RBF Kernel, Polynomial Kernel) để phân tách tuyến tính mà không cần tính toán tọa độ rõ ràng.

4. K láng giềng gần nhất (K-Nearest Neighbors – KNN):

• Giải thuật học lười biếng (Lazy Learning): Không có pha huấn luyện tường minh. Khi dự báo mẫu mới, thuật toán tính khoảng cách (Euclidean, Manhattan) tới toàn bộ các điểm trong tập dữ liệu và lấy biểu quyết đa số từ K điểm láng giềng gần nhất.

Bảng 2.1. So sánh các thuật toán học máy kinh điển

Thuật toán	Bản chất	Độ phức tạp	Giải thích	Ưu điểm	Hạn chế
Logistic Regression	Tuyến tính + Sigmoid	Dự đoán O(d)	Hệ số trọng số	Đơn giản, nhanh; xuất xác suất	Biên phân cách tuyến tính
Decision Tree	Cắt lát không gian	Phụ thuộc độ sâu	Cây luật NẾU–THÌ	Trực quan; ít cần scaling	Dễ quá khớp, nhạy nhiễu
Random Forest	Bagging nhiều cây	Phụ thuộc số cây, N, d và độ sâu	Feature importance	Bền bỉ, giảm phương sai	Mô hình lớn hơn, dự báo chậm hơn
Gradient Boosting	Boosting sửa sai số	Phụ thuộc số vòng và độ sâu	Trung bình	Mạnh với dữ liệu bảng	Nhạy siêu tham số, tốn thời gian
SVM	Cực đại hóa margin	Phụ thuộc kernel và số mẫu	Thấp với kernel	Mạnh với ít mẫu, nhiều chiều	Chậm trên tập lớn
KNN	Khoảng cách mẫu	Brute-force O(Nd) mỗi truy vấn	Láng giềng	Trực quan, không giả định phân phối	Tốn bộ nhớ, dự báo chậm

2.1.7. Hiện tượng Overfitting, Underfitting & Kỹ thuật Điều chuẩn

Sự đánh đổi giữa Độ chệch và Phương sai (Bias-Variance Trade-off): Tổng sai số kỳ vọng của một mô hình học máy được phân rã thành 3 thành phần:

$$E[(y - \hat{f}(x))^2] = Bias[\hat{f}(x)]^2 + Var[\hat{f}(x)] + \sigma^2 \quad (2.15)$$

• Thiếu khớp (Underfitting - High Bias): Mô hình quá đơn giản, không học đủ quy luật từ dữ liệu, sai số trên cả tập Train và Test đều cao.

• Quá khớp (Overfitting - High Variance): Mô hình quá phức tạp, ghi nhớ cả các nhiễu ngẫu nhiên của tập Train, dẫn tới sai số tập Train cực thấp nhưng sai số tập Test vọt lên rất cao.

Các kỹ thuật Điều chuẩn (Regularization): Điều chuẩn là kỹ thuật thêm một số hạng phạt vào hàm mất mát nhằm hạn chế độ lớn của các trọng số, ép mô hình đơn giản hơn:

• Điều chuẩn L2 (Ridge Regression / Weight Decay): Phạt tổng bình phương trọng số. Thu nhỏ trọng số về gần 0 nhưng không triệt tiêu hoàn toàn.

$$\mathcal{L}_{reg} = \mathcal{L} + \lambda \sum_i w_i^2 \quad (2.16)$$

• Điều chuẩn L1 (Lasso Regression): Phạt tổng giá trị tuyệt đối của trọng số. Có khả năng triệt tiêu trọng số về chính xác bằng 0, đóng vai trò chọn lọc đặc trưng tự động.

$\mathcal{L}_{reg} = \mathcal{L} + \lambda \sum_i |w_i|$ (2.17)

• Trong Mạng Nơ-ron: Áp dụng kỹ thuật Dropout (ngẫu nhiên ngắt kết nối một tỉ lệ nơ-ron trong quá trình train để chống đồng thích nghi) và Early Stopping (dừng huấn luyện khi sai số trên tập Validation bắt đầu tăng liên tục).

2.1.8. Từ Học máy truyền thống đến Mạng Nơ-ron Sâu (Deep Neural Network – DNN)

Khi dữ liệu có độ phức tạp cao và phi tuyến mạnh mẽ, việc kết hợp các tầng biến đổi tuyến tính xen kẽ các hàm kích hoạt phi tuyến dẫn tới kiến trúc Mạng nơ-ron sâu (DNN).

Có thể minh họa lý thuyết bằng một kiến trúc Mạng nơ-ron sâu 5 lớp (gồm 4 lớp ẩn Dense liên kết đầy đủ + 1 lớp đầu ra):

Input (X) ⟶ [Dense 1 + ReLU] ⟶ [Dense 2 + ReLU] ⟶ [Dense 3 + ReLU] ⟶ [Dense 4 + ReLU] ⟶ Output Layer (ŷ)

1. Lan truyền xuôi (Forward Propagation): Tại mỗi tầng ẩn $l \in \{1,2,3,4\}$:

$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = ReLU(z^{(l)})$ (2.18)

Tại tầng đầu ra l = 5:

• Với bài toán hồi quy (Housing Price): ŷ = z(5)(kích hoạt tuyến tính).

• Với bài toán phân loại đa lớp: ŷ = Softmax(z(5)).

2. Lan truyền ngược (Backpropagation theo Chain Rule): Để cập nhật ma trận trọng số, ta tính toán gradient sai số từ tầng cuối cùng lội ngược về tầng đầu theo quy tắc chuỗi giải tích:

$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$ (2.19)

$\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot ReLU'(z^{(l)})$ (2.20)

Trong đó δ là sai số lan truyền ngược. Quy tắc chuỗi cho phép truyền gradient qua các lớp; phần thực nghiệm mới sử dụng các mô hình được mô tả riêng bên dưới.

2.1.9. Kiểm định chéo và lựa chọn siêu tham số

Train dùng để ước lượng tham số; validation dùng để kiểm tra các lựa chọn phát triển và check-point; test chỉ dùng cho đánh giá cuối. Trong phân loại, StratifiedKFold giữ tỷ lệ lớp trong từng fold. Với giá nhà, GroupKFold giữ các bản ghi cùng địa chỉ trong một fold. Toàn bộ imputation, scaling và one-hot encoding được fit bên trong từng fold, không fit trên toàn bộ dữ liệu trước khi chia.

Grid search so sánh tập siêu tham số hữu hạn đã định trước. Dùng macro-F1 khi các lớp mất cân bằng và MAE theo tỷ VND khi dự báo giá nhà. Không lựa chọn mô hình bằng chỉ số test. Khoảng cách train–validation–test giúp phát hiện quá khớp; một lần chia và một random seed chưa mô tả hết độ bất định.

2.1.10. Các thước đo đánh giá

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \quad MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \text{ (2.21)}$$

$$RMSE = \sqrt{MSE}, \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \text{ (2.22)}$$

Trong (2.21)–(2.22), n là số mẫu, yi là giá trị thực, ŷi là dự đoán và ȳ là trung bình của nhãn trong tập đánh giá. MAE và RMSE có cùng đơn vị với nhãn, MSE có đơn vị bình phương. R2 so với dự đoán hằng bằng trung bình, có thể âm và không phải phần trăm chính xác.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}, \quad P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN} \text{ (2.23)}$$

$$F_1 = \frac{2PR}{P+R}, \qquad F_{1,macro} = \frac{1}{K}\sum_{k=1}^{K} F_{1,k} \text{ (2.24)}$$

TP, TN, FP và FN lần lượt là số dự đoán dương đúng, âm đúng, dương sai và âm sai trong cách xét một lớp so với các lớp còn lại. Với K lớp, macro-F1 cho mỗi lớp trọng số bằng nhau, vì vậy thể hiện rõ hơn hiệu năng ở lớp ít mẫu.

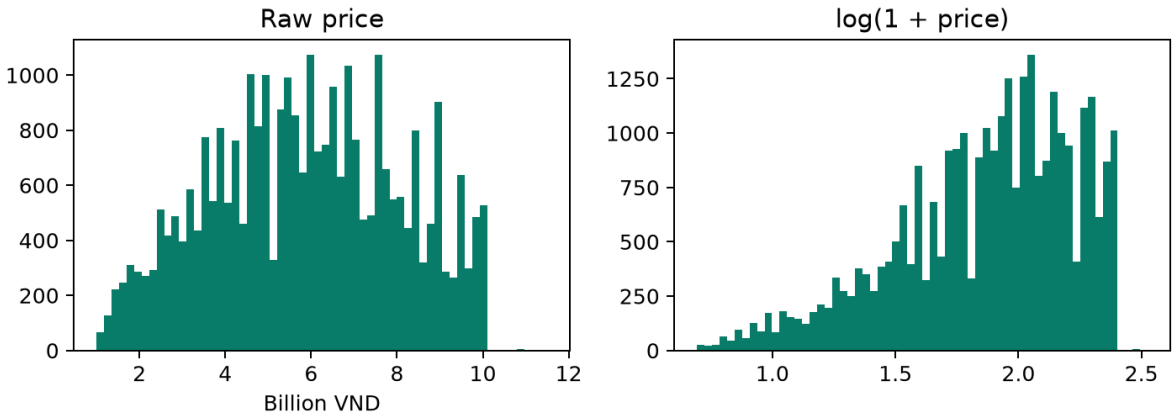
2.2. Phân tích và chuẩn bị dữ liệu thực nghiệm

Bảng 2.2. Dữ liệu của các thực nghiệm tự xây dựng lại

Dataset	Quy mô	Mục tiêu	Train / Val / Test
Diabetes Risk Prediction	50.000 dòng; 41 cột gồm cả nhãn	Ba nhãn High, Low, Moderate	32000 / 8000 / 10000
Vietnam Housing 2024	30.229 dòng; 12 cột gốc	Giá nhà theo tỷ VND	19377 / 4491 / 6361

Diabetes Risk Prediction là dữ liệu tổng hợp phục vụ học máy, không được coi là 50.000 hồ sơ bệnh nhân đã xác minh lâm sàng. Phân phối thực tế gồm High 36.593 mẫu (73,2%), Moderate 12.937 mẫu (25,9%) và Low 470 mẫu (0,9%). Mất cân bằng giữa ba lớp cần được tính đến khi chọn mô hình và đọc chỉ số. Tám đầu vào gồm Age, BMI, HbA1c, Fasting_Blood_Sugar, Blood_Glucose, Blood_Pressure_Systolic, HDL và LDL. Patient_ID, Diabetes_Risk_Score, AI_Health_Recommendation và Doctor_Consultation_Needed không được sử dụng vì là định danh hoặc có thể tiết lộ nhãn.

Dữ liệu giá nhà có các trường Area, Frontage, Access Road, Floors, Bedrooms, Bathrooms cùng hướng nhà, hướng ban công, pháp lý và nội thất. Province được trích từ phần cuối địa chỉ. Pipeline điền thiếu số bằng trung vị train, mã hóa one-hot với nhóm chưa gặp được bỏ qua, và học log(1 + Price) rồi biến đổi ngược trước khi tính sai số. Các bản ghi cùng địa chỉ được giữ trong cùng một tập. Đây là kiểm tra tổng quát hóa sang địa chỉ khác, chưa phải backtest giá nhà theo thời gian.



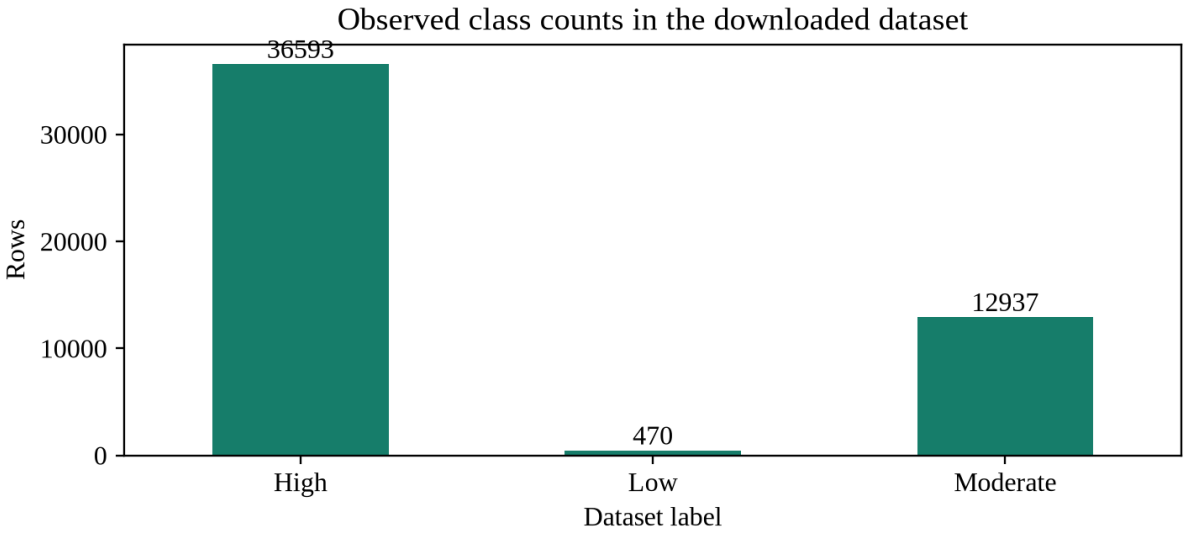
Hình 2.1. Phân phối giá nhà trước và sau biến đổi logarit

2.2.1. Định nghĩa mẫu, nhãn và đơn vị sử dụng

Ở bài toán phân loại, một mẫu là một dòng dữ liệu và đầu ra là một trong ba nhãn High, Moderate hoặc Low do dataset cung cấp. Chỉ tám biến số được sử dụng nhằm tạo một đầu vào có thể kiểm soát trên giao diện. Việc bỏ các biến chứa điểm nguy cơ hoặc khuyến nghị giúp tránh để mô hình học lại một thông tin đã quá gần nhãn. Đây là lựa chọn chống rò rỉ ở cấp ngữ nghĩa, khác với việc xóa cột vì hệ số tương quan cao.

Bảng 2.3. Từ điển đầu vào classifier và giá trị thiếu thực tế

Biến	**Ý nghĩa / đơn vị**	**Số thiếu trong 50.000 dòng**
Age	Tuổi	497
BMI	Chỉ số khối cơ thể	0
HbA1c	Phần trăm	1974
Fasting_Blood_Sugar	Đường huyết lúc đói; mg/dL	0
Blood_Glucose	Glucose; mg/dL	959
Blood_Pressure_Systolic	Huyết áp tâm thu; mmHg	0
HDL / LDL	mg/dL	1.000 ở mỗi biến



Hình 2.2. Số lượng ba nhãn đếm trực tiếp từ tệp dữ liệu tải về

Nếu dự đoán mọi mẫu đều là High, Accuracy sẽ gần tỷ lệ của lớp lớn nhất nhưng hệ thống không nhận diện được hai lớp còn lại. Bởi vậy phép so sánh dùng macro-F1, cho mỗi lớp trọng số bằng nhau. class_weight=balanced tăng đóng góp của lớp hiếm trong quá trình học; nó không tự biến điểm số của Random Forest thành xác suất nguy cơ lâm sàng đã được hiệu chỉnh.

2.2.2. Dữ liệu nhà ở và ý nghĩa của phép chia theo địa chỉ

Một mẫu nhà ở là một tin đăng với giá rao bán, không phải biên bản giao dịch. Nhãn Price được giữ ở đơn vị tỷ VND. Các biến diện tích, mặt tiền, đường vào, tầng, phòng và nhóm thông tin pháp lý/nội thất mô tả một phần tài sản; biến Province chỉ lấy thành phần cuối của chuỗi địa chỉ. Cách trích này đơn giản và tái lập được, nhưng chưa phải chuẩn hóa địa giới hay mã hóa tọa độ địa lý.

Nhiều tin có thể cùng địa chỉ. Nếu chia theo dòng, mô hình có thể thấy những bản ghi rất gần nhau trong cả train lẫn test. Project chuẩn hóa địa chỉ bằng chữ thường và bỏ khoảng trắng hai đầu, rồi dùng địa chỉ làm nhóm cho phép chia và GroupKFold. Kiểm tra sau chia cho kết quả 0 nhóm chung ở cả ba cặp train–validation, train–test và validation–test. Điều này giảm một nguồn rò rỉ, nhưng chưa phát hiện được hai cách viết khác nhau của cùng địa chỉ.

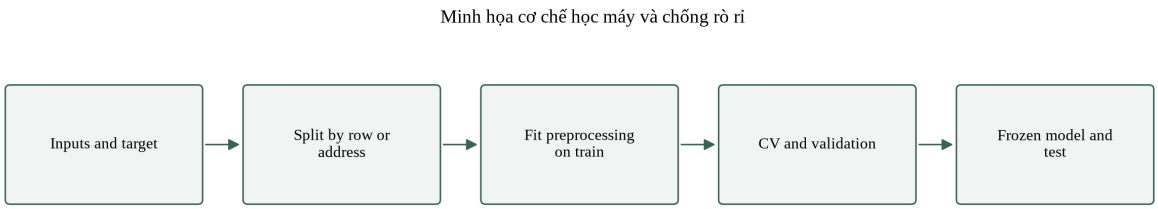
Bảng 2.4. Đối chiếu số dòng và số nhóm sau chia dữ liệu nhà ở

Tập	Số dòng	Số nhóm địa chỉ
train	19377	6560
validation	4491	1640
test	6361	2051

Mặt tiền thiếu khoảng 38,3%, đường vào 44,0%, hướng nhà 70,3% và hướng ban công 82,6%. Điền trung vị cho biến số giữ được mẫu nhưng không khôi phục thông tin thật đã mất. Giá trị Unknown của biến phân loại cũng không đồng nghĩa một đặc điểm có thật của căn nhà.

Vì vậy báo cáo xem missingness là giới hạn dữ liệu, không chỉ là một thao tác kỹ thuật trước huấn luyện.

2.2.3. Pipeline và kiểm soát thông tin giữa các tập



Hình 2.3. Minh họa quy trình học máy; tiền xử lý chỉ được fit trên train

Trong mỗi fold, imputer tính trung vị từ phần train của fold; bộ chuẩn hóa tính trung bình và độ lệch chuẩn ở cùng phạm vi; one-hot encoder chỉ xây danh sách nhóm từ phần đó. Khi chuyển sang validation của fold, các tham số này được giữ nguyên. Sau khi chọn cấu hình, mô hình cuối được fit trên train, còn validation và test vẫn chỉ đi qua transform và predict. Việc đóng gói các bước trong Pipeline giúp quy tắc này được thực thi thống nhất thay vì phụ thuộc thao tác thủ công.

Random Forest không bắt buộc chuẩn hóa thang đo số, nhưng pipeline nhà ở vẫn chuẩn hóa để các ứng viên Ridge và mô hình cây dùng cùng định nghĩa tiền xử lý. log1p làm giảm mức chi phối của giá lớn trong quá trình học; expm1 đưa dự đoán về tỷ VND trước đánh giá. Tối ưu sai số trên thang log không đồng nghĩa tối ưu MAE trên thang giá, nên bước so sánh CV vẫn tính theo đơn vị gốc.

2.3. So sánh và lựa chọn mô hình

Bảng 2.5. So sánh phân loại với StratifiedKFold 3 phần

Mô hình	CV macro-F1
Majority	0.2817
Logistic Regression	0.5027
KNN	0.5055
Decision Tree	0.5610
Random Forest	0.6203
SVM RBF	0.5729

Random Forest được chọn bằng CV macro-F1 0.6203, gồm 80 cây, max_depth=10, class_weight=balanced, random_state=42. Macro-F1 train 0.7105, validation 0.6059 và test 0.6087 cho thấy vẫn có khoảng cách khái quát hóa. Không dùng test để tăng độ sâu cây.

Bảng 2.6. So sánh hồi quy với GroupKFold 3 phần

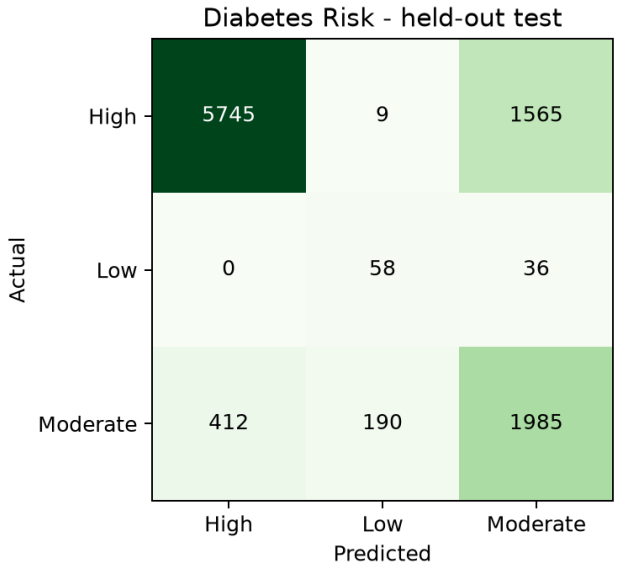
Mô hình	CV MAE (tỷ VND)
Median	1.8623
Log Ridge	1.4915
Log Random Forest	1.3091
Log Gradient Boosting	1.3246

Hồi quy chọn Log Random Forest với 80 cây, max_depth=14, min_samples_leaf=3. Ridge thử alpha 1, 10, 100; Gradient Boosting thử learning_rate 0,04 và 0,08. Tất cả cấu hình cùng một nhóm train và các fold theo địa chỉ. Mô hình trung vị cung cấp mốc đánh giá độ hữu ích của các mô hình học được.

2.4. Kết quả test độc lập và phân tích sai số

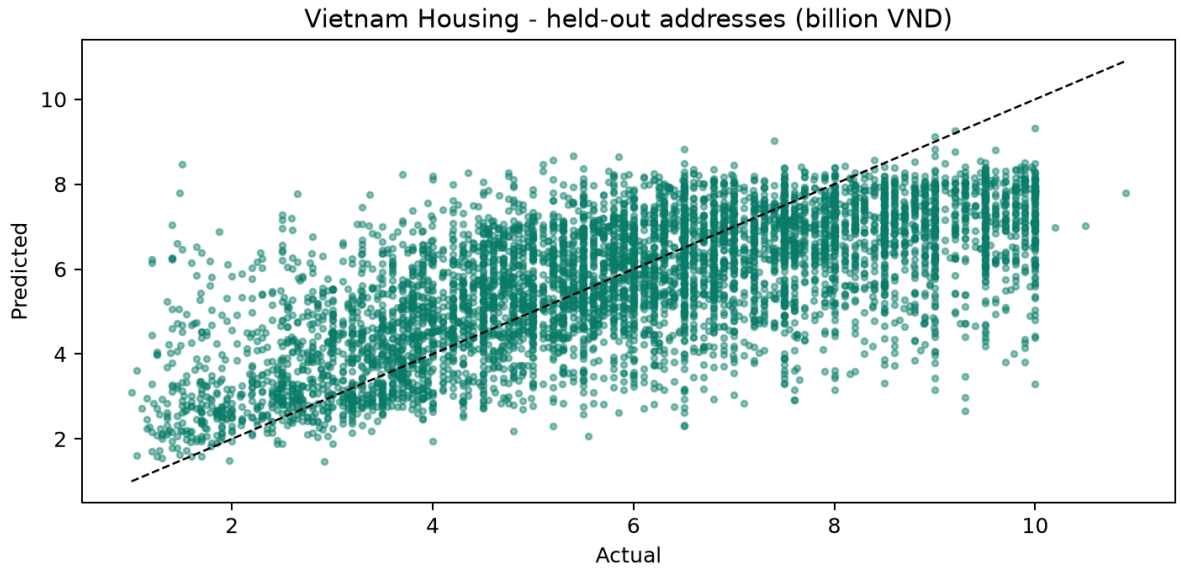
Bảng 2.7. Kết quả được tính từ predictions của main.ipynb

Bài toán	Chỉ số	Kết quả
Phân loại nguy cơ	Accuracy / Precision macro	0.7788 / 0.5708
Phân loại nguy cơ	Recall macro / F1 macro	0.7231 / 0.6087
Giá nhà	MAE / RMSE (tỷ VND)	1.2760 / 1.6394
Giá nhà	MSE / R2	2.6877 / 0.4273



Hình 2.4. Ma trận nhầm lẫn của phân loại nguy cơ trên 10.000 mẫu test

Ma trận nhầm lẫn cho thấy Accuracy chịu ảnh hưởng lớn của lớp High. Macro-precision thấp hơn macro-recall phản ánh chi phí dự đoán nhầm khi dùng class_weight=balanced. Nhãn dự đoán chỉ mô tả quy tắc của dataset, không phải chẩn đoán hoặc nguy cơ bệnh ngoài đời.



Hình 2.5. Giá nhà dự đoán và thực tế tại các địa chỉ giữ riêng

MAE train của giá nhà là 1.0276 tỷ VND, validation 1.2936, test 1.2760; R2 test 0.4273. Sai số tuyệt đối còn lớn đối với định giá cá nhân. Các thông tin vị trí trong địa chỉ, chất lượng tin đăng và thời điểm giao dịch chưa được mô hình hóa đầy đủ. Mức lệch giữa train và test cần được theo dõi trước khi tăng độ phức tạp.

2.4.1. Hiệu năng từng lớp và tác động của mất cân bằng

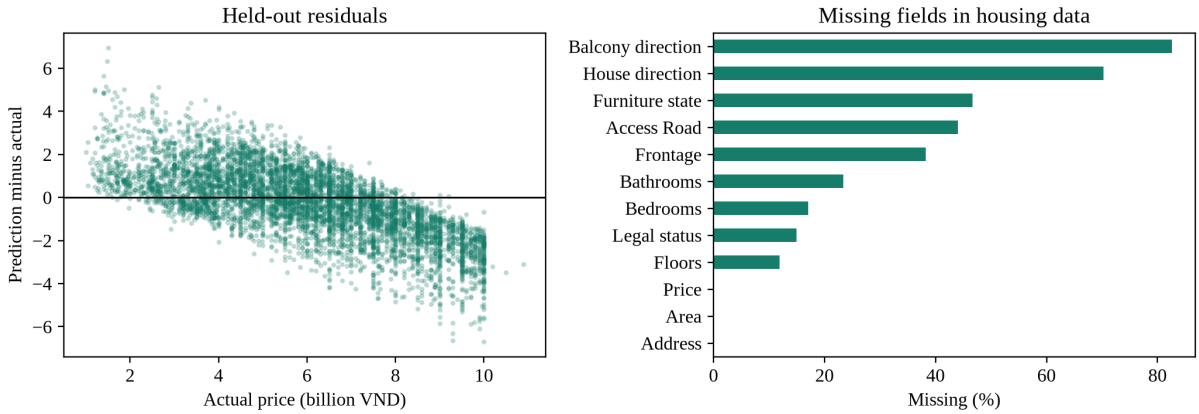
Bảng 2.8. Chỉ số từng lớp tính lại từ diabetes_predictions.csv

Lớp	Precision	Recall	F1	Mẫu test
High	0.9331	0.7849	0.8526	7319
Low	0.2257	0.6170	0.3305	94
Moderate	0.5535	0.7673	0.6431	2587

Lớp Low chỉ có 94 mẫu test. Recall 0,6170 nghĩa là mô hình tìm được khoảng 61,7% số mẫu Low thực tế, trong khi precision 0,2257 cho thấy nhiều dự đoán Low là nhầm từ lớp khác. Với High, precision 0,9331 cao hơn nhưng recall chỉ 0,7849. Nhìn riêng Accuracy 0,7788 sẽ che bớt sự khác nhau này. Kết quả phù hợp với tác động của cân bằng trọng số: tăng khả năng nhận diện lớp ít mẫu nhưng có thể tăng số dương tính giả theo cách xét từng lớp.

Chưa có tập bệnh nhân bên ngoài, nhãn lâm sàng độc lập hoặc đánh giá hiệu chỉnh xác suất. Do đó giao diện chỉ hiển thị nhãn dataset và giải thích phạm vi, không chuyển High thành chẩn đoán mắc bệnh, không đưa phần trăm nguy cơ cá nhân và không gắn khuyến nghị điều trị với đầu ra. Hướng cải tiến có cơ sở là xây dữ liệu xác minh và kiểm tra khả năng chuyển miền trước khi tăng số cây hoặc thêm một mạng sâu.

2.4.2. Sai số nhà ở theo phân khúc và phần dư



Hình 2.6. Phần dư test và tỷ lệ thiếu của dữ liệu nhà ở

Bảng 2.9. Phân tích test theo bốn nhóm lượng tử của giá thực

Khoảng giá thực (tỷ)	Mẫu	MAE (tỷ)	Lệch TB (tỷ)
(0.999, 4.3]	1607	1.197	+0.988
(4.3, 5.95]	1594	0.995	+0.420
(5.95, 7.56]	1570	0.967	-0.470
(7.56, 10.9]	1590	1.943	-1.932

Phần dư được định nghĩa bằng dự đoán trừ giá thực. Nhóm giá thấp có lệch trung bình dương, còn nhóm trên 7,56 tỷ có lệch khoảng −1,932 tỷ và MAE khoảng 1,943 tỷ. Mẫu hình này cho thấy dự đoán có xu hướng co về vùng trung tâm: nhà rẻ bị định giá cao, nhà đắt bị định giá thấp. Đây là mô tả từ test đã khóa, không phải căn cứ để sửa dự đoán theo chính giá thực của từng mẫu.

Trung vị sai số tuyệt đối là 1.038 tỷ VND; phân vị 90% là 2.688 tỷ VND. Hai đại lượng mô tả phân phối lỗi của tập test, không phải khoảng dự báo 90% cho một căn nhà mới. Để có khoảng dự báo cá nhân cần một quy trình hiệu chỉnh riêng và kiểm tra độ bao phủ ngoài mẫu.

2.4.3. Hai tình huống sử dụng trên web

Với giá nhà, người dùng nhập sáu biến số và chọn năm nhóm thuộc tính. Giá trị số trống được điền bằng trung vị train và API trả cảnh báo; nhóm chưa gặp được mã hóa thành vector 0 của nhóm tương ứng và cũng được cảnh báo. Phiên bản hoàn thiện dùng nhóm phổ biến nhất của train làm mặc định thay vì mục đầu danh sách chữ cái. Các giá trị mặc định tạo một kịch bản nhập liệu, không được giới thiệu là một tài sản quan sát thực tế.

Sau khi nhấn Chạy mô hình, API kiểm tra kiểu và tính hữu hạn, biến đổi đặc trưng bằng tham số train, chạy các cây đã xuất và trả giá theo tỷ VND. Một giá trị vượt min–max train không bị tự cắt về biên; hệ thống giữ đầu vào và đánh dấu ngoài phạm vi. Kết quả phù hợp để quan sát mô hình và đối chiếu kịch bản, chưa đủ cho quyết định mua bán.

Demo: https://rag-httm-chatbot.vercel.app/?demo=housing#experiments

Với classifier, giao diện yêu cầu đúng đơn vị đã ghi trong từ điển đầu vào. Giá trị thiếu được xử lý giống quy trình train; sai kiểu, số không hữu hạn hoặc giá trị không dương bị từ chối. Điểm theo lớp của rừng cây là kết quả tổng hợp của mô hình trên nhãn tổng hợp. Vì chưa hiệu chỉnh và chưa xác minh lâm sàng, phần hiển thị ưu tiên nhãn cùng cảnh báo thay vì đặt tên điểm số là xác suất mắc bệnh.

Demo: https://rag-httm-chatbot.vercel.app/?demo=diabetes_classifier#experiments

2.5. Kết luận chương

Với dữ liệu dạng bảng, Random Forest và Gradient Boosting thường cạnh tranh tốt với Deep Learning. Không có một mô hình đơn lẻ phù hợp với mọi tập dữ liệu.

Hai bài toán cho thấy điểm số phụ thuộc cách xác định nhãn, phép chia dữ liệu và thước đo. Macro-F1 giúp nhận rõ hạn chế ở lớp hiếm; chia theo địa chỉ giúp đánh giá giá nhà ngoài các nhóm đã gặp. Dù Random Forest được chọn trong cả hai bài toán, chất lượng đầu vào và miền sử dụng vẫn là giới hạn chính. Các tệp checkpoint, dự đoán và bảng CV được giữ để có thể kiểm tra lại từng kết luận.

Chương 3

MẠNG NƠ-RON TÍCH CHẬP CNN

Một chữ số viết tay có thể xuất hiện ở nhiều vị trí và có độ đậm, độ nghiêng khác nhau. Nếu coi mỗi pixel là một thuộc tính độc lập, mô hình phải học lại nhiều quan hệ giống nhau ở các vùng ảnh khác nhau. Bài toán của chương là nhận dạng chữ số đồng thời khai thác được cấu trúc không gian cục bộ của ảnh.

CNN giải quyết vấn đề bằng kernel dùng chung, các phép biến đổi phi tuyến và giảm kích thước không gian. Mạng được huấn luyện từ đầu trên MNIST để có thể theo dõi đầy đủ từ ảnh gốc đến trọng số, feature map và đầu ra. Mục tiêu không chỉ là đạt Accuracy cao mà còn kiểm tra xem phép tích chập trong mã có đúng với cơ chế đã giải thích và mô hình sai ở những trường hợp nào.

Chương đi từ cấu trúc tầng đến lan truyền gradient, xây kiến trúc cụ thể, đánh giá trên 10.000 ảnh test và đưa mô hình lên web. Sự khác nhau giữa ảnh test chuẩn và ảnh người dùng vẽ trực tiếp được xem là một giới hạn của triển khai, không được bỏ qua khi diễn giải kết quả.

3.1. Lý thuyết CNN

3.1.1. Tại sao cần CNN?

Với ảnh 224×224×3 (RGB), nếu dùng Dense layer thông thường:

- Số tham số đầu vào: 224 × 224 × 3 = 150,528 giá trị
- Một Dense layer với 1000 nơ-ron: 150 triệu tham số — bùng nổ tham số!

CNN giải quyết bằng 3 ý tưởng chính:

1. Local connectivity: Mỗi nơ-ron chỉ nhìn một vùng nhỏ của ảnh
2. Weight sharing: Cùng một kernel dùng chung cho toàn bộ ảnh
3. Translation equivariance: phép tích chập dùng chung kernel khi dịch đầu vào; pooling tạo bất biến cục bộ ở mức độ nhất định.

3.1.2. Phép Tích chập (Convolution)

Với kernel K kích thước (kH × kW) và ảnh đầu vào X kích thước (H × W):

$Z(i,j) = \sum_{u=0}^{k_H-1} \sum_{v=0}^{k_W-1} X(i+u, j+v)K(u,v) + b$ (3.1)

Kích thước đầu ra (với padding P, stride S):

$H_{out} = \left\lfloor \frac{H-k_H+2P}{S} \right\rfloor + 1$ (3.2)

3.1.3. Pooling

Max Pooling: Lấy giá trị lớn nhất trong mỗi vùng → giảm kích thước, giữ đặc trưng nổi bật

$P_{\max}(i,j) = \max_{(u,v)\in R_{ij}} Z(u,v)$ (3.3)

Average Pooling: Lấy trung bình → giảm kích thước mượt hơn

3.1.4. Kiến trúc CNN điển hình

Input → [Conv → BN → ReLU → MaxPool] × N → Flatten → Dense → Softmax

Bảng 3.1. Vai trò của các tầng CNN

Tầng	Vai trò
Conv2D	Trích xuất đặc trưng cục bộ: cạnh, góc, texture
BatchNorm	Chuẩn hóa giúp huấn luyện ổn định hơn
ReLU	Tạo tính phi tuyến
MaxPool	Giảm kích thước, tăng receptive field
Dropout	Điều chuẩn, hạn chế quá khớp
Dense và Softmax	Phân loại đa lớp

3.1.5. Các kiến trúc CNN tiêu biểu

Bảng 3.2. Các kiến trúc CNN và đặc trưng thiết kế đã hiệu chỉnh

Kiến trúc	Năm	Đặc điểm
LeNet-5	1998	Nhận dạng chữ số, convolution và subsampling
AlexNet	2012	ReLU, Dropout và huấn luyện GPU
VGG	2014	Các kernel 3 × 3 xếp chồng
GoogLeNet	2014	Các nhánh Inception
ResNet	2015	Kết nối residual
EfficientNet	2019	Compound scaling độ sâu, độ rộng và độ phân giải

LeNet-5 (1998) hướng tới nhận dạng ký tự và chữ số; các kiến trúc về sau mở rộng khả năng học biểu diễn trên ảnh phức tạp hơn. Chỉ số Top-5 của các biến thể, dataset hoặc chế độ ensemble khác nhau không tạo thành một phép so sánh thống nhất. Bảng 3.2 đối chiếu đặc điểm kiến trúc; các kết quả định lượng của chương được tính riêng trên MNIST.

3.1.6. Backpropagation, loss và optimizer

Lan truyền xuôi tạo logits của từng lớp; CrossEntropyLoss kết hợp log-softmax và negative log likelihood, vì vậy không áp softmax trước loss trong PyTorch. Backpropagation dùng quy tắc chuỗi để tính gradient của kernel và dense layer. Adam cập nhật trọng số dựa trên các ước lượng

moment của gradient; learning rate, weight decay và checkpoint được lựa chọn mà không nhìn test.

$\frac{\partial \mathcal{L}}{\partial K_{uv}} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial Y_{ij}} X_{i+u,j+v}$ (3.4)

Trong (3.4), K là kernel, X là đầu vào và Y là đầu ra trước kích hoạt; chỉ số u,v xác định phần tử kernel và i,j chạy qua vị trí đầu ra. Cùng một trọng số được dùng tại nhiều vị trí nên gradient là tổng đóng góp ở các vị trí đó.

3.1.7. Transfer learning và data augmentation

Transfer learning khởi tạo từ mạng đã học trên tập lớn, thay đầu phân loại rồi đóng băng hoặc fine-tune một số tầng. Nó hữu ích khi dữ liệu đích ít nhưng phải công bố phần pretrained và phần được học lại. Thực nghiệm MNIST dưới đây không dùng pretrained; augmentation chỉ dịch ảnh train ±1 pixel, giữ nguyên validation và test. Không dùng phép xoay mạnh làm thay đổi ý nghĩa giữa các chữ số.

3.2. Dữ liệu ảnh và kiến trúc thực nghiệm

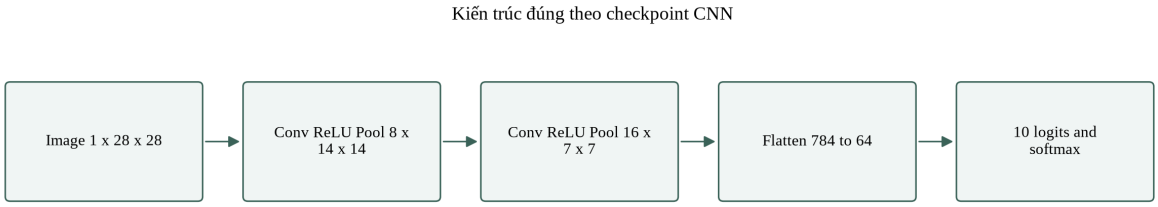
MNIST có 60.000 ảnh train gốc và 10.000 ảnh test độc lập, ảnh xám 28 × 28, nhãn 0–9. Tách 12.000 ảnh từ train làm validation theo stratification, còn 48.000 ảnh huấn luyện. Mỗi pixel được chia 255; không học thống kê từ test. Bộ tải đọc trực tiếp các tệp IDX nén gzip, kiểm tra magic number, số lượng ảnh và kích thước.

Bảng 3.3. Kiến trúc CNN tự huấn luyện

Tầng	Kích thước đầu ra	Tham số
Ảnh vào	1 × 28 × 28	Pixel / 255
Conv1 + ReLU + MaxPool	8 × 14 × 14	Kernel 3; padding 1; pool 2
Conv2 + ReLU + MaxPool	16 × 7 × 7	Kernel 3; padding 1; pool 2
Flatten + Dense + ReLU	64	784 → 64
Dense logits	10	64 → 10

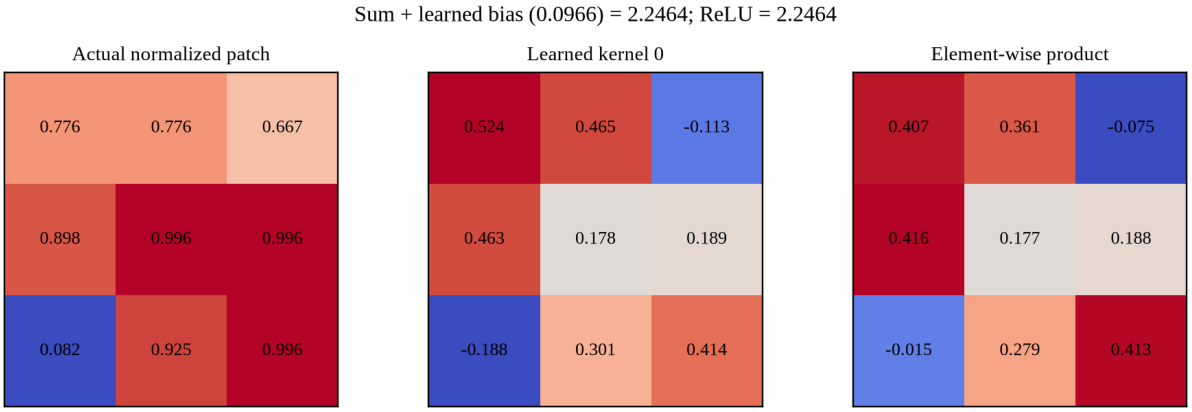
Mạng có 52,138 tham số, train từ đầu 8 epoch trên CPU, Adam learning_rate=0,001, batch_size=128, weight_decay=0,0001 và gradient clipping norm=1. Checkpoint có validation loss nhỏ nhất nằm ở epoch 8. Thời gian thực nghiệm ghi trong metadata là 93.5 giây trên môi trường hiện tại.

3.2.1. Theo dõi một phép tích chập từ ảnh và trọng số thật



Hình 3.1. Luồng tensor đúng với kiến trúc trong checkpoint

Mỗi kernel 3 × 3 được trượt qua ảnh và dùng lại cùng chín trọng số tại mọi vị trí. Trong cài đặt Conv2d, phép toán là tương quan chéo: kernel không bị đảo chiều như một số định nghĩa tích chập toán học. Padding 1 giữ chiều cao và chiều rộng; ReLU đặt giá trị âm về 0; max-pooling 2 × 2 chọn cực đại trên các ô không chồng lấn và giảm mỗi chiều xuống một nửa.

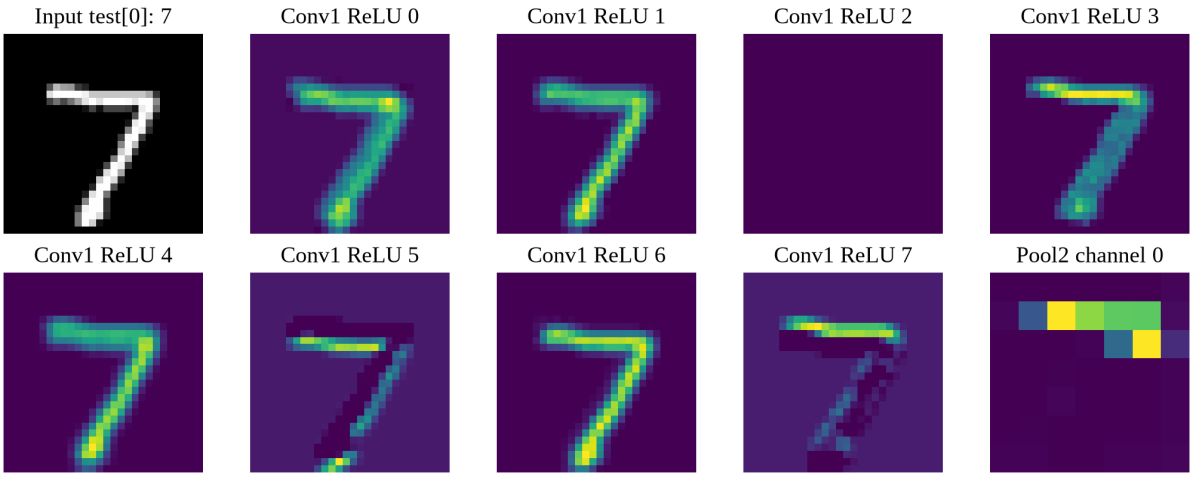


Hình 3.2. Patch MNIST test[0], kernel Conv1 kênh 0 và tích từng phần tử

Hình trên lấy ảnh đầu tiên của test, nhãn thật 7; chọn vị trí kích hoạt lớn nhất của kênh 0 theo một quy tắc cố định. Với chỉ số bắt đầu từ 0, hàng 9, cột 19, tổng chín tích cộng bias 0.096586 bằng 2.246353. Kết quả ReLU đọc trực tiếp từ PyTorch là 2.246353. Sai khác nhỏ chỉ do phép tính số thực, được assert nhỏ hơn 0,00001 trong mã phân tích.

Ví dụ này không dùng một ma trận số đặt sẵn để minh họa. Patch lấy từ tệp IDX, kernel và bias lấy từ checkpoint, ảnh tích từng phần tử được tạo bởi code. Do đó có thể lần ngược mỗi con số về đầu vào và tham số đã học. Tuy nhiên, một activation lớn không tự chứng minh kênh đó biểu diễn một khái niệm có tên như "nét chéo"; đó chỉ là phản ứng số của kernel trên ảnh đang xét.

3.2.2. Feature map, receptive field và số tham số



Hình 3.3. Tám feature map sau ReLU tầng 1 và một kênh sau Pool2 của cùng ảnh

Các feature map cho thấy cùng một ảnh tạo những mức phản ứng khác nhau ở tám kênh. Mỗi kênh được chuẩn màu riêng khi vẽ nên không nên so độ sáng tuyệt đối giữa hai ô ảnh như cùng một thang đo. Mục đích là quan sát vị trí và hình dạng phản ứng; đánh giá độ quan trọng của kênh cần một thí nghiệm riêng như ablation, chưa được thực hiện ở đây.

Theo chuỗi Conv3–Pool2–Conv3–Pool2 với stride của convolution bằng 1, vùng tiếp nhận của một phần tử ở đầu ra Pool2 là 10×10 pixel và bước nhảy tương ứng trên ảnh gốc là 4 pixel. Con số này được suy từ kích thước kernel và stride, không phải kích thước một đối tượng mà mô hình chắc chắn nhận biết. Sau Flatten, tầng dense kết hợp toàn bộ $16 \times 7 \times 7 = 784$ giá trị nên đầu ra phân loại sử dụng thông tin toàn ảnh.

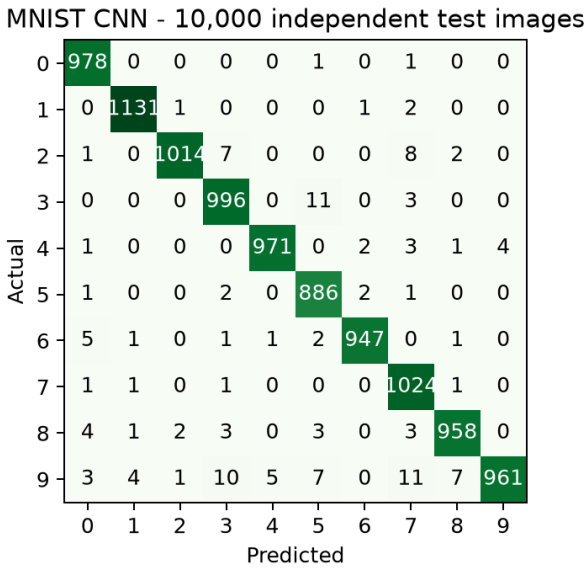
Bảng 3.4. Phân rã 52.138 tham số của mạng

Khối	Cách tính	Tham số
Conv1	$8 \times (1 \times 3 \times 3 + 1)$	80
Conv2	$16 \times (8 \times 3 \times 3 + 1)$	1168
Dense1	$784 \times 64 + 64$	50240
Dense2	$64 \times 10 + 10$	650

Phần lớn tham số nằm ở Dense1, không phải hai tầng tích chập. Điều này giải thích vì sao tăng kích thước ảnh hoặc số kênh có thể làm mô hình nặng lên nhanh nếu vẫn Flatten trực tiếp. Với MNIST, kiến trúc nhỏ giúp huấn luyện CPU và xuất API nhẹ; không có BatchNorm hay Dropout trong checkpoint dù hai tầng này đã được giới thiệu như các lựa chọn CNN phổ biến ở phần lý thuyết.

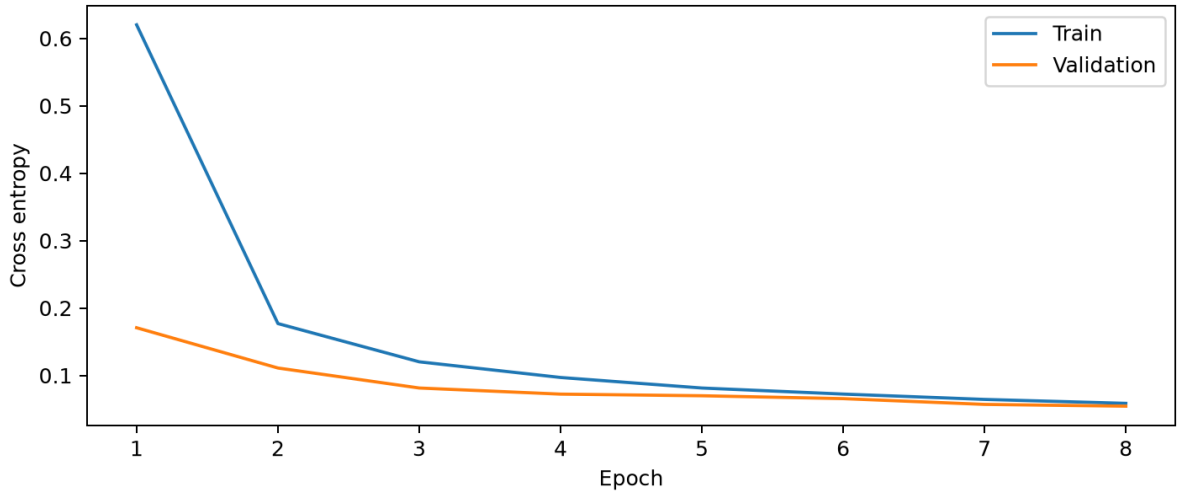
3.3. Kết quả và phân tích dự đoán

Bảng 3.5. Đánh giá CNN trên các tập độc lập



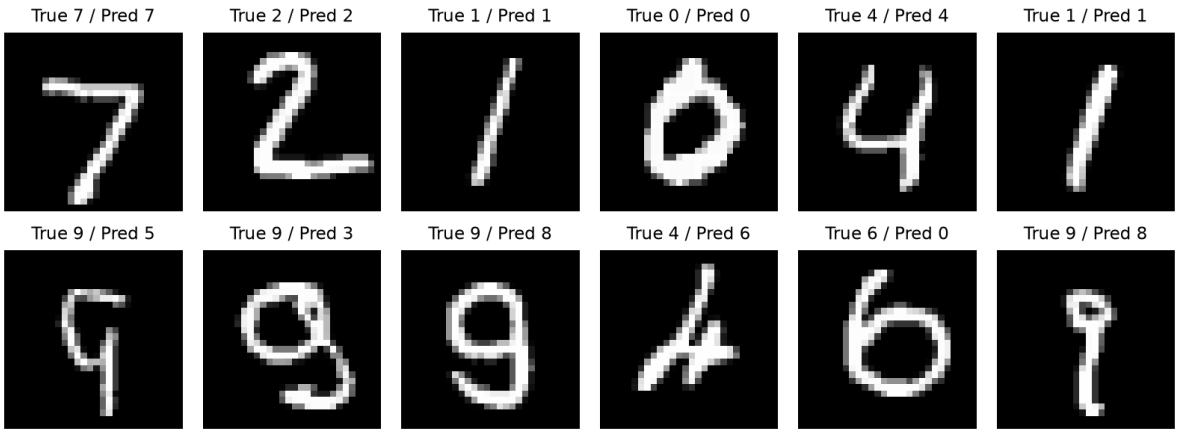
Hình 3.1: Hình 3.4. Learning curve của CNN qua 8 epoch

Tập	Accuracy	Macro-F1
Validation	0.9849	0.9848
Test 10.000 ảnh	0.9866	0.9865



Hình 3.6. Sáu dự đoán đầu và sáu lỗi đầu tiên theo thứ tự test

Accuracy test là 98.66%, tương đương 134 lỗi trên 10.000 ảnh. Các ảnh sai trong Hình 3.6 được chọn bằng quy tắc cố định, không chọn riêng ví dụ đẹp. Learning curve cho thấy validation loss giảm trong cấu hình đã chạy; không kết luận mọi CNN hoặc mọi dữ liệu ảnh đều đạt mức này. Ảnh vẽ tay trên web có thể khác phân phối MNIST nên hiển thị confidence của mô hình thay cho cam kết đúng.



Hình 3.2: Hình 3.5. Ma trận nhầm lẫn CNN trên tập test MNIST

3.3.1. Đọc đường học và đánh giá theo từng chữ số

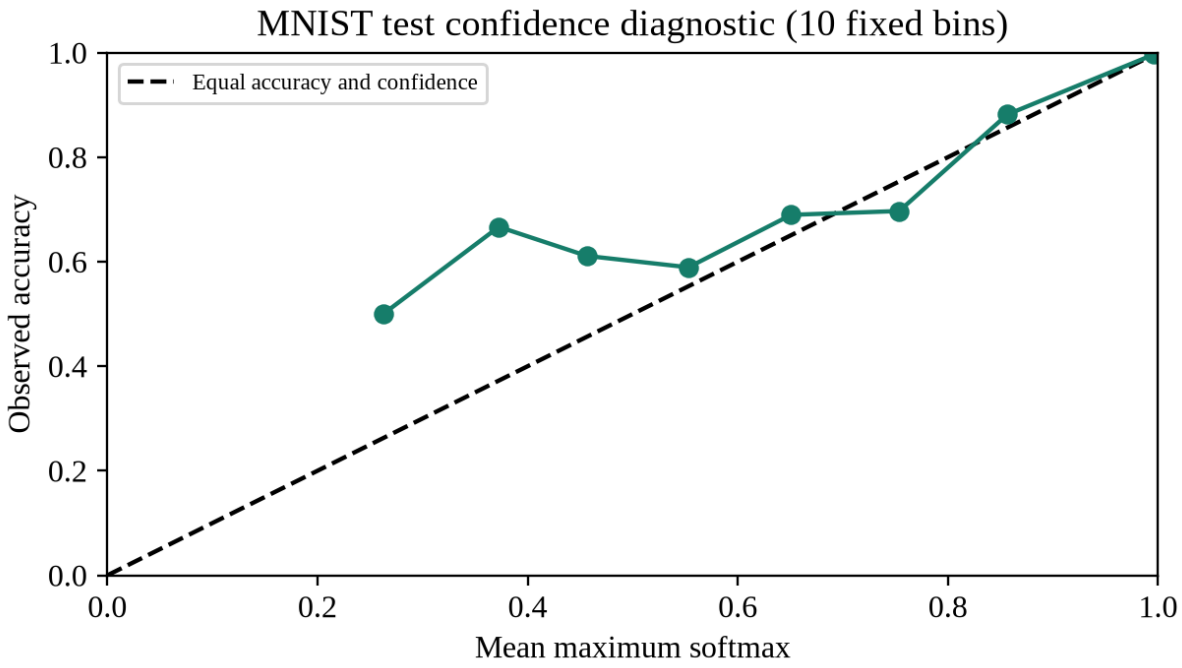
Loss train được đo trong chế độ train có dịch ảnh ngẫu nhiên, còn validation dùng ảnh nguyên và không cập nhật trọng số. Vì điều kiện đầu vào khác nhau, loss train không bắt buộc phải thấp hơn validation ở mọi epoch. Checkpoint epoch 8 được chọn bằng validation loss trong số tám epoch đã định; không phải bằng kết quả tốt nhất trên test và cũng chưa chứng minh rằng huấn luyện lâu hơn sẽ luôn tốt hơn.

Bảng 3.6. Chỉ số từng chữ số tính từ toàn bộ 10.000 dự đoán test

Chữ số	Precision	Recall	F1	Mẫu test
0	0.9839	0.9980	0.9909	980
1	0.9938	0.9965	0.9952	1135
2	0.9961	0.9826	0.9893	1032
3	0.9765	0.9861	0.9813	1010
4	0.9939	0.9888	0.9913	982
5	0.9736	0.9933	0.9834	892
6	0.9947	0.9885	0.9916	958
7	0.9697	0.9961	0.9827	1028
8	0.9876	0.9836	0.9856	974
9	0.9959	0.9524	0.9737	1009

Chữ số 9 có recall khoảng 0,9524, thấp hơn các lớp còn lại trong lần chạy này. Chữ số 7 có recall cao nhưng precision khoảng 0,9697, nghĩa là một phần ảnh thuộc lớp khác bị nhận thành 7. Ma trận nhầm lẫn và các ảnh sai giúp đặt giả thuyết về nét chữ gần nhau; chưa có nhãn nguyên nhân lỗi, vì vậy không quy toàn bộ lỗi cho một đặc điểm nét viết cụ thể.

3.3.2. Confidence không thay thế kiểm tra độ đúng



Hình 3.7. Độ đúng quan sát và confidence trung bình trên các khoảng cố định

Confidence ở đây là softmax lớn nhất. Chia test thành 10 khoảng rộng 0,1 cho ECE mô tả khoảng 0.00243; ECE nhỏ trên tập này không bảo đảm mô hình nhận được ảnh ngoài phân phối. Có 5 ảnh nhận sai dù confidence ít nhất 99%. Các khoảng confidence thấp có ít mẫu nên đường biểu diễn dao động mạnh và không nên diễn giải như một ước lượng ổn định.

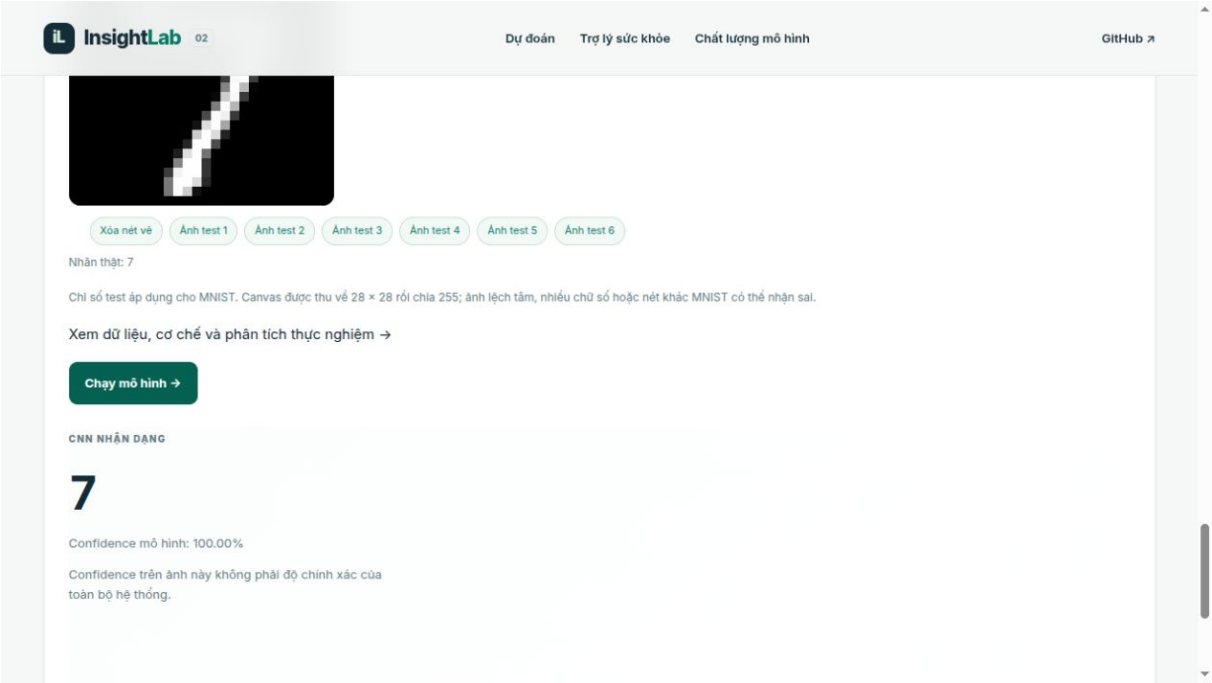
Không dùng biểu đồ test này để chọn một ngưỡng từ chối rồi báo lại chính test như đánh giá độc lập. Nếu muốn bổ sung cơ chế abstention, cần chọn ngưỡng trên validation hoặc một tập hiệu chỉnh riêng, sau đó kiểm tra cả tỷ lệ bao phủ và sai số trên phần được chấp nhận. Đây là hướng phát triển, chưa phải chức năng có hiệu năng được xác minh trong bản hiện tại.

3.3.3. Đường đi của ảnh trên giao diện

Ảnh test mẫu được đưa trực tiếp vào API ở dạng 784 pixel; ảnh canvas được thu từ 280 × 280 xuống 28 × 28 rồi chia 255 giống miền giá trị train. Bản hiện tại chưa tự căn giữa theo trọng tâm hoặc hiệu chỉnh độ dày nét. Vì vậy hướng dẫn yêu cầu một chữ số sáng ở giữa nền đen. Việc công bố đúng bước xử lý có ý nghĩa hơn việc gán Accuracy MNIST cho mọi ảnh người dùng có thể vẽ.

API đã bổ sung kiểm tra ảnh toàn 0 ở phía máy chủ, ngoài kiểm tra tại giao diện. Nhờ đó gửi yêu cầu trực tiếp không còn nhận một nhãn tưởng như hợp lệ cho ảnh trống. Các lỗi thiếu pixel, pixel ngoài 0–255 hoặc NaN cũng bị từ chối. Thay đổi này cải thiện tính đúng đắn của dịch vụ; không được tính là tăng Accuracy của checkpoint vì trọng số không thay đổi.

Demo: `https://rag-httm-chatbot.vercel.app/?demo=cnn#experiments`



Hình 3.8. Ảnh chụp production: ảnh test nhãn 7 được API nhận dạng thành 7

3.4. Suy luận, triển khai và phạm vi mở rộng

Checkpoint PyTorch được xuất thành trọng số JSON. API thực hiện convolution, ReLU, max-pooling, flatten và dense bằng đúng trọng số đã huấn luyện, dùng Python standard library để tránh đóng gói PyTorch nặng vào serverless. Bộ vector đối chiếu kiểm tra logits của API với PyTorch. Giao diện hỗ trợ ảnh test thật, canvas vẽ số, kiểm tra ảnh trống, trạng thái đang xử lý và thông báo lỗi.

ASL được giữ như ví dụ mở rộng của bài toán thị giác: các cử chỉ M/N hoặc U/V gần nhau, nền ảnh và góc chụp có thể làm giảm khả năng tổng quát hóa. Trong lần thực hiện này chưa huấn luyện lại ASL, vì vậy các accuracy và biểu đồ ASL cũ không được dùng như kết quả mới. Tương tự, không khẳng định Keras và PyTorch tương đương từ một phép thử chỉ chạy PyTorch.

Chương 4

MẠNG NƠ-RON HỒI TIẾP RNN

Đối với chuỗi giá, thứ tự quan sát quyết định thông tin nào thực sự có sẵn tại thời điểm dự báo. Chia ngẫu nhiên các cửa sổ chồng lấn có thể làm mô hình nhìn gián tiếp vào tương lai. Hơn nữa, giá ngày mai thường gần giá hôm nay, nên một đường dự báo bám sát giá thực chưa chứng minh mạng đã học được tín hiệu hữu ích.

Chương xây bài toán dự báo một bước bằng 30 log-return đã biết và so sánh RNN, LSTM, GRU với Persistence. Phép chia theo ngày của nhãn, chuẩn hóa chỉ trên train và lựa chọn bằng validation được đặt trước việc đánh giá test. Hai chuỗi FPT và vàng futures cho phép kiểm tra liệu kết luận có nhất quán giữa các miền hay không.

Nội dung chính gồm cơ chế nhớ và cổng, cách tạo cửa sổ, quy trình huấn luyện, đường học và sai số theo đơn vị giá. Phân tích độ bất định bổ sung cho so sánh điểm MAE; phần triển khai giải thích vì sao một baseline đơn giản vẫn có thể được chọn cho web.

4.1. Lý thuyết RNN

4.1.1. Vấn đề với dữ liệu tuần tự

Dense layer không giữ trạng thái qua thời gian. CNN có thể xử lý tín hiệu 1D theo trường tiếp nhận hữu hạn; RNN bổ sung trạng thái truyền qua các bước để biểu diễn ngữ cảnh của chuỗi.

- Giá hôm nay phụ thuộc vào giá hôm qua
- Từ trong câu phụ thuộc vào từ trước đó
- Nhịp tim tương lai phụ thuộc vào lịch sử

→ Cần kiến trúc có bộ nhớ (memory) giữa các bước thời gian.

4.1.2. RNN cơ bản — Vanilla RNN

Tại mỗi bước thời gian t:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b) \qquad \hat{y}_t = W_y h_t + b_y \tag{4.1}$$

Trong đó:

- xi: vector đầu vào tại bước t

• hi: hidden state (bộ nhớ ngắn hạn)

• Wx, Wh và Wγ là ma trận trọng số; các tham số được dùng chung tại mọi bước thời gian.

Backpropagation Through Time (BPTT):

$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \left(\frac{\partial h_k}{\partial \theta} \right)_{direct}$ (4.2)

θ là một tham số dùng chung của mạng; đạo hàm cuối chỉ lấy tác động trực tiếp tại bước k. Tổng theo k cộng các đường mà tham số ảnh hưởng tới loss ở bước t. Đạo hàm giữa hai trạng thái bao gồm toàn bộ các bước trung gian, với đạo hàm đồng nhất khi k=t.

4.1.3. LSTM — Long Short-Term Memory

LSTM thêm cell state ct như ”bộ nhớ dài hạn” và 3 gates kiểm soát luồng thông tin:

Forget Gate: Quyết định quên bao nhiêu cell state cũ

$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ (4.3)

Input Gate: Quyết định thêm thông tin mới nào

$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$ (4.4)

Cập nhật Cell State:

$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ (4.5)

Output Gate: Quyết định đầu ra

$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t)$ (4.6)

4.1.4. GRU — Gated Recurrent Unit

GRU đơn giản hóa LSTM bằng cách gộp forget gate và input gate thành update gate:

$$\begin{aligned} z_t &= \sigma(W_z[h_{t-1}, x_t] + b_z) \\ r_t &= \sigma(W_r[h_{t-1}, x_t] + b_r) \\ \tilde{h}_t &= \tanh(W[r_t \odot h_{t-1}, x_t] + b) \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \end{aligned} \quad (4.7)$$

Bảng 4.1. So sánh cấu trúc recurrent network

Mạng	Trạng thái	Số cổng	Đặc điểm
RNN	Hidden state ht	Không có cổng	Ít tham số, khó học phụ thuộc xa
LSTM	ht và cell state ct	Input, forget, output	Luồng cộng qua cell state hỗ trợ truyền gradient
GRU	Hidden state ht	Reset và update	Ít tham số hơn LSTM cùng kích thước ẩn

4.1.5. Cửa sổ trượt và các kiểu đầu vào đầu ra

Cửa sổ trượt tạo một mẫu bằng L quan sát quá khứ và nhãn của bước kế tiếp. Many-to-one ánh xạ chuỗi thành một giá trị hoặc nhãn; one-to-many sinh chuỗi từ một ngữ cảnh đầu; many-to-many tạo một đầu ra ở mỗi bước hoặc một chuỗi đầu ra khác độ dài. Thực nghiệm dự báo giá trong chương sử dụng many-to-one.

$$X_t = [r_{t-L+1}, \dots, r_t], \quad r_t = \log(P_t/P_{t-1}), \quad \hat{P}_{t+1} = P_t e^{\hat{r}_{t+1}} \text{ (4.8)}$$

Pt là giá đóng cửa, rt là log-return, L=30 là chiều dài cửa sổ. Dự báo tại t dùng dữ liệu không muộn hơn t; nhãn ở t+1. Mỗi cửa sổ khởi tạo trạng thái ẩn bằng 0 và không truyền hidden state từ test về train.

4.1.6. BPTT và ổn định gradient

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \text{ (4.9)}$$

BPTT trải mạng theo thời gian và cộng gradient của cùng tham số qua các bước. Tích nhiều Jacobian có chuẩn nhỏ hơn 1 có thể làm gradient suy giảm, còn chuẩn lớn có thể gây bùng nổ. LSTM/GRU tạo đường truyền có cổng để giảm khó khăn này nhưng không bảo đảm giải quyết mọi phụ thuộc dài hạn. Gradient clipping giới hạn norm; chuẩn hóa chỉ trên train và chọn checkpoint bằng validation giúp ổn định thực nghiệm.

4.2. Dữ liệu, phép chia và cấu hình

Dùng snapshot Yahoo Finance của Tiểu luận 02: FPT.VN có 1.304 phiên và vàng futures GC=F có 1.257 phiên, từ 15/10/2021 đến 14/10/2026. GC=F không được đổi tên thành spot XAU/USD. Lợi suất được tính theo phiên có thật, không nội suy ngày nghỉ. Dữ liệu e-commerce không được đưa vào thực nghiệm mới vì chưa xác minh được thứ tự sự kiện và nhãn đủ để biện minh cho RNN.

Ngày của nhãn quyết định tập dữ liệu: FPT train trước 13/02/2026, validation từ 13/02 đến trước 04/06/2026; vàng train trước 26/02/2026, validation đến trước 09/06/2026. Test kéo dài đến nhãn 14/10/2026. Cách định nghĩa này khác chỉ số ngày đặc trưng của bảng thị trường ở Chương 5; không so sánh trực tiếp hai bảng như cùng tập test. Validation/test được phép dùng lịch sử đã biết trước ngày nhãn, phù hợp dự báo cuốn chiếu một bước.

Bảng 4.2. Phân chia chuỗi theo ngày nhãn

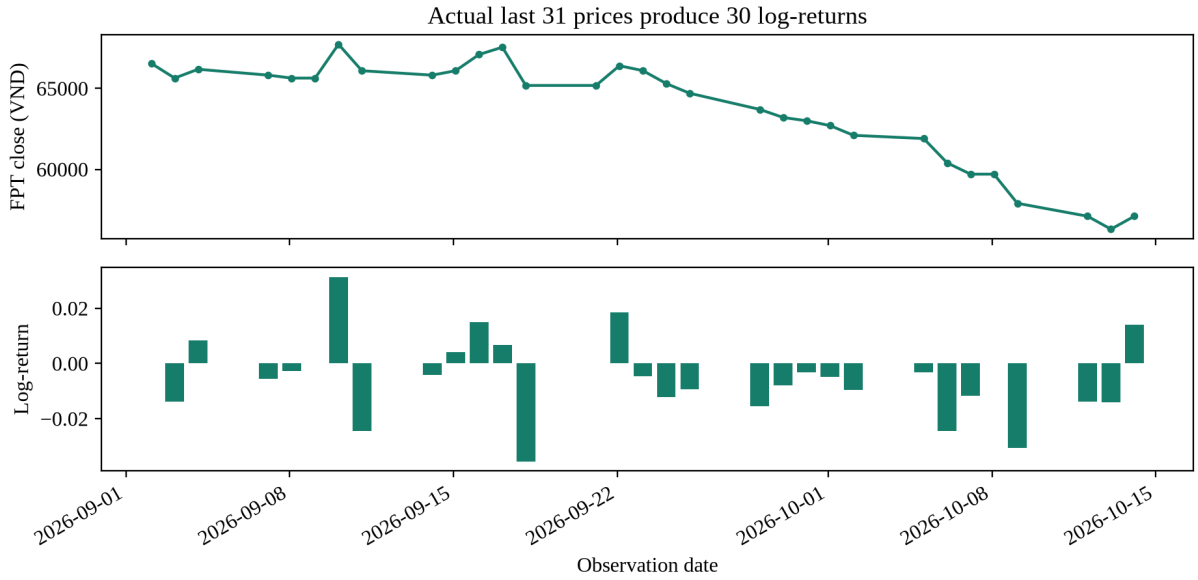
Miền	Train / Val / Test	Khoảng ngày nhãn test
FPT	1099 / 79 / 95	2026-06-04 — 2026-10-14
Gold	1066 / 71 / 89	2026-06-09 — 2026-10-14

Ba mạng dùng 16 hidden units, một recurrent layer và một linear head, Adam learning_rate=0,001, 35 epoch, batch_size=128 và seed=42. Trung bình và độ lệch chuẩn của đầu

vào/nhãn chỉ tính trên train. Checkpoint tối thiểu validation MSE được lưu; mạng triển khai được chọn bằng validation MAE theo đơn vị giá, có Persistence cùng tham gia. Test được giữ cho báo cáo cuối.

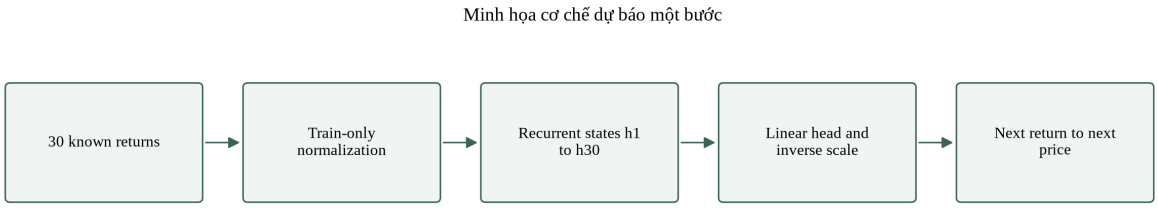
GRU trong PyTorch có hai khác biệt quy ước so với công thức lý thuyết ở trên: reset gate nhân sau phép biến đổi tuyến tính của hidden state trong nhánh ứng viên; update gate z đặt trọng số lên trạng thái cũ thay vì ứng viên. Cụ thể, ht=(1−zt)⊙nt+zt⊙ht−1, với nt là ứng viên. API dùng đúng hai quy ước của checkpoint; kiểm thử cả sáu mạng phát hiện sai khác thứ tự cổng, vị trí reset và cách phối hợp trạng thái.

4.2.1. Từ giá quan sát đến một mẫu dự báo cụ thể



Hình 4.1. 31 giá FPT cuối snapshot và 30 log-return được tính từ chúng

Cửa sổ minh họa lấy từ 2026-09-02 đến 2026-10-14, gồm 31 giá đóng cửa để tạo 30 log-return. Hai giá cuối là 56,300 và 57,100 VND, tạo log-return cuối 0.01410958. Đây chính là một phần đầu vào mặc định của demo, không phải một chuỗi được đặt số để làm đẹp hình. Snapshot dừng ở 14/10/2026; dự báo tiếp theo sau cửa sổ không được gọi là giá đã xác minh của ngày hiện tại.



Hình 4.2. Minh họa cơ chế many-to-one và biến đổi ngược về giá

Mạng nhận tensor [batch, 30, 1], không nhận trực tiếp lịch ngày hoặc mức giá tuyệt đối. Đầu vào là log-return được chuẩn hóa bằng trung bình và độ lệch chuẩn train. Trạng thái ở bước cuối đi qua Linear(16,1), sau đó đầu ra được đưa về thang log-return bằng tham số nhãn train. Cuối

cùng nhân giá gần nhất với exp của log-return dự đoán để ra mức giá một bước tiếp theo.

Chọn log-return giảm sự phụ thuộc trực tiếp vào mức giá và giữ giá dự đoán dương sau biến đổi ngược. Tuy vậy phân phối lợi suất vẫn có thể thay đổi theo chế độ thị trường, và mẫu 30 bước không chứa đầy đủ tin tức hoặc điều kiện vĩ mô. Hai chuỗi dùng checkpoint riêng; không chuyển trọng số FPT sang vàng hoặc chuẩn hóa chung bằng dữ liệu của cả hai miền.

4.2.2. Vai trò của trạng thái và cổng trong thực nghiệm

Vanilla RNN cập nhật cùng một trạng thái ẩn qua 30 bước; mọi thông tin cần dùng phải được nén vào vector đó. LSTM có trạng thái ô nhớ và các cổng để điều tiết lượng thông tin cũ giữ lại, lượng ứng viên được ghi vào và phần được phát ra. GRU kết hợp cơ chế cập nhật và reset trong một trạng thái. Những cấu trúc này mở ra đường truyền khác cho thông tin và gradient, nhưng việc cổng có giúp bài toán cụ thể hay không vẫn phải kiểm tra bằng thực nghiệm.

Các mạng dùng cùng độ rộng 16, cùng optimizer, số epoch và quy tắc checkpoint để giảm khác biệt không cần thiết. Số tham số vẫn không bằng nhau vì LSTM và GRU có nhiều nhánh cổng. Đây là so sánh trong một cấu hình kiểm soát, chưa phải tìm kiếm tối ưu công bằng theo cùng số phép tính hoặc cùng số tham số. Không kết luận một họ mạng nói chung kém hơn chỉ từ cấu hình duy nhất này.

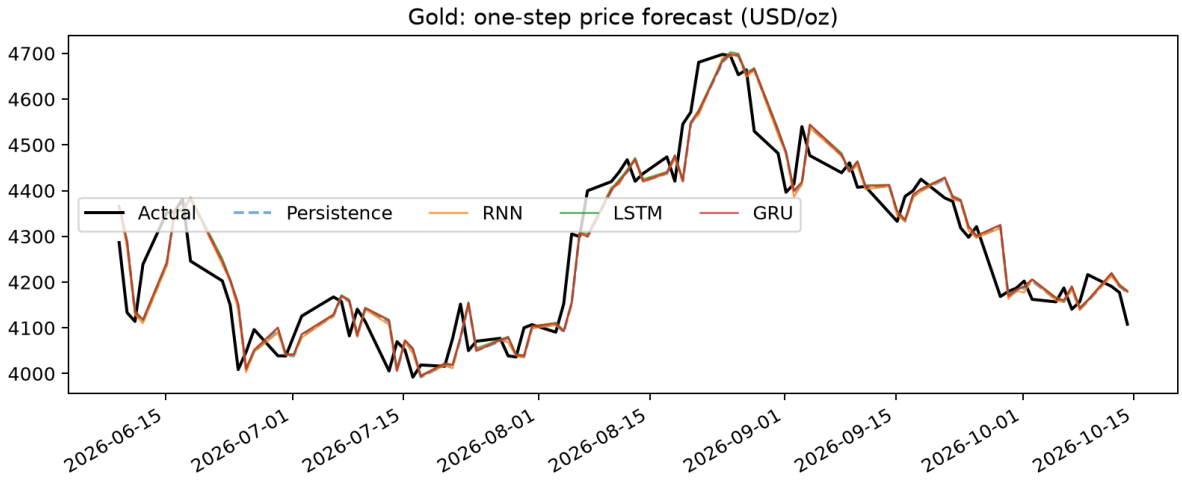
4.3. So sánh RNN, LSTM, GRU và Persistence

Bảng 4.3. Kết quả mạng chuỗi trong đơn vị gốc VND hoặc USD/oz

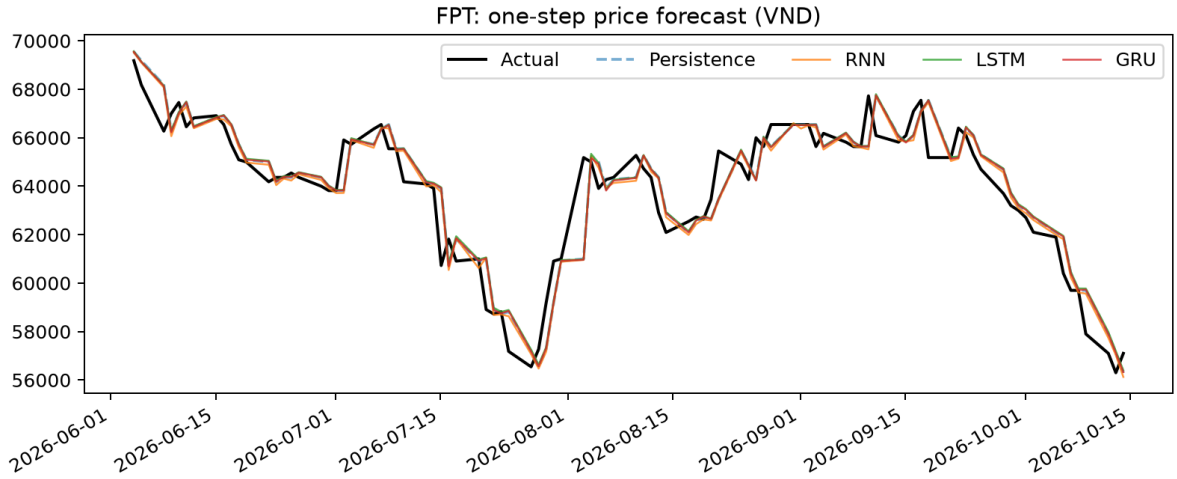
Miền / Mạng	Tham số	Val MAE	Test MAE	Test RMSE
FPT / RNN	321	1270.880	720.604	1032.599
FPT / LSTM	1233	1271.182	745.759	1050.320
FPT / GRU	929	1265.027	737.054	1044.655
Gold / RNN	321	63.676	47.342	61.988
Gold / LSTM	1233	63.707	48.168	63.044
Gold / GRU	929	63.622	48.097	62.834

FPT: Persistence test MAE 734.258 VND. Mạng có validation MAE thấp nhất là GRU (1265.027), trong khi Persistence validation MAE 1266.974. Vì vậy quy tắc trước test chọn GRU để trình diễn trên web.

GOLD: Persistence test MAE 48.089 USD/oz. Mạng có validation MAE thấp nhất là GRU (63.622), trong khi Persistence validation MAE 63.152. Vì vậy quy tắc trước test chọn Persistence để trình diễn trên web.



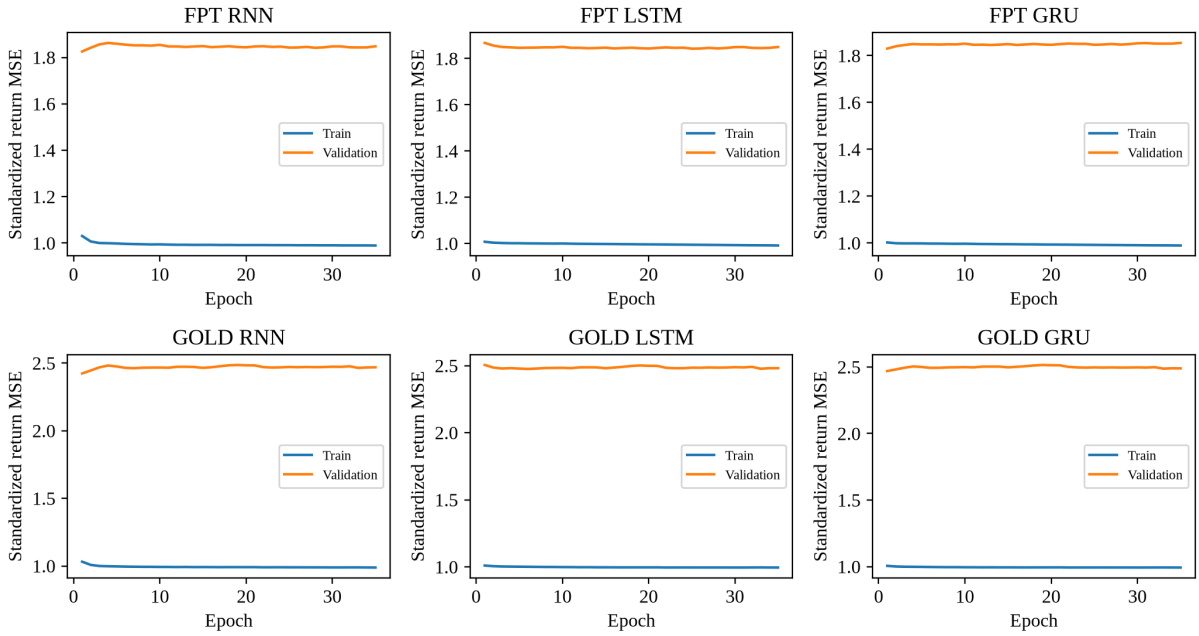
Hình 4.1: Hình 4.3. Dự báo một bước FPT bằng ba mạng và baseline



Hình 4.4. Dự báo một bước vàng futures GC=F

Vanilla RNN có MAE test thấp nhất trong ba mạng ở cả hai miền trong lần chạy này, nhưng không được đổi lựa chọn triển khai sau khi nhìn test. Kết quả bác bỏ việc mặc định LSTM luôn tốt hơn. Chuỗi ngắn, ít dữ liệu, nhiễu thị trường và một random seed hạn chế khả năng khái quát hóa. Mức chênh MAE nhỏ chưa chứng minh lợi ích giao dịch; báo cáo không suy ra lợi nhuận.

4.3.1. Đường học và tính nhất quán của quy tắc lựa chọn



Hình 4.5. Train/validation MSE của sáu mạng trên nhãn log-return đã chuẩn hóa

Đường học được lưu ở từng epoch, không nội suy thành kết quả chưa chạy. Loss trong hình là MSE của nhãn chuẩn hóa; MAE trong bảng kết quả là sai số sau khi đổi về đơn vị giá. Hai thước đo không xếp hạng mô hình giống nhau trong mọi tình huống. Bởi vậy checkpoint được chọn theo validation MSE trong từng lần train, còn lựa chọn kiến trúc phục vụ web dùng validation MAE theo đơn vị gốc, với quy tắc được giữ trước khi xem test.

Mạng có thể học một lợi suất gần 0 và tạo đường giá trông rất sát quan sát vì mỗi dự báo được neo ở giá của phiên trước. Persistence cũng có lợi suất dự đoán bằng 0 mà không học tham số. Do đó biểu đồ bám giá chỉ hỗ trợ quan sát giai đoạn sai lệch; kết luận về độ hữu ích phải dựa trên phần chênh sai số so với baseline trên cùng ngày nhãn.

4.3.2. Chênh lệch so với baseline và độ bất định theo khối

$$\Delta_{MAE} = 100\frac{MAE_{Persistence} - MAE_{model}}{MAE_{Persistence}} \quad (4.10)$$

Giá trị dương của Δ thể hiện giảm MAE so với Persistence. Phân tích bổ sung lấy mẫu lại các cặp sai số cùng ngày bằng circular block bootstrap, khối dài 5 quan sát, 2.000 lần và seed 42. Mỗi lần dùng cùng chỉ số cho mô hình và baseline để giữ tính ghép cặp; lấy theo khối nhằm bảo toàn một phần phụ thuộc thời gian ngắn. Phân vị 2,5% và 97,5% tạo khoảng mô tả 95%.

Bảng 4.4. Chẩn đoán test so với Persistence; không dùng để đổi lựa chọn web

Miền / mạng	Giảm MAE (%)	Khoảng bootstrap 95%
fpt / RNN	+1.860	[-1.252; +5.105]
fpt / LSTM	-1.566	[-3.272; -0.110]
fpt / GRU	-0.381	[-1.087; +0.266]

Miền / mạng	Giảm MAE (%)	Khoảng bootstrap 95%
gold / RNN	+1.553	[+0.291; +2.732]
gold / LSTM	-0.164	[-1.676; +1.368]
gold / GRU	-0.017	[-0.976; +0.926]

FPT/RNN có mức giảm điểm khoảng 1,86% nhưng khoảng bootstrap cắt 0; FPT/GRU gần baseline và cũng có khoảng cắt 0. Gold/RNN có khoảng dương trong thiết lập khối đã dùng, song đây là phân tích sau đánh giá, trên một snapshot và chưa điều chỉnh cho nhiều so sánh. Không suy rộng thành bảo đảm vượt baseline ở thị trường tương lai. LSTM FPT kém baseline trong khoảng mô tả này cũng không chứng minh mọi LSTM đều không phù hợp.

Độ dài khối 5 là lựa chọn phân tích cố định, chưa có nghiên cứu độ nhạy với khối khác hoặc nhiều seed train. Khoảng trên phản ánh biến thiên khi lấy mẫu lại các lỗi đã có, không bao gồm toàn bộ bất định do huấn luyện, thay đổi thị trường hay sai số dữ liệu. Vì vậy nó bổ sung cho điểm MAE chứ không thay thế walk-forward backtest.

4.3.3. Hai demo chuỗi và cách đọc kết quả

Demo FPT nhận giá gần nhất theo VND và 30 log-return từ cũ đến mới. Mô hình được cố định là GRU do lựa chọn validation; không thay thành RNN chỉ vì RNN có test tốt hơn trong lần chạy này. Người dùng có thể thay một hoặc nhiều lợi suất để xem độ nhạy của mô hình. Đó là phân tích kịch bản đầu vào, không phải một phép backtest mới và không có kết quả thực tế để tính sai số ngay trên giao diện.

Demo: `https://rag-httm-chatbot.vercel.app/?demo=sequence#experiments`

Demo vàng chọn GC=F và trả đơn vị USD/oz. Validation chọn Persistence nên dự báo bằng giá gần nhất dù 30 log-return vẫn có trong form chung. Giao diện công bố rõ lựa chọn này thay vì gọi một mạng sâu rồi âm thầm bỏ qua dự đoán của nó. Liên kết dưới mở đúng miền vàng. Cả hai demo đều báo ngày snapshot và không gọi API thị trường trực tiếp để tự làm mới giá.

Demo: `https://rag-httm-chatbot.vercel.app/?demo=sequence&domain=gold#experiments`

Bộ phân tích đầu vào được sửa để không chuyển một phần tử trống giữa hai dấu phẩy thành số 0. Máy chủ vẫn yêu cầu đúng 30 số hữu hạn trong $[-1,1]$ và giá gần nhất dương. Các giới hạn này kiểm tra cấu trúc yêu cầu, không chứng minh kịch bản nằm trong phân phối train. Không dùng giá trị đầu ra để suy ra lợi nhuận, tín hiệu mua bán hoặc mức rủi ro đầu tư cá nhân.

4.4. Hạn chế và khả năng mở rộng

Thực nghiệm là dự báo một bước trên snapshot cố định, không phải dự báo nhiều bước tương lai hoặc dữ liệu trực tiếp. Phân tích bootstrap theo khối ở trên chưa thay thế kiểm tra nhiều seed và

walk-forward backtest. Hướng mở rộng gồm nghiên cứu độ nhạy với chiều dài cửa sổ, các biến đầu vào có timestamp và ngân sách tuning khóa trước. Không dùng test lặp lại để điều chỉnh cấu hình.

Mô hình triển khai giữ checkpoint train lịch sử, không refit bằng test để giữ được dấu vết đánh giá. Người dùng có thể sửa giá gần nhất và 30 log-return để thử kịch bản; kết quả không có khoảng tin cậy đã hiệu chỉnh cho kịch bản ngoài miền. Checkpoint của mọi RNN/LSTM/GRU và training history được lưu riêng để kiểm tra.

Chương 5

KNOWLEDGE GRAPH VÀ RAG

Dự đoán số hoặc nhãn mới giải quyết một phần nhu cầu của người dùng. Khi hỏi giá gần nhất đi cùng chỉ báo nào, hoặc một khái niệm sức khỏe có liên hệ với yếu tố nào, người dùng còn cần ngữ cảnh và bằng chứng. Một mô hình dự đoán không tự trở thành kho tri thức, còn câu trả lời trôi chảy không bảo đảm có căn cứ.

Chương tổ chức tri thức dưới hai dạng bổ trợ: văn bản có thể truy xuất theo mức liên quan và đồ thị thực thể–quan hệ có thể kiểm tra đường nối. Prototype sử dụng TF-IDF, NetworkX và bộ tổng hợp xác định; giao diện sức khỏe dùng BM25 cùng luật đối thoại. Phạm vi này cho phép quan sát luồng bằng chứng mà không gán cho hệ thống năng lực của một mô hình ngôn ngữ chưa được tích hợp.

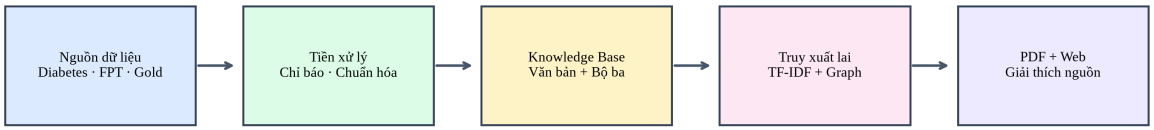
Từ nền tảng KG và RAG, chương chuyển sang dữ liệu ba miền, các bài toán dự đoán, cơ chế truy xuất, giao diện và kiểm thử triển khai. Phần phân tích bổ sung theo dõi một câu hỏi cụ thể qua thứ hạng tài liệu, quan hệ đồ thị và câu trả lời, đồng thời tách rõ điểm truy xuất khỏi chất lượng suy luận.

5.1. MỞ ĐẦU

Knowledge Graph (KG) và Retrieval-Augmented Generation (RAG) xuất phát từ hai truyền thống khác nhau. KG nhấn mạnh biểu diễn tường minh: thực thể nào tồn tại, chúng liên hệ ra sao và ràng buộc nào phải được giữ. RAG xuất phát từ nhu cầu cho mô hình sinh tiếp cận một kho tri thức ngoài tại thời điểm trả lời. Khi ghép hai hướng này, hệ thống vừa có khả năng tìm kiếm mềm theo ngôn ngữ, vừa có cấu trúc để kiểm tra đường suy luận.

Bài làm lựa chọn kiến trúc có thể chạy trọn trong một notebook. TF-IDF đảm nhiệm truy xuất văn bản, NetworkX lưu đồ thị, còn bộ tổng hợp ghép bằng chứng tìm được thành câu trả lời. Dữ liệu FPT.VN và GC=F được đọc từ snapshot đã khóa trong lần tái lập; có chế độ tải mới khi được bật; dữ liệu tiểu đường lấy từ scikit-learn.

Ba miền được chọn tạo ra phép thử đủ khác biệt. Y tế yêu cầu diễn đạt thận trọng; chứng khoán là chuỗi thời gian nội địa; vàng chịu tác động của yếu tố vĩ mô và được niêm yết quốc tế. Một pipeline dùng chung phải giữ được ngữ cảnh riêng của từng miền mà không trộn lẫn bằng chứng.



Hình 5.1. Kiến trúc KG và RAG

5.1.1. LỊCH SỬ PHÁT TRIỂN CỦA KNOWLEDGE GRAPH

Ý tưởng nối các khái niệm thành mạng ngữ nghĩa đã xuất hiện từ những nghiên cứu ban đầu về trí nhớ và biểu diễn tri thức. Sau đó, hệ chuyên gia đưa luật IF-THEN vào các miền hẹp. Bước ngoặt của Web ngữ nghĩa là việc chuẩn hóa RDF, RDFS, OWL và SPARQL, khiến bộ ba chủ thể-vị từ-đối tượng trở thành một ngôn ngữ trao đổi máy đọc được,.

Năm 2012, cách gọi Knowledge Graph được phổ biến rộng rãi qua công cụ tìm kiếm của Google. Trong doanh nghiệp, KG tiếp tục được dùng cho danh mục sản phẩm, phát hiện gian lận, quản trị dữ liệu chủ và gợi ý nội dung. Điểm chung không phải một loại cơ sở dữ liệu cụ thể, mà là cách coi quan hệ giữa các thực thể là công dân hạng nhất.

Ở bài này, đồ thị được cài bằng Labeled Property Graph đơn giản. Mỗi nút có type và domain; mỗi cạnh có nhãn quan hệ. Lựa chọn này gần với cách Neo4j tổ chức dữ liệu và thuận tiện cho trực quan hóa, dù chưa khai thác đầy đủ suy luận ontology như OWL.

5.1.2. LỊCH SỬ PHÁT TRIỂN CỦA RAG

Trước RAG, các hệ hỏi đáp mở thường tách bộ tìm kiếm tài liệu và bộ đọc hiểu. REALM đưa cơ chế truy xuất vào quá trình tiền huấn luyện. Lewis và cộng sự sau đó định danh kiến trúc Retrieval-Augmented Generation, kết hợp bộ truy xuất dense với mô hình sinh seq2seq cho nhiệm vụ giàu tri thức.

Sự phổ biến của mô hình ngôn ngữ lớn khiến RAG trở thành một mẫu triển khai thực dụng: tài liệu có thể cập nhật mà không cần huấn luyện lại toàn bộ mô hình, đồng thời câu trả lời giữ được liên kết về nguồn. Tuy nhiên, vector search thường ưu tiên đoạn giống câu hỏi và có thể bỏ qua quan hệ gián tiếp. GraphRAG giải quyết phần thiếu này bằng việc xây thực thể, quan hệ, cụm và tóm tắt theo cấu trúc.

Nguyên mẫu trong notebook không cố tái tạo toàn bộ Microsoft GraphRAG. Nó giữ hai ý cốt lõi: truy xuất đồng thời từ văn bản và đồ thị, sau đó đưa cả hai nhóm bằng chứng vào bước tổng hợp.

5.1.3. VAI TRÒ VÀ Ý NGHĨA TRONG AI

KG giúp mô hình có từ điển thực thể thống nhất, quan hệ có tên và đường đi có thể kiểm tra. RAG giúp tri thức ngoài đi vào ngữ cảnh trả lời đúng thời điểm. Kết hợp hai thành phần làm giảm nguy cơ câu trả lời chỉ dựa trên tương đồng bề mặt, nhưng không đồng nghĩa loại bỏ hoàn toàn sai sót.

Trong y tế, giá trị quan trọng nhất là provenance: người đọc phải biết nhận xét dựa trên biến nào và nguồn nào. Trong tài chính, dữ liệu có tính thời điểm nên ngày quan sát phải đi cùng con số. Trong cả hai trường hợp, câu trả lời nên phân biệt sự kiện đo được, quan hệ thường gặp và kết luận mang tính dự báo.

Ý nghĩa học thuật của bài nằm ở khả năng chạy lại. Người chấm có thể Run All, xem dữ liệu thô đã lưu, đối chiếu bộ ba, kiểm tra tài liệu được truy xuất và mở PDF được sinh từ cùng một trạng thái chạy.

5.1.4. PHẠM VI, CÂU HỎI VÀ ĐÓNG GÓP

Bài đặt ba câu hỏi: (1) một cấu trúc KG thống nhất có mô tả được ba miền khác nhau không; (2) truy xuất lai có cung cấp nhiều bằng chứng hơn vector search đơn lẻ không; và (3) có thể đóng gói toàn bộ quy trình trong một notebook duy nhất không.

Đóng góp thực hành gồm lớp KnowledgeGraphCore, bộ tải Yahoo Finance không cần API key, các bước tạo chỉ báo, VectorRetriever, truy xuất đồ thị theo miền, đánh giá precision@3, ứng dụng dự đoán, chatbot sức khỏe có nguồn và bộ sinh PDF. Mọi tệp phụ đều được tạo lại từ notebook.

Ba nguồn dữ liệu được mô tả bằng số dòng, số cột, khoảng thời gian, đơn vị và biến đích. Bảng được sinh trực tiếp từ DataFrame sau xử lý nên khớp với lần chạy hiện tại.

Mô tả ba nguồn dữ liệu tại thời điểm chạy

Miền	Nguồn	Số dòng	Số cột	Phạm vi	Biến đích/đơn vị
Tiểu đường	scikit-learn load_diabetes	442	11	442 mẫu nghiên cứu	tiến triển sau 1 năm
Chứng khoán	Yahoo Finance chart API - FPT.VN	1304	10	15/10/2021 - 14/10/2026	Close phiên kế tiếp
Giá vàng	Yahoo Finance chart API - GC=F	1257	10	15/10/2021 - 14/10/2026	Close phiên kế tiếp

Hình 5.2. Thông tin ba miền dữ liệu

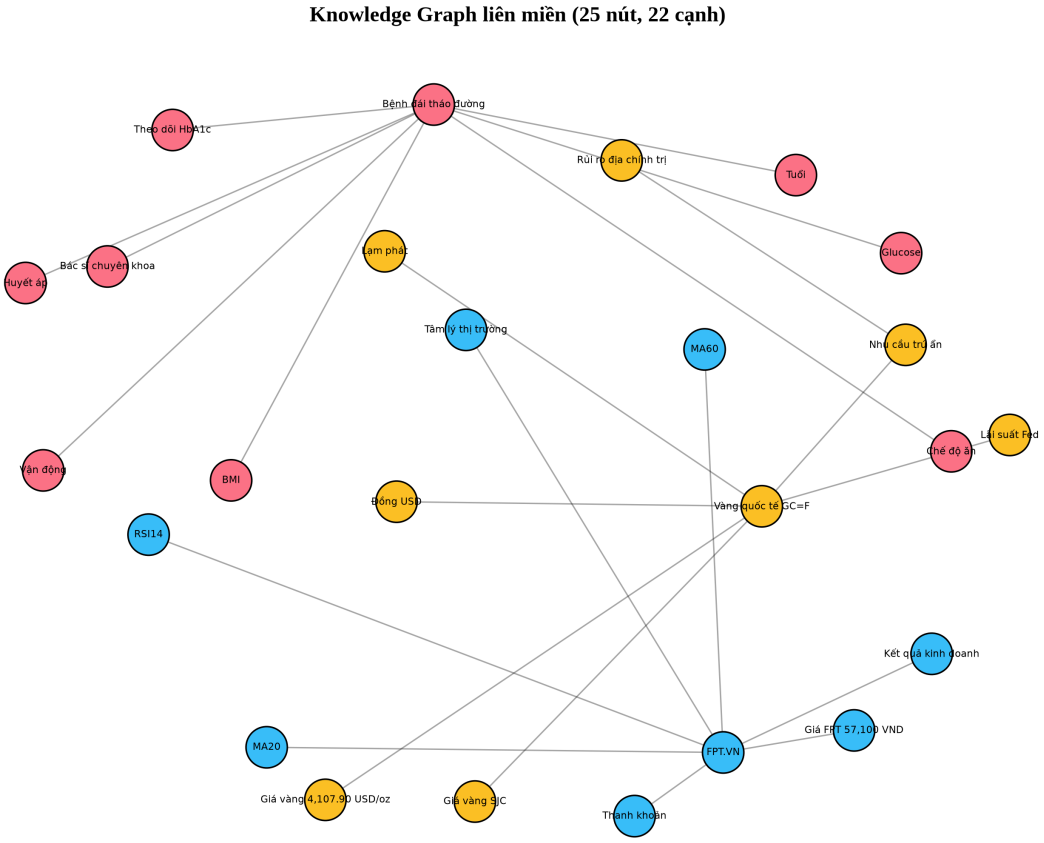
5.2. KNOWLEDGE GRAPH - KNOWLEDGE BASE

Knowledge Base trong bài gồm hai lớp. Lớp văn bản giữ các đoạn mô tả dữ liệu, phương pháp và kết quả theo từng miền. Lớp đồ thị giữ thực thể cùng các quan hệ có hướng. Hai lớp dùng chung nhãn domain để ngăn truy xuất nhầm giữa tiểu đường, cổ phiếu và vàng.

Đồ thị tại lần chạy này có 25 nút và 22 cạnh. Kích thước nhỏ giúp người đọc kiểm tra thủ công. Khi mở rộng, cùng schema có thể chuyển sang Neo4j hoặc một triple store mà không thay

đổi ý nghĩa bộ ba.

Knowledge Base không chỉ chứa kiến thức tĩnh. Các nút quan sát như giá FPT hoặc giá vàng gần nhất được tạo từ dữ liệu tải về. Cách làm này nối câu trả lời với đúng trạng thái thị trường tại thời điểm chạy.



Hình 5.3. Đồ thị tri thức đa miền

5.2.1. BỘ BA, THỰC THỂ VÀ QUAN HỆ

Một bộ ba có dạng (s, p, o), trong đó s là thực thể nguồn, p là quan hệ có tên và o là thực thể hoặc giá trị đích. Ví dụ: (MA20, MÔ_TẢ_XU_HƯỚNG_NGẮN_HẠN_CỦA, FPT.VN). Bộ ba này không khẳng định xu hướng sẽ tiếp diễn; nó chỉ diễn tả vai trò của chỉ báo.

Thực thể được gắn type như Condition, Biomarker, Stock, TechnicalIndicator, Commodity hoặc MacroFactor. Thuộc tính domain tách ba không gian nghiệp vụ. Tên quan hệ dùng động từ viết hoa để người đọc có thể đọc thành câu.

Việc chọn quan hệ là công đoạn có tính chủ quan. Báo cáo ưu tiên từ CÓ_THỂ_TÁC_ĐỘNG hoặc CÓ_QUAN_HỆ... thay vì GÂY_RA khi dữ liệu không đủ để chứng minh nhân quả.

5.2.2. KNOWLEDGE GRAPH CORE ENGINE

Lớp KnowledgeGraphCore bọc MultiDiGraph của NetworkX. add_entity quản lý nút và thuộc tính; add_relation thêm cạnh có hướng; neighbors trả về bằng chứng một bước; multi_hop tìm

các đường đơn trong giới hạn số hop. Cấu trúc MultiDiGraph cho phép hai thực thể có nhiều loại quan hệ.

Độ phức tạp của truy vấn lân cận tỷ lệ với bậc của nút. Truy vấn mọi đường đơn có thể tăng nhanh theo kích thước đồ thị, vì vậy max_hops phải được giới hạn. Với đồ thị sản xuất, cần chỉ mục, phân quyền và chiến lược truy vấn cụ thể thay vì duyệt toàn bộ.

Toàn bộ cạnh được xuất thành knowledge_graph_triples.csv. Đây là điểm kiểm tra thuận tiện: mỗi dòng trong tệp phải đọc được như một phát biểu, không có nút mồ côi vô nghĩa và không chứa con số không rõ ngày.

KnowledgeGraphCore: thực thể, quan hệ và multi-hop Tieuluan02_06_Nhom2_Giangkt.ipynb

```
class KnowledgeGraphCore:
    def __init__(self):
        self.graph = nx.MultiDiGraph()

    def add_entity(self, entity_id, entity_type="Entity", **properties):
        self.graph.add_node(entity_id, type=entity_type, **properties)

    def add_relation(self, subject, predicate, obj, weight=1.0, **properties):
        if subject not in self.graph: self.add_entity(subject)
        if obj not in self.graph: self.add_entity(obj)
        self.graph.add_edge(subject, obj, key=predicate,
                            relation=predicate, weight=float(weight), **properties)

    def multi_hop(self, start, max_hops=2):
        paths = []
        for target in self.graph.nodes:
            if target != start:
                paths.extend(nx.all_simple_paths(self.graph, start, target,
    cutoff=max_hops))
        return paths
```

Hình 5.4. Cấu trúc KnowledgeGraphCore

5.2.3. RDF VÀ LABELED PROPERTY GRAPH

RDF phù hợp khi cần URI toàn cục, liên kết dữ liệu và suy luận theo chuẩn W3C. Labeled Property Graph thuận tiện khi cạnh cần thuộc tính và truy vấn đường đi là trọng tâm. Bài chọn mô hình thứ hai vì mục tiêu là nguyên mẫu phân tích và trực quan hóa bằng Python.

Nếu chuyển sang RDF, các entity_id cần ánh xạ thành IRI; relation thành predicate; giá trị số thành literal có datatype. Nếu chuyển sang Neo4j, type có thể thành node label, relation thành relationship type và các trường domain/source thành property.

Không có mô hình nào tốt tuyệt đối. Tiêu chí chọn là khả năng tích hợp với hệ thống, yêu cầu suy luận, quy mô và kỹ năng vận hành của nhóm.

5.2.4. CHẤT LƯỢNG VÀ QUẢN TRỊ TRI THỨC

Một KG hữu ích cần hơn số lượng cạnh. Bốn kiểm tra tối thiểu gồm: tính duy nhất của định danh, tính hợp lệ của miền-phạm vi, nguồn gốc quan hệ và thời hạn của dữ liệu động. Các quan hệ y tế và tài chính phải có mức tin cậy hoặc câu chữ thể hiện độ bất định.

Bài hiện lưu source ở lớp tài liệu và domain ở lớp đồ thị. Bản sản xuất nên bổ sung created_at, valid_from, valid_to, confidence và evidence_id cho từng cạnh. Khi đó, một quan hệ đã cũ có thể bị loại trước khi đưa vào prompt.

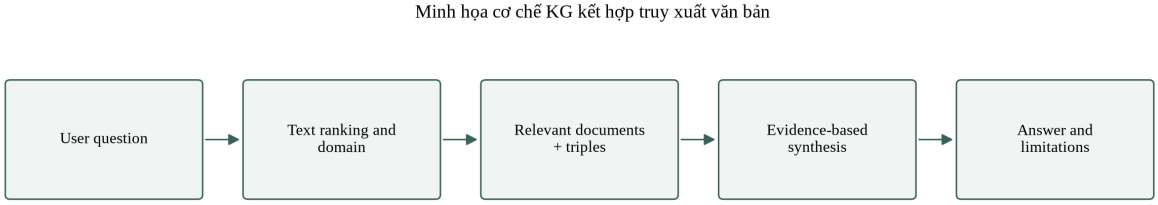
Kiểm soát chất lượng cần vòng phản hồi của chuyên gia. Bác sĩ xác nhận ontology y tế; chuyên viên tài chính rà soát khái niệm và đơn vị; kỹ sư dữ liệu kiểm tra schema cùng pipeline cập nhật.

5.3. RAG - RETRIEVAL-AUGMENTED GENERATION

Pipeline RAG gồm ingestion, indexing, retrieval và generation. Văn bản được chia thành các đơn vị có nguồn; TF-IDF biến chúng thành vector; cosine similarity xếp hạng theo câu hỏi. Bước sinh ghép hai đoạn phù hợp nhất với các bộ ba liên quan.

TF-IDF được chọn vì minh bạch, nhẹ và tái lập. Hạn chế là không hiểu đồng nghĩa sâu như embedding. Để giảm nhược điểm, câu hỏi được nhận diện miền bằng một từ điển alias và GraphRAG bổ sung các quan hệ từ đồ thị.

Mỗi câu trả lời trả về cả documents, score, triples và domain. Cấu trúc này quan trọng hơn phần câu chữ: giao diện hoặc báo cáo có thể trình bày bằng chứng mà không phải phân tích ngược chuỗi văn bản.



Hình 5.5. Minh họa luồng truy xuất và tổng hợp bằng chứng

5.3.1. NẠP, LẬP CHỈ MỤC VÀ TRUY XUẤT

Các tài liệu ngắn được xây từ mô tả bộ dữ liệu và kết quả thực nghiệm. Trong hệ thống lớn, ingestion cần đọc PDF, HTML hoặc cơ sở dữ liệu; chuẩn hóa Unicode; bỏ trùng; chia đoạn theo ngữ nghĩa; và gắn metadata trước khi embedding.

TfidfVectorizer dùng unigram và bigram. Bigram giúp phân biệt các cụm như giá vàng, tiểu đường, lãi suất Fed. top_k=3 giữ ngữ cảnh ngắn và dễ kiểm tra. Precision@3 theo domain được dùng như phép đo đơn giản cho độ đúng của truy xuất.

Điểm cosine chỉ phản ánh độ gần từ vựng, không phải độ đúng. Vì vậy báo cáo không diễn giải score như xác suất và không tự động loại bỏ nguồn chuyên môn chỉ vì score thấp.

VectorRetriever dùng TF-IDF và cosine similarity Tieuluan02_06_Nhom2_Giangkt.ipynb

```
class VectorRetriever:
    def __init__(self, docs):
        self.docs = docs
        self.vectorizer = TfidfVectorizer(
            lowercase=True, ngram_range=(1, 2), min_df=1,
            token_pattern=r"(?u)\b\w+\b"
        )
        self.matrix = self.vectorizer.fit_transform([d["text"] for d in docs])

    def search(self, query, top_k=3):
        query_vector = self.vectorizer.transform([query])
        scores = cosine_similarity(query_vector, self.matrix).ravel()
        order = np.argsort(scores)[::-1][:top_k]
        return [{**self.docs[i], "score": float(scores[i])} for i in order]
```

Hình 5.6. Lập chỉ mục và truy xuất TF-IDF

5.3.2. TRUY XUẤT ĐỒ THỊ VÀ SUY LUẬN NHIỀU BƯỚC

graph_evidence nhận diện miền, chọn các nút cùng domain hoặc trùng từ khóa, rồi lấy quan hệ vào và ra. Phương án này đủ cho đồ thị nhỏ. Khi đồ thị lớn, cần entity linking, tìm kiếm theo embedding của nhãn và truy vấn Cypher có giới hạn.

Suy luận nhiều bước được minh họa bởi chuỗi Rủi ro địa chính trị → Nhu cầu trú ẩn → Vàng quốc tế. Vector search có thể tìm một đoạn nói về địa chính trị nhưng không mặc nhiên biểu diễn hai quan hệ riêng. Đồ thị giúp giữ cấu trúc của lời giải thích.

Đường đi chỉ là bằng chứng cấu trúc, không tự chứng minh nhân quả. Mọi kết luận vẫn phụ thuộc chất lượng cạnh và nguồn đã dùng để tạo cạnh.

GraphRAG: ghép bằng chứng văn bản và bộ ba Tieuluan02_06_Nhom2_Giangkt.ipynb

```
def graphrag_answer(query, top_k=3):
    docs = retriever.search(query, top_k=top_k)
    triples = graph_evidence(query)
    domain = detect_domain(query)
    domain_docs = [d for d in docs if d["domain"] in (domain, "method")]
    statements = [d["text"] for d in domain_docs[:2]]
    graph_text = [f"{s} -[{p}]-> {o}" for s, p, o in triples[:3]]
    answer = " ".join(statements)
    if graph_text:
        answer += " Quan hệ đồ thị: " + "; ".join(graph_text)
    return {"answer": answer, "documents": domain_docs, "triples": triples}
```

Hình 5.7. Truy xuất bằng chứng đồ thị

5.3.3. TỔNG HỢP CÂU TRẢ LỜI CÓ CĂN CỨ

Bộ sinh trong bài là deterministic synthesizer. Nó chọn hai tài liệu phù hợp, nối tối đa ba quan hệ đồ thị và thêm cảnh báo theo miền. Kết quả ít tự nhiên hơn LLM nhưng có lợi thế là chạy offline và không tự thêm sự kiện ngoài bằng chứng.

Có thể thay bộ sinh bằng một LLM cục bộ hoặc dịch vụ API. Khi đó prompt phải tách rõ QUESTION, TEXT EVIDENCE, GRAPH EVIDENCE và yêu cầu từ chối nếu bằng chứng thiếu. Đầu ra nên ở dạng JSON để kiểm tra citation_id.

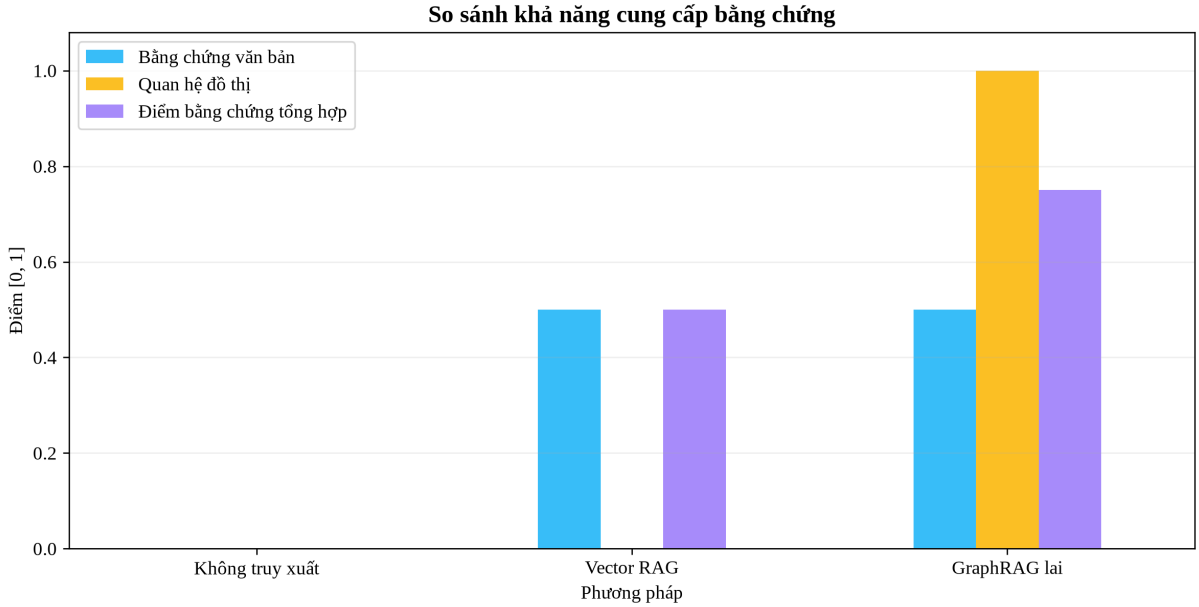
Thiết kế production cần chống prompt injection từ tài liệu, lọc dữ liệu nhạy cảm, theo dõi phiên bản chỉ mục và ghi log các nguồn đã đưa cho mô hình.

5.3.4. ĐÁNH GIÁ RAG VÀ GRAPHRAG

Bộ kiểm thử gồm sáu câu hỏi, hai câu cho mỗi miền. vector_precision@3 đo tỷ lệ tài liệu thuộc đúng miền trong ba kết quả đầu. graph_evidence ghi nhận hệ thống có tìm được cạnh liên quan hay không. Điểm tổng hợp cộng một phần thưởng nhỏ khi cả hai kênh đều có bằng chứng.

Ở lần chạy này, precision@3 trung bình là 0.500; tỷ lệ câu hỏi có bằng chứng đồ thị là 1.000; điểm bằng chứng lai là 0.750.

Đây là smoke test, chưa phải benchmark khoa học. Một đánh giá đầy đủ cần nhiều câu hỏi do người độc lập soạn, đáp án chuẩn, chấm faithfulness, answer relevance, citation correctness, độ trễ và chi phí.



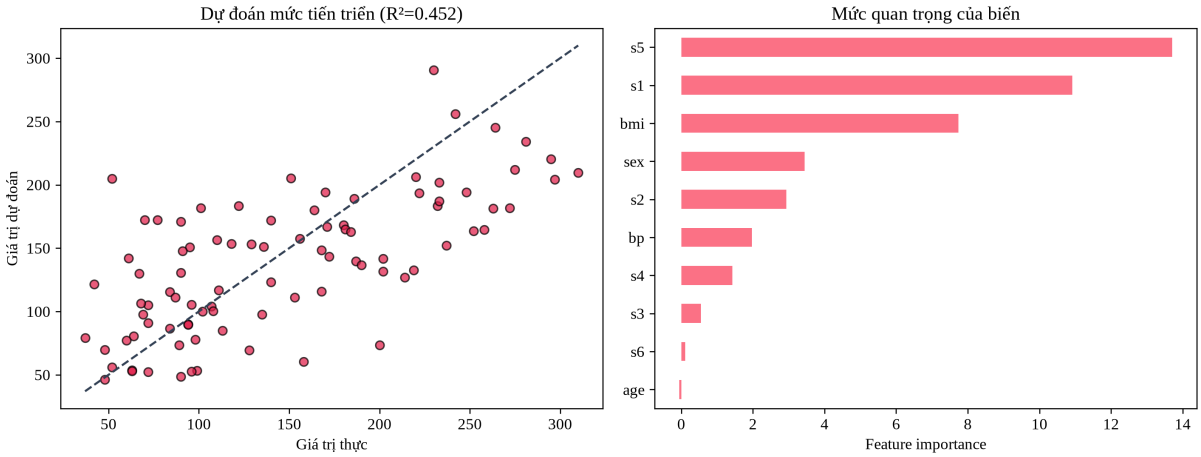
Hình 5.8. Đánh giá smoke test RAG và GraphRAG

5.4. KB VÀ RAG CHO BỆNH TIỂU ĐƯỜNG

Dữ liệu diabetes gồm 442 người đã mắc bệnh, 10 thuộc tính nền và điểm tiến triển sau một năm. Đầu ra là hồi quy nghiên cứu, không phải xác suất mắc tiểu đường.

Dữ liệu được chia thành 282 mẫu train, 71 mẫu hiệu chỉnh khoảng dự báo và 89 mẫu test. K-fold 5 phần chỉ chạy trên train; so sánh Median baseline, Linear, Ridge, ElasticNet, Huber, SVR RBF, Random Forest và Gradient Boosting. Thêm mô hình phi tuyến không đồng nghĩa chắc chắn tốt hơn; quyết định vẫn dựa trên CV MAE.

Mô hình được chọn theo CV: Ridge; CV MAE 44.44; test MAE 42.94; R2 0.452. MAE của dự báo trung vị là 62.28. Khoảng dự báo mục tiêu 90% có độ bao phủ test 86.5%.



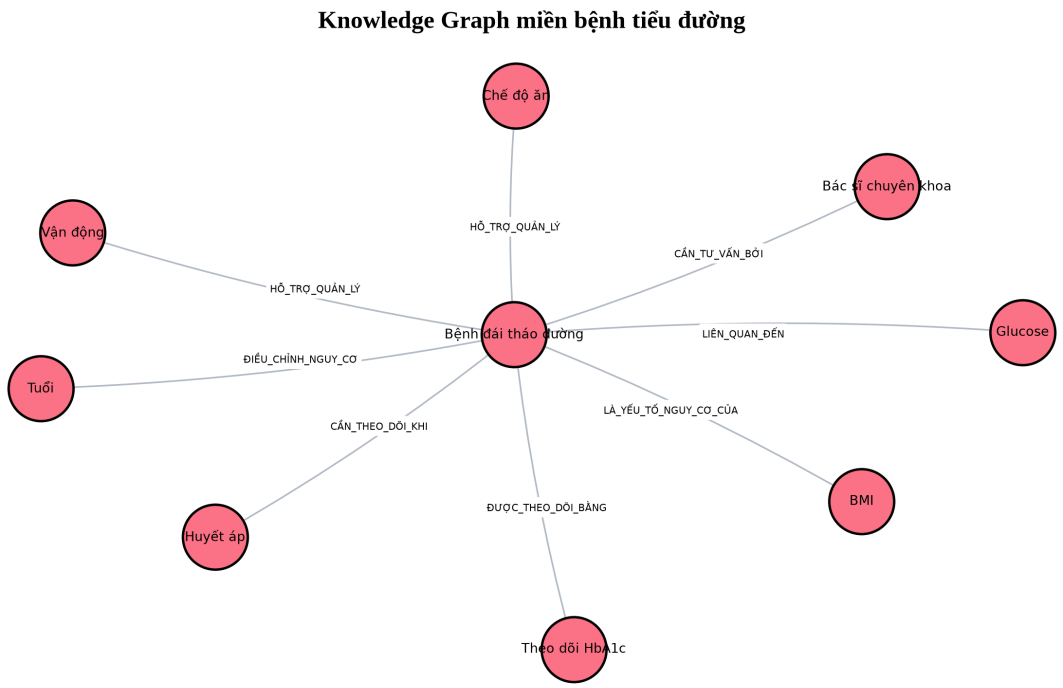
Hình 5.9. Mô hình hồi quy tiến triển tiểu đường

5.4.1. ONTOLOGY VÀ XỬ LÝ DỮ LIỆU Y TẾ

Ontology tối giản gồm Condition, Biomarker, RiskFactor, Lifestyle, Monitoring và Professional. Các quan hệ như LIÊN_QUAN_ĐẾN hoặc HỖ_TRỢ_QUẢN_LÝ được chọn để tránh biến một tương quan thành chỉ định điều trị.

Dữ liệu không có tên bệnh nhân hay định danh cá nhân. Trong tình huống thực, hồ sơ y tế cần mã hóa, phân quyền theo vai trò, nhật ký truy cập và quy trình chấp thuận. Không nên đưa thông tin nhận dạng vào dịch vụ embedding công cộng khi chưa có cơ sở pháp lý.

Mô hình Ridge được chọn theo MAE trung bình của 5 fold. Permutation importance được tính trên tập test để cùng một cách giải thích áp dụng được cho cả mô hình tuyến tính và mô hình cây.



Hình 5.10. Đồ thị tri thức y tế

5.4.2. THỰC NGHIỆM HỎI ĐÁP Y TẾ

Câu hỏi mẫu hỏi các yếu tố liên quan đến quản lý bệnh tiểu đường. Vector retriever tìm mô tả theo dõi đường huyết, HbA1c, huyết áp, cân nặng, ăn uống và vận động. Đồ thị bổ sung các cạnh nối bệnh với theo dõi và chuyên gia.

Câu trả lời sinh ra trong lần chạy: Theo dõi đường huyết, HbA1c, huyết áp, cân nặng, chế độ ăn và vận động là các thành phần thường gặp trong quản lý bệnh đái tháo đường; quyết định điều trị cần bác sĩ. Bộ dữ liệu diabetes của scikit-learn có 442 quan sát, 10 biến nền ở thang đo gốc và biến đích phản ánh mức tiến triển bệnh sau một năm. Quan hệ đồ thị liên quan: Bệnh đái tháo đường -[ĐƯỢC_THEO_DÕI_BẰNG]→ Theo dõi HbA1c; Bệnh đái tháo đường -[CẦN_TƯ_VẤN_BỞI]→ Bác sĩ chuyên khoa; Glucose -[LIÊN_QUAN_ĐẾN]→ Bệnh đái tháo đường. Kết quả chỉ có giá trị minh họa; cần tham khảo nhân viên y tế cho quyết định cá nhân.

Biểu đồ đối chiếu giá trị thực - dự đoán và feature importance cho thấy mức độ phù hợp cũng như biến nào được mô hình sử dụng nhiều nhất.

CV + calibration riêng cho mô hình sức khỏe Tieuluan02_06_Nhom2_Giangkt.ipynb

```
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=.2, random_state=42)
X_train, X_cal, y_train, y_cal = train_test_split(
    X_dev, y_dev, test_size=.2, random_state=42)
name, search, rows = select_model(
    diabetes_candidates, X_train, y_train,
    KFold(5, shuffle=True, random_state=42), "Tiểu đường")
model = search.best_estimator_
interval = calibrate(model, X_cal, y_cal, X_test, y_test)
pred = model.predict(X_test)
```

Hình 5.11. Huấn luyện và đánh giá mô hình y tế

5.4.3. GIỚI HẠN VÀ HƯỚNG MỞ RỘNG Y TẾ

Bộ dữ liệu nhỏ, không đại diện cho dân số Việt Nam và không chứa bối cảnh điều trị. Chỉ số đích mang ý nghĩa nghiên cứu, khác với chẩn đoán đái tháo đường trong thực hành. Vì vậy không thể suy rộng độ chính xác mô hình thành hiệu quả lâm sàng.

Hướng mở rộng là dùng dữ liệu đã khử định danh, xác định rõ biến kết cục, đánh giá theo nhóm tuổi/giới và mời chuyên gia lâm sàng kiểm tra ontology. Ngoài độ chính xác, cần đo calibration, fairness và khả năng giải thích.

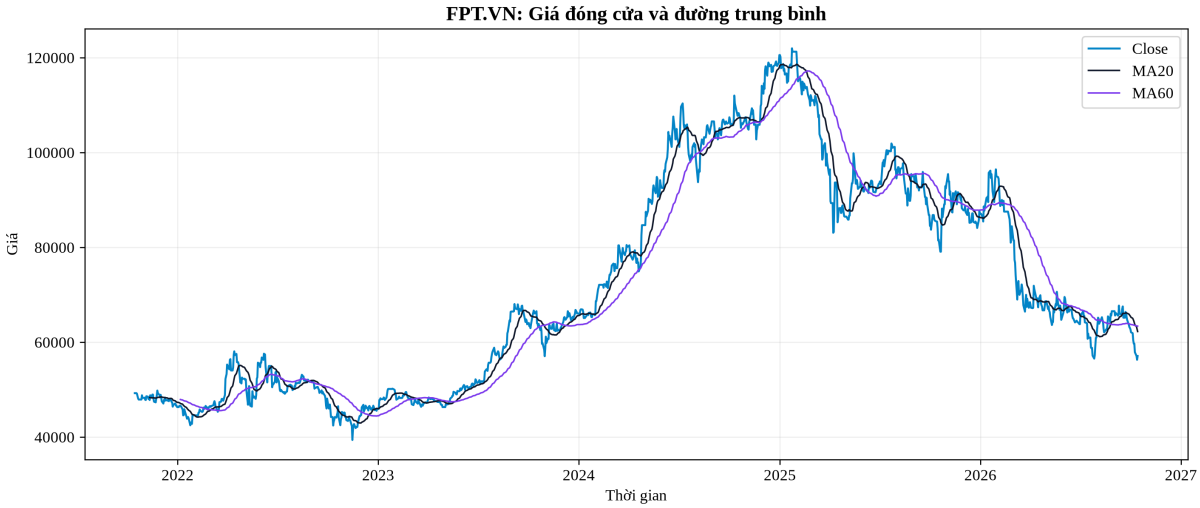
Với RAG y tế, mọi đoạn văn cần version và ngày hiệu lực. Hướng dẫn cũ phải được thu hồi khỏi chỉ mục thay vì chỉ thêm tài liệu mới.

5.5. KB VÀ RAG CHO CHỨNG KHOÁN

Dữ liệu FPT.VN được lấy từ Yahoo Finance chart API - FPT.VN. Khoảng thời gian 15/10/2021 - 14/10/2026 có 1304 quan sát. Giá đóng cửa gần nhất là 57,100 VND.

Lợi suất toàn kỳ là 15.91%, biến động năm hóa 27.63%, RSI14 8.2. Mỗi con số được gắn ngày chạy và nguồn dữ liệu.

Knowledge Graph nối FPT.VN với giá gần nhất, MA20, MA60, RSI, thanh khoản, kết quả kinh doanh và tâm lý thị trường. Các quan hệ mô tả vai trò chỉ báo, không tự động tạo khuyến nghị mua bán.



Hình 5.12. Chuỗi giá và chỉ báo FPT

5.5.1. DỮ LIỆU VÀ TIỀN XỬ LÝ FPT

Chuỗi OHLCV được sắp xếp tăng dần theo ngày; dòng thiếu Close bị loại. Return_1D, MA20, MA60, volatility20 và RSI14 được tính từ dữ liệu lịch sử. Đặc trưng bổ sung gồm thay đổi khối lượng, biên độ High-Low, chênh lệch Open-Close và các độ trễ một phiên.

Tại điểm cuối, MA20 = 62,260, MA60 = 63,396. Bộ học có 988 mẫu train và 85 mẫu test theo thứ tự thời gian.

Biến đích là giá Close của phiên kế tiếp. Việc dịch target một bước rồi drop dòng cuối giúp đặc trưng tại ngày t không chứa giá của ngày t+1.

Tải và tiền xử lý dữ liệu thị trường Tieuluan02_06_Nhom2_Giangkt.ipynb

```
def yahoo_chart(symbol, range_="5y", interval="1d"):
    url = f"https://query1.finance.yahoo.com/v8/finance/chart/{symbol}"
    response = requests.get(url, params={"range": range_, "interval": interval},
                            timeout=25, headers={"User-Agent": "Mozilla/5.0"})
    response.raise_for_status()
    result = response.json()["chart"]["result"][0]
    quote = result["indicators"]["quote"][0]
    frame = pd.DataFrame(quote, index=pd.to_datetime(result["timestamp"], unit="s"))
    frame.columns = [c.title() for c in frame.columns]
    return frame[["Open", "High", "Low", "Close", "Volume"]].dropna(subset=["Close"])

def add_indicators(df):
    out = df.copy().sort_index()
    out["Return_1D"] = out["Close"].pct_change()
    out["MA20"] = out["Close"].rolling(20).mean()
    out["MA60"] = out["Close"].rolling(60).mean()
    out["Volatility20"] = out["Return_1D"].rolling(20).std() * np.sqrt(252)
    return out
```

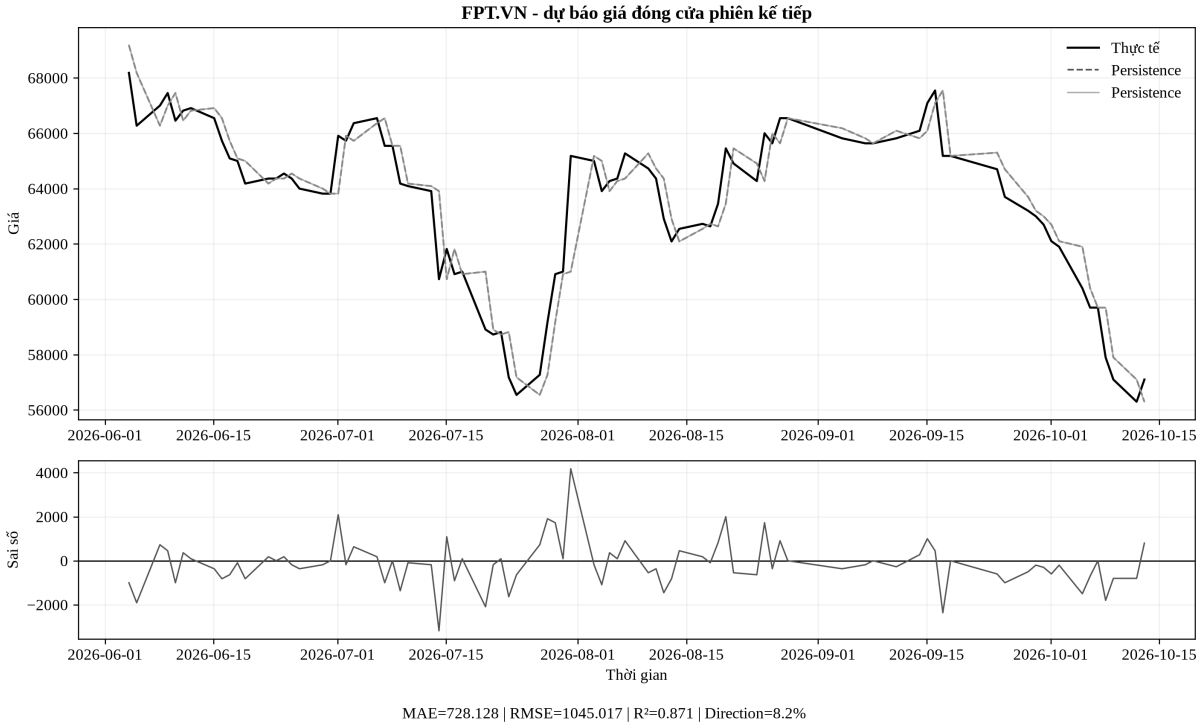
Hình 5.13. Pipeline xử lý dữ liệu thị trường

5.5.2. HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH FPT

Lịch sử tải được mở rộng từ 2 năm lên 5 năm. Chín họ cấu hình gồm Persistence, Ridge giá, Median lợi suất, Ridge lợi suất, Huber, SVR RBF, Random Forest và hai dạng Gradient Boosting. Tuning độ co lợi suất, cửa sổ học gần nhất và siêu tham số trên TimeSeriesSplit 5 phần, mỗi phần xác nhận 63 phiên, gap=1. Tập test giữ nguyên mốc của bản 2.0.

Chia theo thời gian: 988 train, 67 hiệu chỉnh, 85 test; bỏ một quan sát tại mỗi ranh giới. Mô hình Persistence đạt MAE 728.13, R2 0.871; baseline MAE 728.13. Cải thiện trên test 0.00%.

Test lịch sử từ 2026-06-04 đến 2026-10-13; MAE bản trước 728.13 VND. Bao phủ khoảng dự báo lịch sử: 97.6%. Kết quả hiện tại chưa cho thấy mô hình học máy thắng Persistence. Dữ liệu chuỗi thời gian không thỏa giả định trao đổi như mẫu độc lập nên mức 90% chỉ là mục tiêu, không phải bảo đảm cho tương lai.



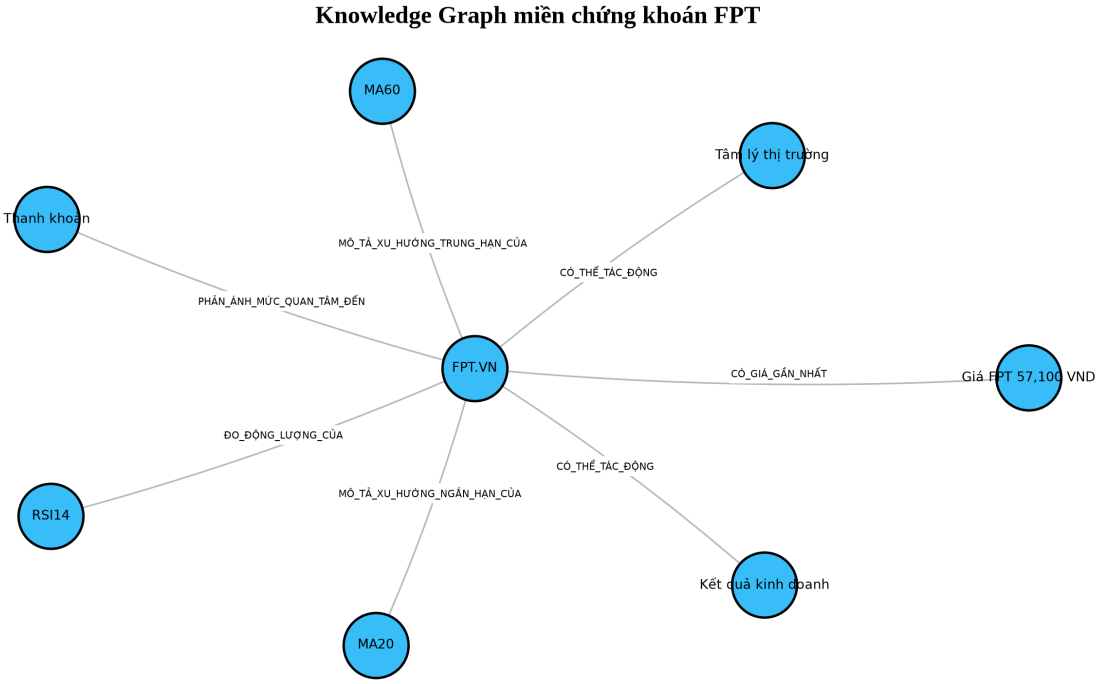
Hình 5.14. Dự báo FPT so với Persistence

5.5.3. HỎI ĐÁP CHỨNG KHOÁN VỚI GRAPHRAG

Câu trả lời mẫu: FPT.VN có MA20 62,260, MA60 63,396, RSI14 8.2 và biến động năm hóa 27.6%. Đường trung bình động mô tả xu hướng quá khứ chứ không bảo đảm giá tương lai. RSI là chỉ báo động lượng và cần kết hợp thanh khoản, cơ bản doanh nghiệp cùng bối cảnh thị trường. Quan hệ đồ thị liên quan: FPT.VN -[CÓ_GIÁ_GẦN_NHẤT]→ Giá FPT 57,100 VND; MA20 -[MÔ_TẢ_XU_HƯỚNG_NGẮN_HẠN_CỦA]→ FPT.VN; MA60 -[MÔ_TẢ_XU_HƯỚNG_TRUNG_HẠN_CỦA]→ FPT.VN. Đây là phân tích dữ liệu quá khứ, không phải khuyến nghị mua hoặc bán.

Tài liệu cung cấp số liệu động; đồ thị giải thích vai trò của chỉ báo và liên kết multi-hop. Bộ lọc domain ngăn tài liệu vàng hoặc y tế đi vào câu trả lời chứng khoán.

Kết quả mô hình được ghi thành tài liệu trong KB với model name, thời gian, MAE và R2. Vì vậy RAG có thể dẫn lại đúng lần huấn luyện thay vì tạo ra độ chính xác không tồn tại.



Hình 5.15. Đồ thị tri thức chứng khoán

5.5.4. RỦI RO VÀ HƯỚNG MỞ RỘNG

Corporate actions, thanh khoản, cách điều chỉnh giá và survivorship bias có thể làm sai phân tích. Yahoo Finance phù hợp bài tập nhưng nên đối chiếu nguồn chính thức nếu dùng trong nghiên cứu giao dịch.

Dự báo Close phiên kế tiếp thường có R2 cao do giá hôm nay gần giá ngày mai; vì vậy phải đọc cùng persistence baseline và directional accuracy. R2 cao không đồng nghĩa chiến lược có lợi nhuận.

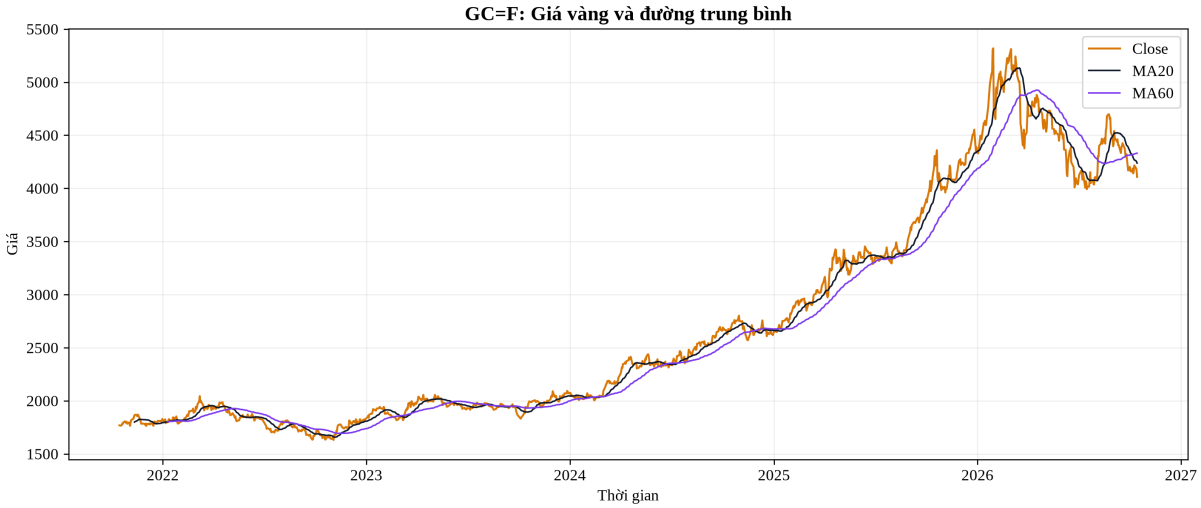
Hướng mở rộng là walk-forward backtest có phí, trượt giá, khoảng tin cậy và kiểm tra thay đổi chế độ thị trường. Mọi đầu ra vẫn cần cảnh báo không phải khuyến nghị đầu tư.

5.6. KB VÀ RAG CHO GIÁ VÀNG

Dữ liệu hợp đồng vàng GC=F được lấy từ Yahoo Finance chart API - GC=F. Khoảng 15/10/2021 - 14/10/2026 có 1257 quan sát; giá gần nhất 4,107.90 USD/oz.

Lợi suất toàn kỳ 132.31%, biến động năm hóa 19.07%, RSI14 31.6. GC=F là tham chiếu quốc tế, không đồng nhất với giá vàng miếng SJC.

Knowledge Graph mô tả lãi suất Fed, USD, lạm phát, địa chính trị và nhu cầu trú ẩn. Những cạnh này là khung giải thích có điều kiện, không phải quan hệ nhân quả cố định.



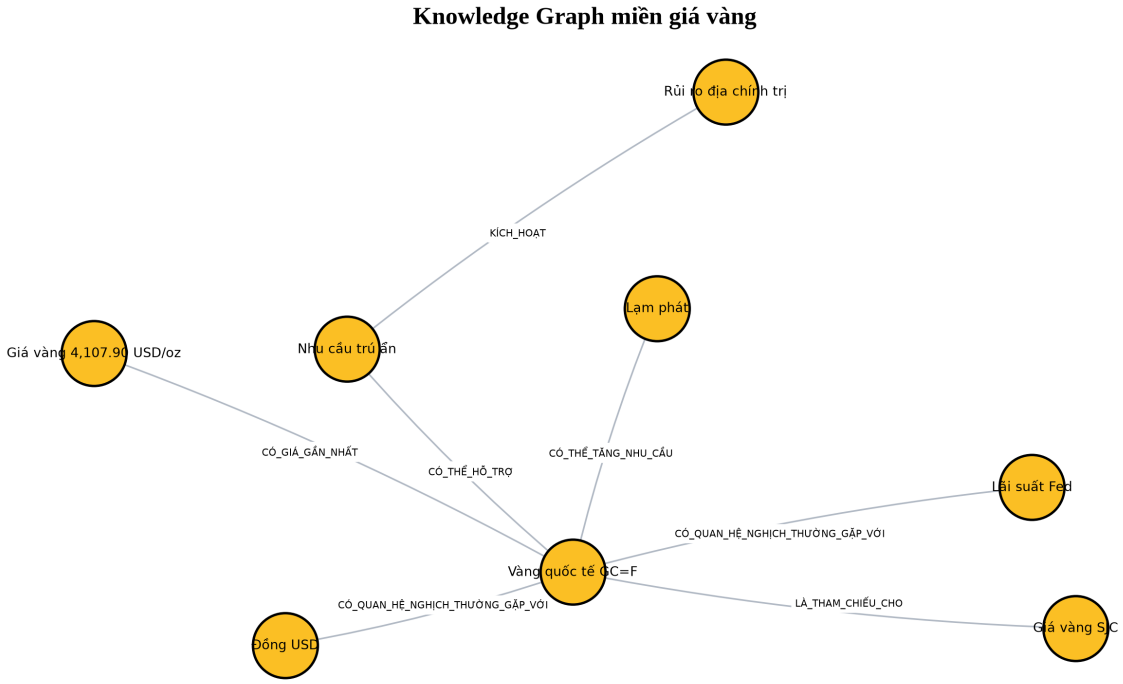
Hình 5.16. Chuỗi giá và chỉ báo vàng

5.6.1. DỮ LIỆU, CHỈ BÁO VÀ KNOWLEDGE GRAPH VÀNG

MA20 hiện là 4,237.78, MA60 là 4,330.82. Chuỗi giữ nguyên lịch phiên của nguồn; không nội suy ngày nghỉ. Biến đích là Close phiên kế tiếp.

Đặc trưng dùng cùng một công thức với FPT để so sánh công bằng: giá và độ trễ, return, MA20, MA60, volatility20, RSI14, thay đổi volume, biên độ ngày và chênh lệch Open-Close.

Chuỗi Rủi ro địa chính trị -> Nhu cầu trú ẩn -> Vàng quốc tế minh họa truy vấn nhiều bước. Quan hệ Vàng quốc tế -> Giá vàng SJC chỉ là tham chiếu, cần thêm tỷ giá và cơ chế thị trường trong nước để giải thích chênh lệch.



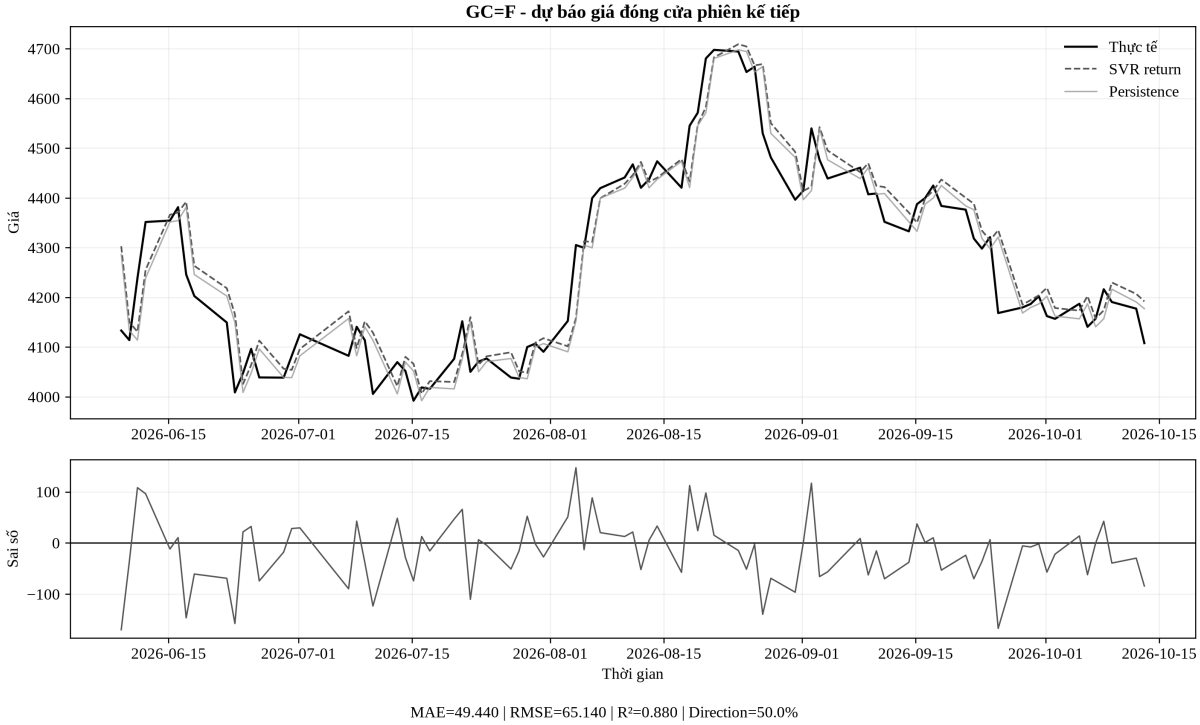
Hình 5.17. Đồ thị tri thức vàng

5.6.2. HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH VÀNG

Vàng dùng cùng quy trình lựa chọn có baseline và mô hình lợi suất như FPT. Không ép chọn mô hình phức tạp nếu Persistence có CV MAE thấp hơn. Train, hiệu chỉnh và test nằm theo đúng thứ tự thời gian, có khoảng cách một phiên ở ranh giới.

Mô hình được chọn: SVR return; CV MAE 38.792; test MAE 49.440; R2 0.880. Baseline MAE 48.267; cải thiện -2.43%.

Test từ 2026-06-09 đến 2026-10-13; bản trước có MAE 50.4701. Mức giảm sai số so với bản trước là 2.04%, nhưng vẫn cần đối chiếu baseline. Khoảng bootstrap khối 5 phiên, 2.000 lần lặp cho mức cải thiện so với baseline: -6.10% đến 2.68%. Khoảng chứa 0 chưa chứng minh cải thiện rõ ràng. GC=F không phải vàng SJC.



Hình 5.18. Mô hình vàng và baseline

5.6.3. HỎI ĐÁP VĨ MÔ VỚI GRAPHRAG

Câu trả lời mẫu: Giá vàng thường nhạy với lãi suất thực, sức mạnh đồng USD, kỳ vọng lạm phát, nhu cầu dự trữ và rủi ro địa chính trị; quan hệ có thể thay đổi theo từng giai đoạn. Chuỗi vàng GC=F gồm 1257 phiên từ 15/10/2021 đến 14/10/2026; giá gần nhất 4,107.90 USD/oz. Quan hệ đồ thị liên quan: Vàng quốc tế GC=F -[CÓ_GIÁ_GẦN_NHẤT]→ Giá vàng 4,107.90 USD/oz; Vàng quốc tế GC=F -[LÀ_THAM_CHIẾU_CHO]→ Giá vàng SJC; Lãi suất Fed -[CÓ_QUAN_HỆ_NGHỊCH_THƯỜNG_GẶP_VỚI]→ Vàng quốc tế GC=F. Đây là phân tích dữ liệu quá khứ, không phải khuyến nghị mua hoặc bán.

Truy xuất văn bản nêu điều kiện và độ bất định; đồ thị cung cấp các cạnh thường gặp và đường đi qua nhu cầu trú ẩn. Hai nguồn bằng chứng cùng xuất hiện giúp câu trả lời tránh tuyệt

đối hóa.

Nếu thay bộ sinh mẫu bằng LLM, prompt phải cấm tự tạo mục tiêu giá hoặc xác suất tăng giảm khi evidence không cung cấp. Citation ID và timestamp nên là trường bắt buộc trong JSON đầu ra.

TimeSeriesSplit có gap và mô hình baseline Tieuluan02_06_Nhom2_Giangkt.ipynb

```
train = data.loc[data.index < cal_start].iloc[:-1]
cal = data.loc[(data.index >= cal_start) & (data.index < test_start)].iloc[:-1]
test = data.loc[(data.index >= test_start) & (data.index < "2026-10-14")]
cv = TimeSeriesSplit(n_splits=5, gap=1, test_size=63)
name, search, rows = select_model(candidates, Xtr, ytr, cv, domain)
model = search.best_estimator_
interval = calibrate(model, Xcal, ycal, Xte, yte, relative=True)
pred = model.predict(Xte)
baseline = Xte["Close"].to_numpy()
```

Hình 5.19. Quy trình học máy cho thị trường

5.6.4. GIỚI HẠN VÀ HƯỚNG MỞ RỘNG

Hợp đồng tương lai có ngày đáo hạn và cơ chế nối chuỗi. Spot XAU/USD, futures GC=F và vàng SJC là ba sản phẩm khác nhau; không thể trộn USD/oz với VND/lượng nếu chưa quy đổi.

Mô hình hiện chưa dùng lãi suất thực, DXY, tỷ giá USD/VND hay vị thế ETF. Hướng mở rộng là gắn timestamp cho các yếu tố vĩ mô, thử độ trễ và lưu confidence cùng evidence trên mỗi cạnh.

Tương tự chứng khoán, dự báo một bước chỉ minh họa quy trình machine learning có kiểm soát. Nó không phải khuyến nghị mua, bán hay nắm giữ vàng.

5.7. DEPLOYMENT

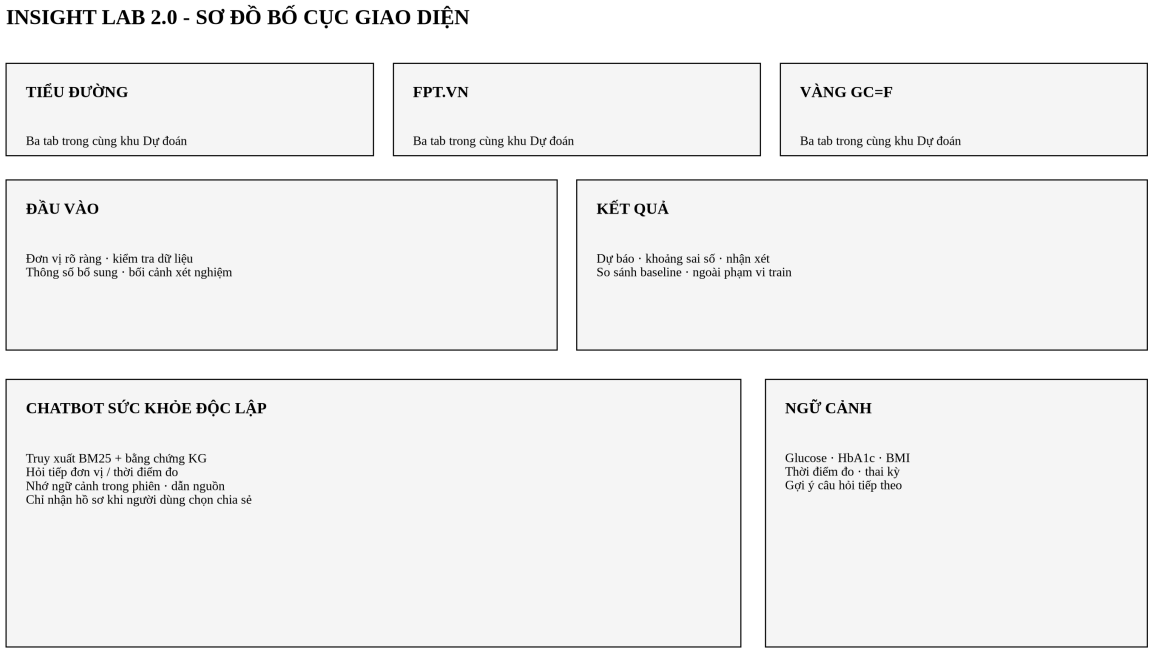
Insight Lab 2.0 chia thành khu Dự đoán với ba tab Tiểu đường, FPT, Vàng và khu Trợ lý sức khỏe riêng. Người dùng xem kết quả tại từng tab; khi muốn thảo luận, chủ động chọn chia sẻ kết quả sức khỏe cho chatbot.

Web hiển thị biểu đồ lịch sử giá, dự báo điểm, khoảng sai số hiệu chỉnh và so sánh baseline. Đầu vào trống, sai đơn vị hoặc vượt giới hạn được kiểm tra trước suy luận. Giá trị ngoài phạm vi train được đánh dấu để tránh hiểu sai mức tin cậy.

Web: `https://rag-httm-chatbot.vercel.app/`

GitHub: `https://github.com/jellalaz/HTTM_ThayQue`

Hình bố cục dưới đây là sơ đồ minh họa, không phải ảnh chụp màn hình.



Hình 5.20. Sơ đồ bố cục giao diện gốc

5.7.1. SUY LUẬN MÔ HÌNH TRÊN TRÌNH DUYỆT

Notebook xuất cấu trúc estimator được chọn: scaler, hệ số tuyến tính, vector hỗ trợ và kernel RBF của SVR, cấu trúc cây hoặc công thức Persistence/lợi suất. JavaScript dùng cùng đặc trưng và tham số. Các vector đối chiếu Python/JavaScript được lưu để kiểm tra sai số đóng gói.

Mô hình sức khỏe có sáu trường chính và bốn trường bổ sung. Trường bổ sung trống dùng trung vị tập train và được công bố ở kết quả. HbA1c và bối cảnh đo dùng cho nhận xét xét nghiệm riêng, không bị đưa nhầm vào mô hình chưa học chúng. Điểm tiến triển không được dùng quyết định chẩn đoán hoặc cấp cứu.

Cấu hình thị trường được chọn trên dữ liệu trước test rồi fit lại cho web, chừa 63 phiên gần nhất để hiệu chỉnh khoảng dự báo và bỏ cách một phiên. Chỉ số test lịch sử thuộc bản fit trước đó, không phải đánh giá ngoài mẫu của bản refit. Web hiển thị riêng hai giai đoạn. Cảnh báo ngoài miền của mô hình lợi suất xét đặc trưng tương đối, không chỉ mức giá tuyệt đối. Web không gửi hồ sơ sức khỏe ra ngoài.

Xuất estimator và web tự chứa Tieuluan02_06_Nhom2_Giangkt.ipynb

```
web_config["models"]["diabetes"] = {
    "estimator": export_estimator(diabetes_model),
    "features": list(X.columns),
    "metrics": diabetes_metrics,
    "interval": diabetes_interval,
}
model_json = json.dumps(web_config, ensure_ascii=False, allow_nan=False)
html = html_template.replace("__MODEL_CONFIG__", model_json)
(WEB_DIR / "index.html").write_text(html, encoding="utf-8")
```

Hình 5.21. Xuất estimator sang JavaScript

5.7.2. CHATBOT DỰ ĐOÁN, TƯ VẤN VÀ HỎI ĐÁP

Chatbot nhận đường huyết, HbA1c, BMI, tuổi, huyết áp tâm thu/tâm trương, huyết áp trung bình, cholesterol toàn phần, LDL, HDL, triglyceride và các đặc trưng nghiên cứu s4/s5. Câu 'huyết áp 90' được hỏi lại loại số đo; thiếu đơn vị thì hỏi tiếp. Một tin nhắn có thể chứa nhiều chỉ số, đơn vị được gắn riêng với từng giá trị để tránh lấy nhầm mmol/L của HDL cho glucose.

Các ngưỡng xét nghiệm tham khảo NIDDK/ADA. Đường huyết thấp được xử lý trước bảng phân loại đường cao. Sau bữa ăn thông thường không bị đánh đồng với nghiệm pháp 75 g glucose. Thai kỳ và trẻ em cần bối cảnh riêng. Câu hỏi kê liều thuốc được chuyển thành hướng dẫn trao đổi với bác sĩ.

Kho tri thức mở rộng thành 20 chủ đề, bổ sung nguồn AHA về huyết áp, MedlinePlus về mỡ máu và Mayo Clinic về xét nghiệm cholesterol. Nội dung, đường quan hệ KG và danh mục nguồn được xuất ra health_knowledge.json. BM25 truy xuất kết hợp bộ nhận diện số đo; chưa kết nối LLM ngoài. Khi chia sẻ form, chatbot nhận đủ các trường đã nhập, nhưng loại giá trị trung vị thay thế khỏi nhận xét y khoa. Mã giới tính 1/2 không tự dùng chọn ngưỡng HDL; s5 không tự quy đổi thành xét nghiệm triglyceride. Hồ sơ chỉ giữ trong phiên.

5.8. Đánh giá tổng hợp

Kết quả cho thấy một notebook có thể bao quát tải dữ liệu, tiền xử lý, KG, RAG, đánh giá, báo cáo và web. GraphRAG lai cung cấp thêm loại bằng chứng mà Vector RAG không có: quan hệ có tên và đường đi giữa các thực thể.

Đổi lại, KG yêu cầu thiết kế ontology, kiểm soát quan hệ và cập nhật theo thời gian. Chi phí này chỉ đáng bỏ ra khi bài toán cần giải thích, liên kết đa bước hoặc quản trị tri thức dài hạn.

Bài làm đạt mục tiêu nguyên mẫu tái lập. Hạn chế chính là corpus nhỏ, đánh giá ít câu hỏi, generator dạng mẫu và chưa có chuyên gia miền thẩm định. Những giới hạn này được ghi rõ thay vì suy rộng kết quả.

5.8.1. Kết quả huấn luyện ba miền

Bảng 5.1. Tổng hợp kết quả giữ từ Tiểu luận 02

Miền	Mô hình	CV MAE	Test MAE	R2
diabetes	Ridge	44.443	42.936	0.452
stock	Persistence	1215.974	728.128	0.871
gold	SVR return	38.792	49.440	0.880

Mô hình được chọn theo CV MAE trên tập train. Chỉ số test được tính trên phần dữ liệu giữ lại.

5.8.2. Kết quả truy xuất và bộ ba tri thức

Bảng 5.2. Kết quả smoke test được giữ nguyên

Câu hỏi	Miền	P@3	Graph	Hybrid
Theo dõi gì khi quản lý bệnh tiểu đường?	diabetes	0.67	1	0.92
Mô hình tiểu đường có sai số bao nhiêu?	diabetes	0.33	1	0.58
Giá FPT gần nhất và RSI là bao nhiêu?	stock	0.67	1	0.92
MA20 của cổ phiếu có ý nghĩa gì?	stock	0.33	1	0.58
Giá vàng gần nhất là bao nhiêu?	gold	0.33	1	0.58
Fed, USD và địa chính trị ảnh hưởng vàng ra sao?	gold	0.67	1	0.92

Bảng này là phép kiểm tra hồi quy cho pipeline. Nó không thay thế đánh giá câu trả lời bởi chuyên gia.

Bảng 5.3. Bộ ba tri thức của hệ thống

Subject	Predicate	Object
Bệnh đái tháo đường	ĐƯỢC_THEO_DÕI_BẰNG	Theo dõi HbA1c
Bệnh đái tháo đường	CẦN_TƯ_VẤN_BỞI	Bác sĩ chuyên khoa
Glucose	LIÊN_QUAN_ĐẾN	Bệnh đái tháo đường
BMI	LÀ_YẾU_TỐ_NGUY_CƠ_CỦA	Bệnh đái tháo đường
Huyết áp	CẦN_THEO_DÕI_KHI	Bệnh đái tháo đường
Tuổi	ĐIỀU_CHỈNH_NGUY_CƠ	Bệnh đái tháo đường
Chế độ ăn	HỖ_TRỢ_QUẢN_LÝ	Bệnh đái tháo đường
Vận động	HỖ_TRỢ_QUẢN_LÝ	Bệnh đái tháo đường

Subject	Predicate	Object
FPT.VN	CÓ_GIÁ_GẦN_NHẤT	Giá FPT 57,100 VND
MA20	MÔ_TẢ_XU_HƯỚNG_NGẮN_HẠN_CỦA	FPT.VN
MA60	MÔ_TẢ_XU_HƯỚNG_TRUNG_HẠN_CỦA	FPT.VN
RSI14	ĐO_ĐỘNG_LƯỢNG_CỦA	FPT.VN
Thanh khoản	PHẢN_ÁNH_MỨC_QUAN_TÂM_ĐẾN	FPT.VN
Kết quả kinh doanh	CÓ_THỂ_TÁC_ĐỘNG	FPT.VN
Tâm lý thị trường	CÓ_THỂ_TÁC_ĐỘNG	FPT.VN
Vàng quốc tế GC=F	CÓ_GIÁ_GẦN_NHẤT	Giá vàng 4,107.90 USD/oz
Vàng quốc tế GC=F	LÀ_THAM_CHIẾU_CHO	Giá vàng SJC
Lãi suất Fed	CÓ_QUAN_HỆ_NGHỊCH_THƯỜNG_GẶP_VỚI	Vàng quốc tế GC=F
Đồng USD	CÓ_QUAN_HỆ_NGHỊCH_THƯỜNG_GẶP_VỚI	Vàng quốc tế GC=F
Lạm phát	CÓ_THỂ_TĂNG_NHU_CẦU	Vàng quốc tế GC=F
Rủi ro địa chính trị	KÍCH_HOẠT	Nhu cầu trú ẩn
Nhu cầu trú ẩn	CÓ_THỂ_HỖ_TRỢ	Vàng quốc tế GC=F

5.8.3. Chunking, embedding và giới hạn đánh giá

Embedding là biểu diễn văn bản bằng vector; dense embedding học ngữ nghĩa từ dữ liệu huấn luyện, còn TF-IDF là vector thưa theo tần suất từ. Vector database bổ sung chỉ mục và quản trị truy vấn ở quy mô lớn; prototype hiện lưu ma trận TF-IDF trong bộ nhớ, chưa vận hành một vector database độc lập. Corpus gồm các tài liệu ngắn đã là đơn vị truy xuất, nên không tạo hàng loạt chunk chồng lấn không cần thiết. Với PDF dài, cần chunk theo heading, giữ metadata và thử kích thước/overlap bằng một tập câu hỏi riêng.

Điểm hybrid cũ cộng 0,25 khi có bằng chứng đồ thị là điểm kỹ thuật tự định nghĩa, không phải Accuracy, faithfulness hay xác suất đúng của câu trả lời. Precision theo domain chỉ đo mức đúng miền. Việc mọi câu hỏi tìm được một cạnh không chứng minh cạnh đó đủ để trả lời. Cần phân biệt truy xuất văn bản, độ đúng đường đi và độ bám sát bằng chứng khi đánh giá generator.

5.8.4. Đồng bộ notebook và các chức năng web bổ sung

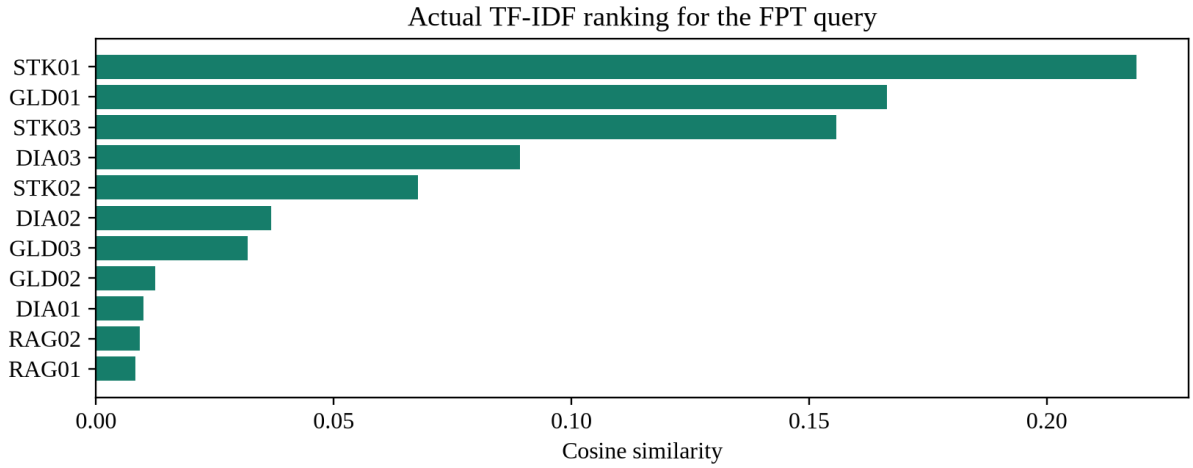
main.ipynb chứa các bước thực nghiệm mới, pipeline KG/RAG từ Tiểu luận 02, xuất mô hình và đóng gói web. Chế độ full là cấu hình của số liệu trong báo cáo; quick dùng ít dữ liệu/epoch hơn để kiểm tra, không thay số liệu full trong báo cáo. Manifest lưu seed, phiên bản thư viện,

hash dataset, checkpoint và cấu hình chạy.

Giao diện Insight Lab giữ khu dự đoán ba miền, trợ lý sức khỏe, màu sắc và bố cục chính. Khu thực nghiệm bổ sung giá nhà, phân loại nhãn nguy cơ tổng hợp, nhận dạng chữ số CNN và dự báo chuỗi. Các mô hình mới sử dụng API /api/predict và trọng số JSON xuất từ notebook. API không gọi dịch vụ LLM, không cần secret và có kiểm tra kích thước cũng như giá trị đầu vào. Chatbot sức khỏe cũ vẫn xử lý trong trình duyệt; dữ liệu nhập vào các demo API được gửi tới máy chủ inference để tính dự đoán.

5.8.5. Theo dõi một câu hỏi qua các bước truy xuất

Xét câu hỏi "Giá FPT gần nhất và RSI là bao nhiêu?". Câu hỏi gồm một thực thể FPT và hai nhu cầu thông tin: giá quan sát gần nhất, chỉ báo RSI. Tìm đúng miền stock mới là bước đầu; muốn trả lời đủ cần các tài liệu hoặc thuộc tính cung cấp cả hai giá trị và ngày dữ liệu. Prototype không dùng điểm số tương đồng để tự sinh thêm một giá hoặc RSI chưa tồn tại trong nguồn.



Hình 5.22. Thứ hạng TF-IDF thực tế cho câu hỏi FPT trên toàn corpus

Bảng 5.4. Năm tài liệu đứng đầu của ví dụ truy xuất

Hạng	Tài liệu	Miền	Cosine
1	STK01	stock	0.218913
2	GLD01	gold	0.166334
3	STK03	stock	0.155685
4	DIA03	diabetes	0.089192
5	STK02	stock	0.067704

Kết quả này cũng bộc lộ một lỗi truy xuất: GLD01 thuộc miền vàng đứng thứ hai vì có nhiều từ chỉ dữ liệu thị trường gần với câu hỏi, trong khi STK02 chứa thông tin chỉ báo FPT chỉ đứng thứ năm. Top-3 vì vậy chưa cung cấp đủ bằng chứng cho phần RSI. Nếu chỉ ghép ba đoạn đầu, câu trả lời không được tự bù một giá trị RSI. Lọc theo thực thể/miền hoặc tách câu hỏi thành hai

nhu cầu truy xuất là hướng sửa cần kiểm tra trên tập câu hỏi riêng, không được coi là cải thiện đã đo trong lần chạy này.

Corpus thực nghiệm có 11 tài liệu và 508 đặc trưng TF-IDF unigram/bigram. Query được transform bằng cùng vocabulary đã fit trên corpus rồi tính cosine với từng dòng ma trận. Top-3 chỉ là một ngân sách truy xuất; tài liệu thứ tư không trở thành sai về nội dung chỉ vì bị cắt khỏi ngữ cảnh. Các điểm trong hình là kết quả tính thật, không phải xác suất tài liệu đúng hoặc mức tin cậy của câu trả lời.

Bằng chứng STK01: Chuỗi FPT.VN gồm 1304 phiên từ 15/10/2021 đến 14/10/2026; giá đóng cửa gần nhất 57,100 VND.

Bằng chứng GLD01: Chuỗi vàng GC=F gồm 1257 phiên từ 15/10/2021 đến 14/10/2026; giá gần nhất 4,107.90 USD/oz.

Bằng chứng STK03: Đường trung bình động mô tả xu hướng quá khứ chứ không bảo đảm giá tương lai. RSI là chỉ báo động lượng và cần kết hợp thanh khoản, cơ bản doanh nghiệp cùng bối cảnh thị trường.

Nhánh đồ thị nhận diện miền và lấy các bộ ba liên quan để bổ sung quan hệ giữa mã tài sản, chỉ báo và yếu tố tác động. NetworkX lưu đỉnh/cạnh trong bộ nhớ; không có bước suy diễn tất cả đường đi hoặc xếp hạng cộng đồng đồ thị quy mô lớn. Generator hiện ghép văn bản theo cấu trúc xác định. Do đó cách gọi GraphRAG trong prototype chỉ mô tả việc bổ sung bằng chứng đồ thị, không đồng nhất với một hệ thống GraphRAG dùng LLM để trích xuất và tổng hợp cộng đồng.

Trong CSV bộ ba thật, FPT.VN nối với "Giá FPT 57,100 VND" qua quan hệ CÓ_GIÁ_GẦN_NHẤT; RSI14 nối tới FPT.VN qua ĐO_ĐỘNG_LƯỢNG_CỦA. Cạnh thứ hai chỉ xác định RSI14 đo động lượng của FPT, không chứa giá trị RSI số. Vì thế nhánh đồ thị có thể bổ sung quan hệ nhưng chưa đủ thay tài liệu STK02 bị bỏ khỏi top-3. Cần phân biệt bằng chứng về một quan hệ với bằng chứng về một thuộc tính định lượng tại một thời điểm.

5.8.6. Hai luồng hỏi đáp và giới hạn của câu trả lời

Notebook truy xuất trên corpus đa miền ngắn, trong đó một phần tài liệu thị trường được tạo từ snapshot đã xử lý. Chatbot trên web lại là luồng sức khỏe có corpus riêng, BM25, thực thể và trạng thái đối thoại. Hai luồng dùng chung ý tưởng tìm bằng chứng trước khi trả lời nhưng không dùng cùng chỉ mục, cùng số tài liệu hoặc cùng điểm đánh giá. Không lấy Precision@3 của notebook làm độ chính xác cho chatbot web.

Trong web, câu hỏi được chuẩn hóa để tìm chủ đề; lịch sử đối thoại giữ ngữ cảnh khi cần hỏi thêm đơn vị hoặc mục đích. Luật kiểm tra đơn vị chạy trước các nhánh cần diễn giải chỉ số. Khi tìm được chủ đề, phần trả lời đưa thông tin tham khảo, yếu tố liên quan và cảnh báo phạm vi. Khi thiếu dữ liệu hoặc câu hỏi vượt khả năng, hệ thống phải nêu thiếu thông tin thay vì tạo một kết luận chắc chắn.

Tình huống kiểm tra có thể bắt đầu bằng câu hỏi về HbA1c, sau đó hỏi tiếp về một chỉ số glucose chưa có đơn vị để quan sát yêu cầu làm rõ. Đây là kịch bản tương tác, không phải một

hồ sơ bệnh nhân hoặc bằng chứng về năng lực chẩn đoán. Nội dung sức khỏe được giới hạn ở giáo dục và giải thích giao diện; không dùng dự đoán của classifier tổng hợp để thay lời khuyên chuyên môn.

Demo hỏi đáp: `https://rag-httm-chatbot.vercel.app/#chatbot`

5.8.7. Phân biệt ba miền dự đoán gốc và các mô hình bổ sung

Khu dự đoán gốc có ba miền: hồi quy tiến triển bệnh trên bộ diabetes 442 mẫu, dự báo FPT và dự báo vàng với đặc trưng thị trường. Hồi quy y tế trả một thang tiến triển bệnh của dataset, không phải nhãn High/Low/Moderate của 50.000 mẫu tổng hợp ở Chương 2. Hai mô hình thị trường gốc sử dụng đặc trưng được dựng từ lịch sử, trong khi Chương 4 sử dụng 30 log-return và checkpoint recurrent riêng. Giao diện giữ cả hai để đối chiếu phương pháp, không hợp nhất kết quả của chúng thành cùng một phép thử.

Để dùng khu gốc, chọn miền, kiểm tra nhãn và đơn vị từng trường rồi chạy dự đoán. Các giá trị mặc định của thị trường dựa trên snapshot đi kèm cấu hình. Nếu sửa đặc trưng thủ công, người dùng đang tạo kịch bản; không được hiểu đầu ra là cập nhật giá thời gian thực. Phần KG bổ trợ cách đọc yếu tố và quan hệ; nó không tự sửa sai số của regressor hoặc tạo chứng cứ cho một dự báo chưa có nhãn thực.

Demo ba miền gốc: `https://rag-httm-chatbot.vercel.app/#predict`

5.8.8. Đường triển khai, kiểm thử và khả năng tái lập

Mô hình mới được huấn luyện trong notebook, lưu checkpoint và xuất tham số sang JSON nén. Vercel nạp bundle để chạy suy luận bằng Python standard library, không train lại theo yêu cầu người dùng. Rừng cây giữ ngưỡng và lá; CNN giữ kernel/dense; recurrent giữ trọng số cổng và tham số scale. Các tham số tiền xử lý được xuất cùng mô hình để tránh dùng một phép biến đổi khác giữa notebook và web.

Đối chiếu suy luận không chỉ kiểm tra nhãn cuối: 30 vector bảng so sánh giá hoặc điểm theo lớp, sáu ảnh so sánh toàn bộ logits, sáu checkpoint recurrent so sánh giá dự đoán. Sai lệch số thực được kiểm tra với ngưỡng nhỏ trong test tự động. Trường hợp float32 của cây và thứ tự cổng GRU được xử lý theo đúng thư viện train; đây là những chi tiết có thể gây sai kết quả dù giao diện vẫn hoạt động bình thường.

Kiểm thử âm gồm ảnh trống, sai số pixel, NaN/Infinity, giá trị số sai kiểu, chuỗi thiếu phần tử và miền không tồn tại. Kiểm thử trình duyệt xác nhận thao tác chọn ảnh, chạy API, đổi miền và trạng thái lỗi. Các kiểm tra này chứng minh đường xử lý thực thi đúng các trường hợp đã thử, không thay thế đánh giá thống kê ngoài mẫu. Mọi lần thay đổi trọng số hoặc preprocessing cần xuất bundle và chạy lại đối chiếu.

Mỗi biểu đồ phân tích mới có danh sách tệp đầu vào và SHA-256 trong figure_lineage.json. Các CSV prediction, lịch sử train và phép chia được giữ để tính lại bảng kết quả. Các sơ đồ có nhãn minh họa cơ chế chỉ giải thích luồng tính toán; chúng không được xem là ảnh dữ liệu thu

thập. Cách tổ chức này cho phép kiểm tra báo cáo từ kết quả số trở về mã và dữ liệu thay vì chỉ dựa vào mô tả bằng lời.

Web: `https://rag-httm-chatbot.vercel.app/`

GitHub: `https://github.com/jellalaz/HTTM_ThayQue`

5.9. Kết luận chương

Điều quan trọng nhất rút ra là không nên coi GraphRAG như một nhãn công nghệ thay thế cho thiết kế dữ liệu. Chất lượng câu trả lời bắt đầu từ việc định nghĩa thực thể, quan hệ, nguồn và phạm vi kết luận. Notebook và web dùng chung artifact để đối chiếu kết quả.

Chương 6

Học sâu trên đồ thị

Từ bảng dữ liệu đến cấu trúc liên kết

Trong dữ liệu bảng, mỗi hàng thường được xử lý bằng cùng một hàm dự đoán. Trong đồ thị, đặc trưng của một nút còn phụ thuộc vào những nút liên quan. Một bệnh, một xét nghiệm và một tài liệu có thể liên kết bằng các quan hệ khác nhau. Mạng nơ-ron đồ thị (GNN) học biểu diễn bằng cách trao đổi và tổng hợp thông tin dọc theo các cạnh. Chương này bổ sung nền tảng lý thuyết cho phần đồ thị tri thức; demo hiện dùng truy vấn Neo4j và chưa huấn luyện GNN.

Đặt đồ thị $G = (V, E)$ với n nút, ma trận đặc trưng $X \in \mathbb{R}^{n \times d}$ và ma trận kề A. Với đồ thị có hướng, A_{ij} và A_{ji} có thể khác nhau. Với đồ thị tri thức, một cạnh còn mang loại quan hệ và nguồn bằng chứng. Nếu bỏ các thuộc tính này để tạo đồ thị vô hướng, ta đã thay đổi ý nghĩa dữ liệu; phép chuyển đổi phải được công bố trong thiết kế thí nghiệm.

Cơ chế truyền thông điệp

Một tầng truyền thông điệp có thể viết dưới dạng:

$$m_v^{(l)} = \mathrm{AGG}_{u \in \mathcal{N}(v)} M_l(h_v^{(l)}, h_u^{(l)}, e_{uv}), \qquad h_v^{(l+1)} = U_l(h_v^{(l)}, m_v^{(l)}).$$

Hàm tổng hợp phải không phụ thuộc thứ tự liệt kê các nút láng giềng. Tổng, trung bình và cực đại là các lựa chọn thông dụng. Hàm cập nhật kết hợp thông tin của chính nút với thông tin từ láng giềng. Sau hai tầng, một nút có thể nhận thông tin từ vùng lân cận hai bước, nhưng điều đó không đồng nghĩa với suy luận logic hai bước có bảo đảm.

Trong một GCN, thêm cạnh tự nối rồi chuẩn hóa theo bậc:

$$\widetilde{A} = A + I, \quad \widetilde{D}_{ii} = \sum_j \widetilde{A}_{ij}, \qquad H^{(l+1)} = \sigma(\widetilde{D}^{-1/2}\widetilde{A}\widetilde{D}^{-1/2}H^{(l)}W^{(l)}).$$

Ma trận $W^{(l)}$ được học từ hàm mất mát. Chuẩn hóa giúp hạn chế việc nút bậc cao đóng góp tổng thông tin lớn chỉ vì có nhiều cạnh. Đối với bài toán phân loại nút, mất mát chỉ tính trên các nút có nhãn thuộc tập huấn luyện. Việc dùng cấu trúc của nút test trong quá trình truyền thông điệp cần được ghi rõ là thiết lập transductive.

Các biến thể và lựa chọn phù hợp

GraphSAGE lấy mẫu láng giềng và học hàm tổng hợp, phù hợp khi cần xử lý các nút mới có đặc trưng. GAT học trọng số chú ý giữa các nút liên quan, nhưng hệ số chú ý không tự trở thành lời giải thích nhân quả. Với đồ thị nhiều loại quan hệ, R-GCN dùng các phép biến đổi phụ thuộc loại cạnh. Lựa chọn kiến trúc phải theo dữ liệu và bài toán đánh giá, thay vì chọn mô hình phức tạp chỉ để tăng số thành phần.

Ba đầu ra thường gặp là nhãn nút, khả năng tồn tại cạnh và nhãn toàn đồ thị. Các đầu ra này khác với câu trả lời của chatbot. GNN có thể giúp truy xuất hoặc xếp hạng bằng chứng, còn phần diễn đạt vẫn cần một cơ chế tạo phản hồi và kiểm tra nguồn. Nếu dự đoán một cạnh chưa có trong KB, hệ thống phải đánh dấu đó là giả thuyết của mô hình, không tự thêm như một sự thật đã kiểm chứng.

Áp dụng vào kho tri thức của project

Kho đang chạy có 60 nút Concept và 32 cạnh RELATED_TO. Tên quan hệ thật nằm ở thuộc tính label, mã tài liệu ở document. Ví dụ, khái niệm HbA1c nối với đường huyết trung bình bằng quan hệ phản ánh. Có thể tạo một ma trận kề từ các cặp này và vector đặc trưng từ văn bản, nhưng kho nhỏ và thiếu nhãn nhiệm vụ khiến việc huấn luyện một GNN chưa có cơ sở đánh giá đáng tin cậy.

Vì vậy bản triển khai giữ cách tra cứu có nguồn: tìm đoạn văn phù hợp rồi lấy các cạnh cùng mã tài liệu. Điểm mạnh là mỗi quan hệ đều kiểm tra được. Điểm yếu là hệ thống chưa học biểu diễn liên kết và chưa suy luận trên các quan hệ thiếu. Đây là ranh giới rõ giữa phần lý thuyết GNN và sản phẩm Neo4j thực tế.

Thiết kế một thí nghiệm mở rộng

Nếu mở rộng, cần xây dựng nhãn và quy trình kiểm chứng trước. Với dự đoán liên kết, tách các cạnh test trước khi tạo đặc trưng, kiểm soát cạnh đảo và các quan hệ trùng nghĩa. Mẫu âm phải phù hợp miền, vì một cạnh chưa được ghi nhận không nhất thiết là cạnh sai. So sánh với các đường cơ sở như tương đồng văn bản và số láng giềng chung trên cùng phép chia.

Với phân loại nút, ghi rõ số nút train, validation và test, cách tạo đặc trưng và liệu test có tham gia cấu trúc đồ thị hay không. Báo cáo macro-F1, kết quả nhiều seed và thời gian chạy. Không dùng kết quả phân loại nút của một đồ thị trích dẫn để kết luận hiệu quả tư vấn y tế.

Hạn chế cần kiểm soát

Quá nhiều tầng có thể làm biểu diễn các nút gần nhau trở nên khó phân biệt. Nút trung tâm có thể tập trung nhiều thông tin, còn lỗi liên kết làm lan truyền tín hiệu sai. Một KB có nguồn yếu vẫn tạo ra đầu vào yếu cho GNN. Cần kiểm soát nguồn, loại quan hệ và phiên bản dữ liệu trước khi tối ưu kiến trúc.

Chương 7

Agent, hệ đa agent và học tăng cường

Khái niệm agent

Agent nhận quan sát, duy trì trạng thái phục vụ nhiệm vụ và lựa chọn hành động trong một môi trường. Một hàm dự đoán chỉ ánh xạ đầu vào sang nhãn; một agent còn cần xác định khi nào gọi hàm đó, xử lý trường hợp thiếu dữ liệu và chuyển kết quả cho thành phần khác. Mức tự chủ phụ thuộc vào phạm vi hành động được trao và cơ chế phản hồi.

Trong project, agent là các đối tượng phần mềm với tên, bộ xử lý thông điệp và bộ nhớ cục bộ cho lượt xử lý. MessageBus định tuyến REQUEST tới đúng handler và ghi INFORM hoặc FAILURE khi hoàn tất. Việc triển khai trong một tiến trình giúp tái lập dễ hơn, nhưng không cung cấp tính độc lập triển khai của một hệ phân tán.

Phối hợp và giao thức

Điều phối tập trung giúp xác định thứ tự xử lý và hạn chế gọi công cụ không cần thiết. Truy xuất và mô hình có chức năng khác nhau: một bên trả bằng chứng, bên kia trả kết quả tính toán. Agent đối chiếu kiểm tra phạm vi diễn giải; agent phản hồi tổng hợp nội dung. Kết quả của hai nhánh đều đi đến đầu ra trợ giúp theo sơ đồ trên bảng.

Một giao thức tối thiểu cần xác định người gửi, người nhận, loại thông điệp, nội dung và điều kiện thất bại. Phiên bản hiện có bốn trường này, cộng nhật ký tuần tự theo từng yêu cầu. Khi mở rộng cần thêm mã hội thoại, mã yêu cầu, hạn xử lý và cơ chế chống xử lý lặp. Không nên đánh đồng việc ghi một dòng log với việc một agent đã thực thi: handler phải thật sự nhận yêu cầu và tạo kết quả.

Mô hình học tăng cường

Một quá trình quyết định Markov được mô tả bởi trạng thái, hành động, động lực chuyển trạng thái, phần thưởng và hệ số chiết khấu. Mục tiêu là tối đa hóa tổng phần thưởng kỳ vọng. Trong Q-learning dạng bảng:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right].$$

Hành động khám phá giúp quan sát các lựa chọn chưa biết; khai thác chọn giá trị hiện cao nhất. Cần tách đánh giá khỏi huấn luyện, cố định seed khi tái lập và tránh đánh giá chỉ trên các episode có khám phá nếu mục tiêu là đo chính sách cuối cùng.

Kết hợp học với luật

Luật biểu diễn những điều kiện phải kiểm tra trực tiếp, chẳng hạn chưa đủ trường thì chưa suy luận, không có tài liệu thì không khẳng định đã có bằng chứng. Chính sách học chỉ lựa chọn trong phạm vi cho phép. Cách kết hợp này có thể viết là chọn hành động tốt nhất trên tập hành động hợp lệ $\mathcal{A}_{safe}(s)$, hoặc kiểm tra và thay thế hành động sau khi đề xuất.

Bản demo dùng môi trường một bước với bốn trạng thái và ba hành động. Do episode kết thúc ngay, biểu thức cập nhật chỉ còn $Q \leftarrow Q + \alpha(r - Q)$. Đây là minh họa học chính sách điều phối, không phải agent tự học điều trị. Phần thưởng được thiết kế thủ công nên kết quả tốt chỉ chứng minh chính sách đã học được quy tắc của môi trường mô phỏng.

Liên hệ các phần tiểu luận

Các mô hình học máy và học sâu cung cấp công cụ tính toán; đồ thị tri thức cung cấp bằng chứng có cấu trúc; truy xuất chọn thông tin theo yêu cầu. Hệ đa agent kết nối các công cụ đó thành quy trình phục vụ người dùng. Đóng góp mới của project nằm ở việc tích hợp hai nhánh, lưu dấu vết thực thi, đưa dữ liệu lên Neo4j và tạo giao diện trình diễn có thể kiểm tra.

Tiểu luận 01 của Phạm Mai Linh được dùng làm tài liệu tham khảo nền tảng theo yêu cầu của sinh viên. Tiểu luận 02 và 03 của Kiều Trường Giang là nguồn kế thừa chính. Báo cáo của Nguyễn Tự Kiên được tham khảo về cấu trúc trình bày. Các kết quả thử nghiệm mới của GlucoGuide được lưu riêng trong results để tránh lẫn với kết quả trong tài liệu nguồn.

Phần II

Project tích hợp

Chương 1

Giới thiệu

1.1 Bài toán và sản phẩm

Project xây dựng GlucoGuide, một ứng dụng web tích hợp hai nhánh xử lý của sơ đồ học phần. Nhánh dữ liệu sử dụng mô hình mạng nơ-ron nhiều lớp để phân loại nhãn nguy cơ tổng hợp và dự đoán điểm tiến triển bệnh trong dữ liệu nghiên cứu. Nhánh tri thức kế thừa kho tài liệu của tiểu luận 02, tổ chức các khái niệm cùng quan hệ trên Neo4j và truy xuất văn bản theo câu hỏi. Kết quả hai nhánh được trình bày cùng nguồn bằng chứng và nhật ký xử lý.

Sinh viên thực hiện là Kiều Trường Giang, B23DCCN254, nhóm học phần 06, nhóm thực hiện 02. Project được phát triển từ các sản phẩm của sinh viên, có tham khảo bố cục và cách mô tả kiến trúc trong báo cáo của Nguyễn Tự Kiên. Các số liệu của bài tham khảo không được dùng làm kết quả của project này.

1.2 Phạm vi đầu ra

Hai bộ dữ liệu có định nghĩa nhãn khác nhau. Bộ phân loại gồm các nhãn High, Moderate và Low của dữ liệu tổng hợp. Bộ hồi quy gồm những người đã mắc bệnh và một thước đo tiến triển sau một năm. Vì vậy không thể ghép một nhãn High với một điểm hồi quy để suy ra xác suất mắc bệnh. Ứng dụng dùng hai biểu mẫu riêng và công bố phạm vi ngay tại đầu ra.

Sản phẩm hiện sử dụng tổng hợp trích đoạn xác định. Câu trả lời được ghép từ các đoạn văn truy xuất được và thông tin về phạm vi mô hình; không gọi mô hình ngôn ngữ bên ngoài. Lựa chọn này giúp kiểm tra nguồn và tái lập kết quả, nhưng khả năng đối thoại tự nhiên bị giới hạn bởi từ vựng và độ phủ của kho tài liệu.

1.3 Mục tiêu đánh giá

Phân loại được đo bằng accuracy và macro-F1. Hồi quy được đo bằng MAE và hệ số xác định. Truy xuất cần kiểm tra cả tài liệu được chọn và quan hệ có nguồn trên Neo4j. Kiểm tra ứng dụng tập trung vào các tình huống có bằng chứng, thiếu bằng chứng, thiếu đầu vào, giá trị không hợp lệ và cơ sở dữ liệu đồ thị không hoạt động.

Chương 2

Agent và cơ chế phối hợp

2.1 Vai trò xử lý

Hệ thống phân chia trách nhiệm thành tiếp nhận, kiểm tra dữ liệu, mô hình, tri thức, chính sách lựa chọn bước phản hồi, đối chiếu phạm vi và phản hồi. Bản triển khai hiện điều phối các vai trò này trong một tiến trình FastAPI. Nhật ký lưu người gửi, người nhận, loại thông điệp và nội dung. Đây là mô hình điều phối tập trung; các agent được gọi qua MessageBus với handler và bộ nhớ trong lượt, chưa chạy như những dịch vụ tự chủ độc lập.

Một thông điệp REQUEST thể hiện yêu cầu xử lý. INFORM ghi nhận kết quả và FAILURE thể hiện thành phần không thực hiện được. Khi Neo4j không hoạt động, hệ thống ghi FAILURE và chỉ dùng các đoạn văn cục bộ, không giả lập kết quả đồ thị. Khi thiếu đầu vào mô hình, hệ thống trả lại danh sách trường cần bổ sung trước khi suy luận.

2.2 Reinforcement Learning kết hợp luật

Chính sách được học trong môi trường mô phỏng luồng xử lý, không mô phỏng hiệu quả điều trị. Trạng thái gồm hai biến nhị phân: đã có kết quả mô hình và đã có bằng chứng truy xuất. Bốn trạng thái tương ứng với bốn tổ hợp của hai biến này. Ba hành động là hỏi làm rõ, trả lời có nguồn và giải thích mô hình kèm nguồn.

Hàm thưởng bằng một nếu hành động phù hợp trạng thái, bằng âm một trong trường hợp còn lại. Khi chưa có bằng chứng, hành động mục tiêu là hỏi làm rõ. Khi có bằng chứng nhưng chưa có kết quả mô hình, hành động mục tiêu là trả lời có nguồn. Khi có cả hai, hành động mục tiêu là giải thích mô hình và dẫn nguồn. Đây là hàm thưởng do người thiết kế quy định; không phải đánh giá của bệnh nhân hay chuyên gia.

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r - Q(s, a)].$$

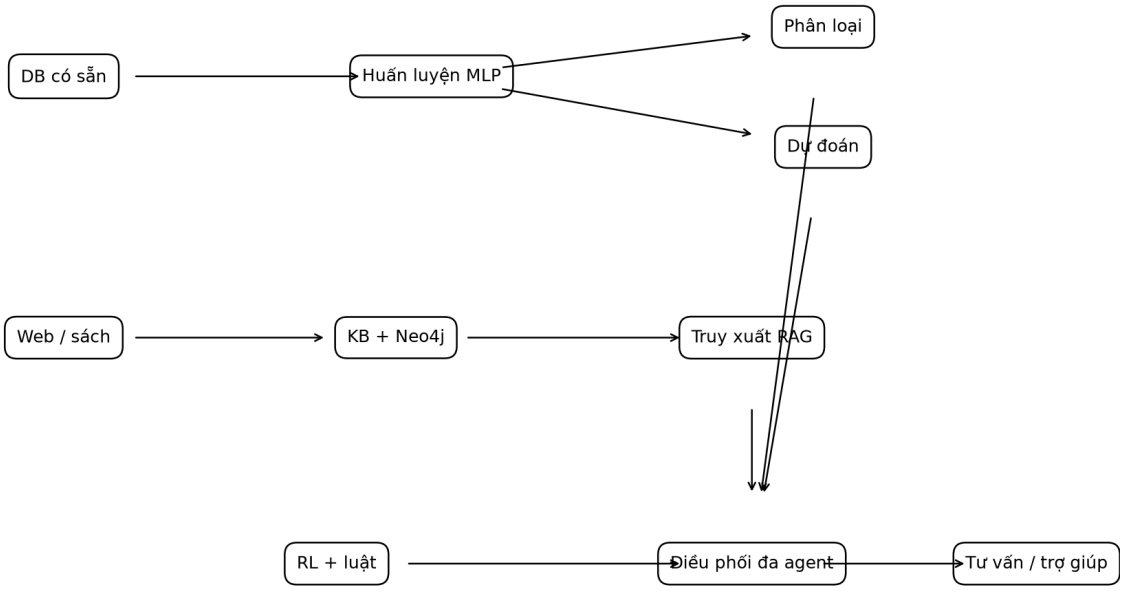
Mỗi episode kết thúc sau một hành động, nên không có thành phần bootstrap tương lai. Cấu hình gồm 3.000 episode, seed 254, tốc độ học 0,15 và xác suất khám phá 0,2. Vì bài toán chỉ có một bước, thí nghiệm minh họa cơ chế học giá trị hành động nhưng chưa chứng minh khả năng lập kế hoạch dài hạn. Luật thiếu bằng chứng luôn ép hành động hỏi làm rõ, kể cả khi chính sách học đề xuất hành động khác.

2.3 Giới hạn của kiến trúc

Việc hiển thị nhiều vai trò trong nhật ký không tự chứng minh tính tự chủ của một hệ đa agent. Bản hiện tại đã có handler và bộ nhớ theo lượt; hướng mở rộng là bộ nhớ hội thoại bền vững, cơ chế chất vấn khi có bất đồng và triển khai dịch vụ độc lập. Báo cáo phân biệt rõ những vai trò đã điều phối được với các năng lực còn cần phát triển.

Chương 3

Kiến trúc tổng quan



Hình 3.1: Hai nhánh hội tụ về điều phối và trợ giúp, dựng lại từ sơ đồ trên bảng.

3.1 Nhánh dữ liệu

DB có sẵn cung cấp dữ liệu cho quá trình huấn luyện. Pipeline phân loại gồm điền trung vị, chuẩn hóa và MLP. Pipeline hồi quy chuẩn hóa dữ liệu rồi suy luận bằng MLP. Hai đầu ra đều đi tới vùng phản hồi của ứng dụng. Trong lúc phục vụ, mô hình được nạp từ checkpoint; ứng dụng không huấn luyện lại theo câu hỏi của người dùng.

3.2 Nhánh tri thức

Nguồn tri thức là các đoạn văn đã được tổ chức trong tiểu luận 02. Mỗi tài liệu có mã, tiêu đề, từ khóa, văn bản, định danh nguồn và các bộ ba. Project bảo toàn những trường này khi chuyển

sang kho tri thức mới. Việc cập nhật hướng dẫn y tế từ web chưa tự động hóa; phiên bản kho tài liệu phải được xem như một snapshot cần rà soát trước khi mở rộng phạm vi sử dụng.

Một bộ ba gồm khái niệm đầu, tên quan hệ và khái niệm cuối. Neo4j lưu hai đầu dưới nhãn Concept; quan hệ RELATED_TO có thuộc tính label biểu diễn tên quan hệ, document biểu diễn tài liệu và evidence biểu diễn đoạn văn làm căn cứ. Ràng buộc duy nhất theo tên khái niệm và thao tác MERGE giúp nạp lại dữ liệu mà không tạo các bản sao giống hệt.

3.3 Luồng người dùng

Câu hỏi kiến thức được chuyển đến truy xuất. Yêu cầu mô hình phải đi qua kiểm tra các trường trước khi suy luận. Văn bản và kết quả đồ thị được chuyển sang đối chiếu; chính sách học và luật xác định kiểu phản hồi. Giao diện hiển thị câu trả lời, tài liệu nguồn và các bước xử lý. Các trường của hai mô hình được tách riêng để tránh biến đổi sai đơn vị hoặc tự suy diễn giá trị còn thiếu.

Chương 4

Các thành phần và thực nghiệm

4.1 Phân loại

Dữ liệu tổng hợp gồm 50.000 dòng. Tám biến đầu vào là tuổi, BMI, HbA1c, đường huyết lúc đói, glucose, huyết áp tâm thu, HDL và LDL. Các biến chứa điểm nguy cơ, khuyến nghị sức khỏe và yêu cầu gặp bác sĩ không được đưa vào mô hình. Phép chia phân tầng với seed 254 giữ 40.000 dòng huấn luyện và 10.000 dòng kiểm tra.

MLP có ba tầng ẩn 64, 32 và 16 đơn vị, dừng sớm và tối đa 150 vòng lặp. Trung vị và tham số chuẩn hóa được học trên tập huấn luyện ngoài. Cơ chế early stopping của thư viện chia tiếp một phần dữ liệu cho validation; do tiền xử lý nằm trước MLP, các tham số tiền xử lý đã nhìn thấy phần validation nội bộ này. Tập test ngoài vẫn độc lập, nhưng để kiểm soát validation nghiêm ngặt hơn cần tách validation trước toàn bộ pipeline.

Logistic Regression dùng trọng số lớp cân bằng làm đối chứng. MLP dùng cấu hình mặc định không cân bằng trọng số. Vì hai cấu hình có cách xử lý mất cân bằng khác nhau, chênh lệch kết quả không chỉ phản ánh kiến trúc mạng. Accuracy cần đọc cùng macro-F1, đặc biệt với lớp Low hiếm.

4.2 Hồi quy

Bộ diabetes từ scikit-learn được tải ở dạng chưa chuẩn hóa, gồm 442 quan sát với 10 biến đầu vào. Phép chia seed 254 giữ 353 mẫu huấn luyện và 89 mẫu kiểm tra. Ridge với hệ số điều chuẩn 10 là đối chứng. MLP có ba tầng ẩn 64, 32 và 16, dừng sớm và tối đa 1.000 vòng lặp.

Các biến s1 đến s6 là những trường nghiên cứu của bộ dữ liệu. Không được thay tên hoặc đơn vị bằng suy đoán khi nhập hồ sơ ngoài đời. Mục tiêu của demo là minh họa quy trình suy luận từ mô hình đã lưu. Chất lượng trên bộ dữ liệu nhỏ không chứng minh khả năng chuyển sang quần thể Việt Nam.

4.3 Kết quả kiểm tra

Mô hình	Accuracy/MAE	F1/R2	Mẫu test
LogisticRegression	0.6965	0.5005	10000
MLPClassifier	0.8180	0.5441	10000
Ridge	44.0555	0.4791	89
MLPRegressor	44.3225	0.4758	89

MLP cải thiện accuracy và macro-F1 so với Logistic Regression trong lần chạy này. Tuy nhiên macro-F1 còn thấp so với accuracy, phản ánh hiệu năng không đồng đều giữa các lớp. Ở hồi quy, Ridge có MAE thấp hơn và hệ số xác định cao hơn MLP. Báo cáo vẫn trình diễn MLP để minh họa nhánh Deep Learning, đồng thời công bố đầy đủ đối chứng và không nhận MLP là mô hình tốt nhất.

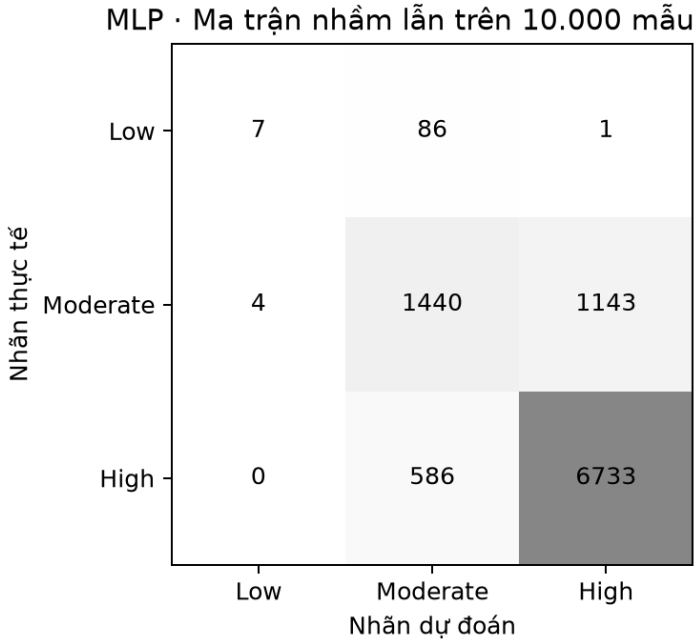
4.4 Truy xuất và đồ thị

Văn bản được biểu diễn bằng TF-IDF với unigram và bigram, bỏ dấu khi chuẩn hóa. Câu hỏi được biến đổi bằng cùng từ vựng. Điểm truy xuất là tích vô hướng của các vector đã chuẩn hóa. Hệ thống giữ tối đa ba đoạn có điểm lớn hơn giá trị lớn nhất giữa 0,04 và 65% điểm của đoạn đứng đầu. Ngưỡng này là quy tắc triển khai chưa được chọn trên một bộ validation độc lập.

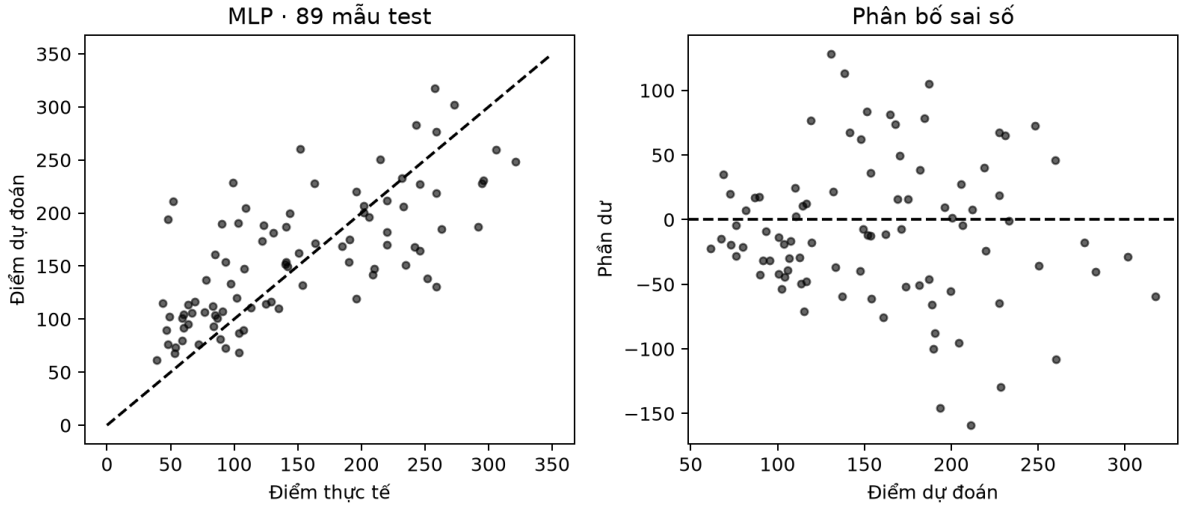
Neo4j lấy các cạnh có thuộc tính document trùng với mã tài liệu đã truy xuất. Do đó đồ thị cung cấp cấu trúc và căn cứ bổ sung cho truy xuất văn bản; đây chưa phải một bộ máy suy luận đa bước tổng quát. Một nhãn quan hệ không chứng minh quan hệ nhân quả. Khi không có tài liệu đủ điểm, hệ thống yêu cầu người dùng làm rõ.

4.5 Phân tích sai số theo lớp

Ma trận nhầm lẫn được tạo từ toàn bộ 10.000 dự đoán ngoài mẫu, không lấy từ tập huấn luyện. Tập test có 7.319 mẫu High, 2.587 mẫu Moderate và 94 mẫu Low. MLP dự đoán đúng 6.733 mẫu High, 1.440 mẫu Moderate nhưng chỉ 7 mẫu Low. Vì vậy accuracy 81,8% che giấu khả năng nhận diện lớp hiếm còn yếu. Recall của Low chỉ khoảng 7,45%; đây là hạn chế cần nêu khi bảo vệ báo cáo. Tỷ lệ lỗi giữa Moderate và High cho thấy ranh giới của nhãn tổng hợp chưa được mô hình tách tốt. Cải thiện tiếp theo cần so sánh trọng số lớp, điều chỉnh ngưỡng và cách lấy mẫu trên validation, đồng thời giữ nguyên test để tránh chọn cấu hình theo kết quả test. Không đổi tên nhãn tổng hợp thành chẩn đoán y tế.



Hình 4.1: Ma trận nhầm lẫn của MLP trên tập kiểm tra phân tầng.



Hình 4.2: Dự đoán và phần dư của MLP trên 89 mẫu kiểm tra.

4.6 Phân tích phần dư hồi quy

Đường chéo thể hiện dự đoán bằng giá trị thật. Những điểm cách xa đường chéo đóng góp lớn vào MAE. Phần dư được tính bằng giá trị thực trừ dự đoán; phần dư dương biểu thị dự đoán thấp hơn thực tế. Không thể dùng một chỉ số tổng hợp để bảo đảm mọi nhóm đầu vào có sai số tương đương. Bản này chưa xây dựng khoảng dự báo và chưa hiệu chỉnh bất định.

4.7 Hợp đồng dữ liệu và API

GET /api/schema cung cấp thứ tự đặc trưng. GET /api/examples cung cấp một mẫu từ mỗi bộ dữ liệu để trình diễn. POST /api/chat nhận question, task và values. Với task knowledge, hệ thống truy xuất văn bản; với classification hoặc regression, values phải đủ các trường tương ứng. Kết quả gồm answer, result, sources, graph_evidence và trace.

Trường result của phân loại gồm label, scores và scope. scores là đầu ra xác suất lớp của mô hình trên nhãn tổng hợp, chưa được hiệu chỉnh. Trường result của hồi quy gồm value và scope. Việc tách cấu trúc này tránh dùng chung một con số cho hai ý nghĩa khác nhau. Dữ liệu mẫu là mẫu của bộ nghiên cứu, không phải hồ sơ do người dùng cung cấp.

Kiểm tra đầu vào hiện yêu cầu số hữu hạn, giới hạn trị tuyệt đối và đủ số trường. Đây là kiểm tra kỹ thuật, chưa phải kiểm định lâm sàng từng đơn vị. Giá trị nhập bằng số nhưng sai đơn vị vẫn có thể làm kết quả vô nghĩa; vì vậy demo được giới hạn ở dữ liệu nghiên cứu và có nút điền mẫu.

4.8 Kết quả kiểm thử tích hợp

Script test_scenarios.py thực hiện tám trường hợp: câu hỏi có nguồn; câu hỏi ngoài từ vựng; suy luận phân loại; suy luận hồi quy; thiếu trường; giá trị không hữu hạn; tên bài toán không hợp lệ; lỗi Neo4j có kiểm soát. Tám trường hợp đều đạt điều kiện kiểm tra ở lần chạy lưu trong results/api_scenarios.json. Kiểm tra lỗi cơ sở dữ liệu được thực hiện bằng thay thế driver trong test, không phải chủ động tắt cơ sở dữ liệu đang phục vụ người dùng.

Kiểm thử xác nhận hợp đồng phản hồi và nhánh lỗi, chưa đo độ chính xác truy xuất trên tập câu hỏi lớn. Trường hợp ngoài miền sử dụng chuỗi không có trong từ vựng, do đó không chứng minh hệ thống phát hiện mọi câu hỏi ngoài miền bằng tiếng Việt. Những giới hạn này giúp tách kiểm thử chức năng khỏi đánh giá chất lượng tư vấn.

4.9 Đối chiếu với sơ đồ yêu cầu

Thành phần	Hiện thực trong sản phẩm

DB và Deep Learning	Hai bộ dữ liệu tách biệt, MLPClassifier và MLPRegressor; có checkpoint và kết quả test.
Phân loại và dự đoán	Hai lựa chọn trong phòng thực nghiệm, cùng chuyển kết quả đến phản hồi và nhật ký.
Web/sách đến KB	Kế thừa KB đã biên soạn từ tiểu luận 02; bảo toàn mã nguồn tài liệu, không tự động thu thập web ở thời điểm chạy.
KB và RAG	TF-IDF chọn đoạn văn; Neo4j trả các quan hệ cùng mã tài liệu; phản hồi trích đoạn có nguồn.
Agent và RL + rule	Handler và MessageBus xử lý yêu cầu; chính sách học một bước chọn kiểu phản hồi, luật chặn thiếu bằng chứng.
Tư vấn, trợ giúp	Chatbot giải thích kiến thức và phạm vi đầu ra; chưa sinh đối thoại tự do bằng LLM.

Chương 5

Code và demo

5.1 Địa chỉ và cách chạy

Demo cục bộ: `http://localhost:8765`. Neo4j Browser: `http://localhost:7474/browser/`. Repository đích: `https://github.com/jellalaz/He_Thong_Thong_Minh_Thay_Que`. Demo công khai tại thời điểm hoàn thiện: `https://bear-parental-pair-mercury.trycloudflare.com`. Đây là đường hầm tạm thời, phụ thuộc máy và tiến trình đang chạy; không phải hosting thường trực. Địa chỉ localhost chỉ hoạt động trên máy đang chạy dịch vụ. Báo cáo không coi địa chỉ này là một website công khai.

Cài các thư viện trong requirements.txt vào môi trường riêng. Chạy scripts/train_models.py để tạo checkpoint, scripts/train_policy.py để tạo chính sách và scripts/ingest_graph.py để nạp đồ thị. Sau đó chạy uvicorn app.main:app từ thư mục project. Cấu hình Neo4j nằm trong .env và không đưa lên repository.

5.2 Kịch bản trình diễn

Kịch bản thứ nhất: hỏi "HbA1c là gì?". Kiểm tra câu trả lời có mã tài liệu, kiểm tra bước truy xuất và số quan hệ Neo4j. Kịch bản thứ hai: hỏi một chủ đề không có trong kho, chẳng hạn lịch sử một đội bóng. Hệ thống phải yêu cầu làm rõ hoặc báo thiếu bằng chứng, không tự tạo nguồn.

Kịch bản thứ ba: nhập đủ tám trường phân loại bằng một mẫu của dữ liệu. Ghi lại nhãn, thông báo phạm vi và nhật ký mô hình. Kịch bản thứ tư: nhập đủ mười trường hồi quy từ mẫu nghiên cứu, kiểm tra đầu ra số và thông báo đây là điểm tiến triển. Hai kịch bản không dùng chung biểu mẫu.

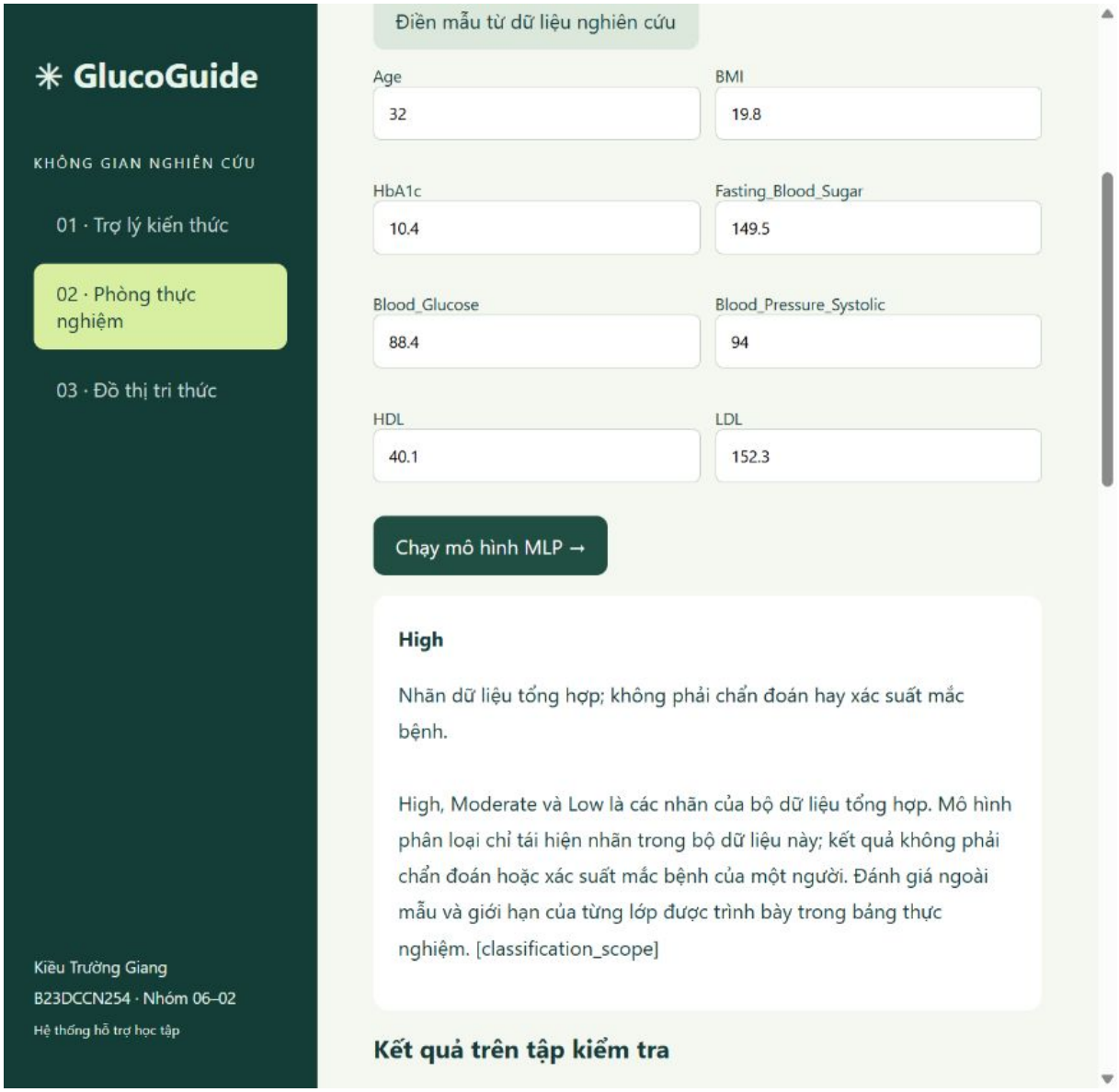
Kịch bản thứ năm: bỏ một trường bắt buộc. Hệ thống phải nêu trường còn thiếu. Kịch bản thứ sáu: nhập giá trị không hữu hạn hoặc chuỗi ký tự vào API, hệ thống phải từ chối. Kịch bản thứ bảy: tắt Neo4j rồi hỏi kiến thức, kiểm tra nhật ký FAILURE và việc công bố chỉ dùng văn bản. Kịch bản thứ tám: mở Neo4j Browser và thực hiện truy vấn dưới đây để đối chiếu với giao diện đồ thị.

```
MATCH (a:Concept)-[r:RELATED_TO]->(b:Concept)
RETURN a,r,b LIMIT 100;
```

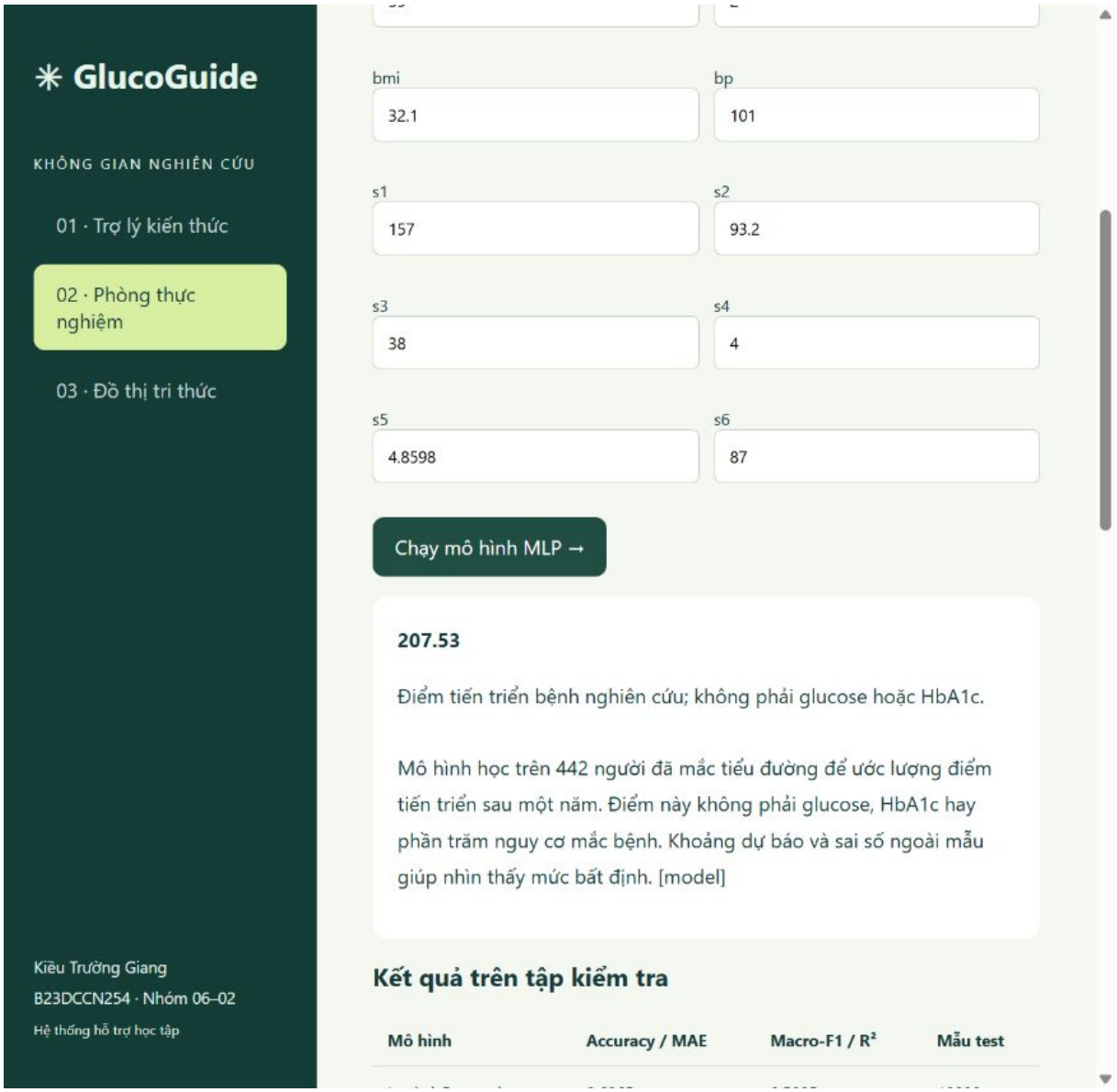
5.3 Hạn chế và hướng hoàn thiện

Bản hiện tại chưa có bộ nhớ hội thoại, chưa đọc phiếu xét nghiệm, chưa dùng LLM để diễn đạt và chưa có hội đồng tranh biện độc lập. Các phần này không được nhận là đã hoàn thành. Kiểm chứng mô hình mới chỉ dùng một seed; chưa có đánh giá chuyên gia, chưa có dữ liệu ngoài miền và chưa kiểm thử tải lớn. RL học quy trình mô phỏng, không quyết định điều trị.

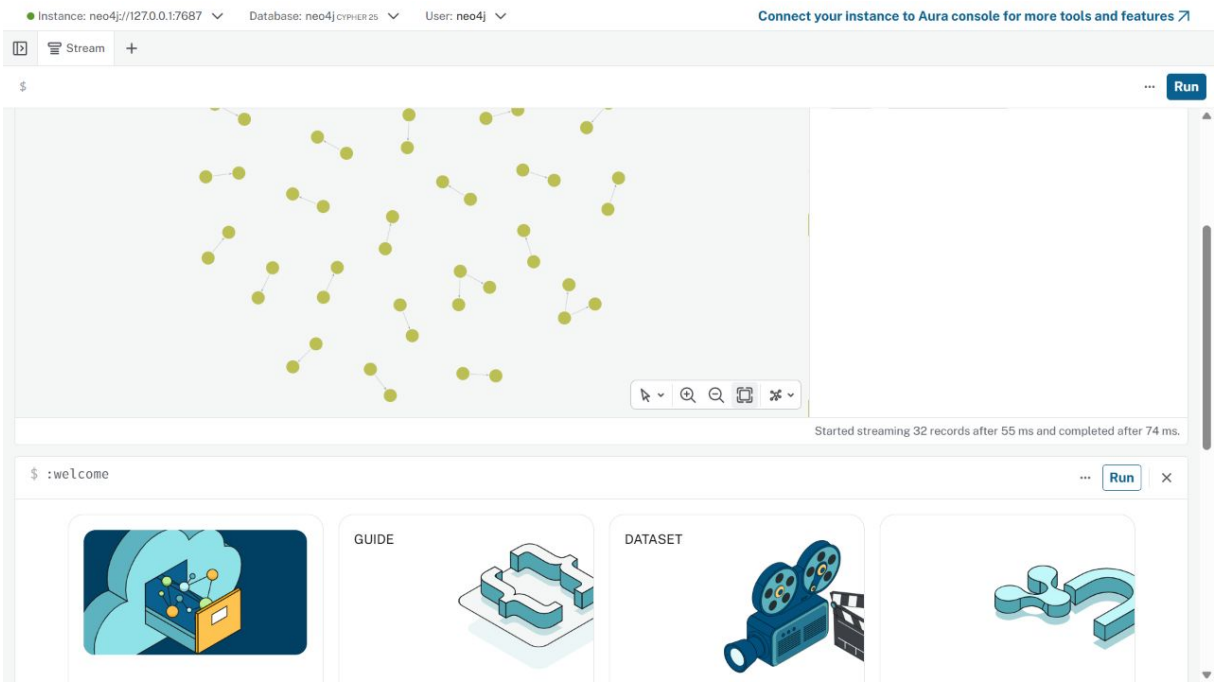
Hướng ưu tiên là mở rộng giao thức agent và bộ nhớ hội thoại, mở rộng tập câu hỏi đánh giá truy xuất độc lập, bổ sung kiểm tra đơn vị có nguồn và cập nhật KB từ tài liệu được kiểm chứng. Sau khi có các thành phần đó mới đánh giá lợi ích phối hợp so với từng thành phần đơn lẻ bằng các phép loại bỏ thành phần trên cùng tập kiểm tra.



Hình 5.1: Suy luận phân loại từ mẫu của bộ dữ liệu.

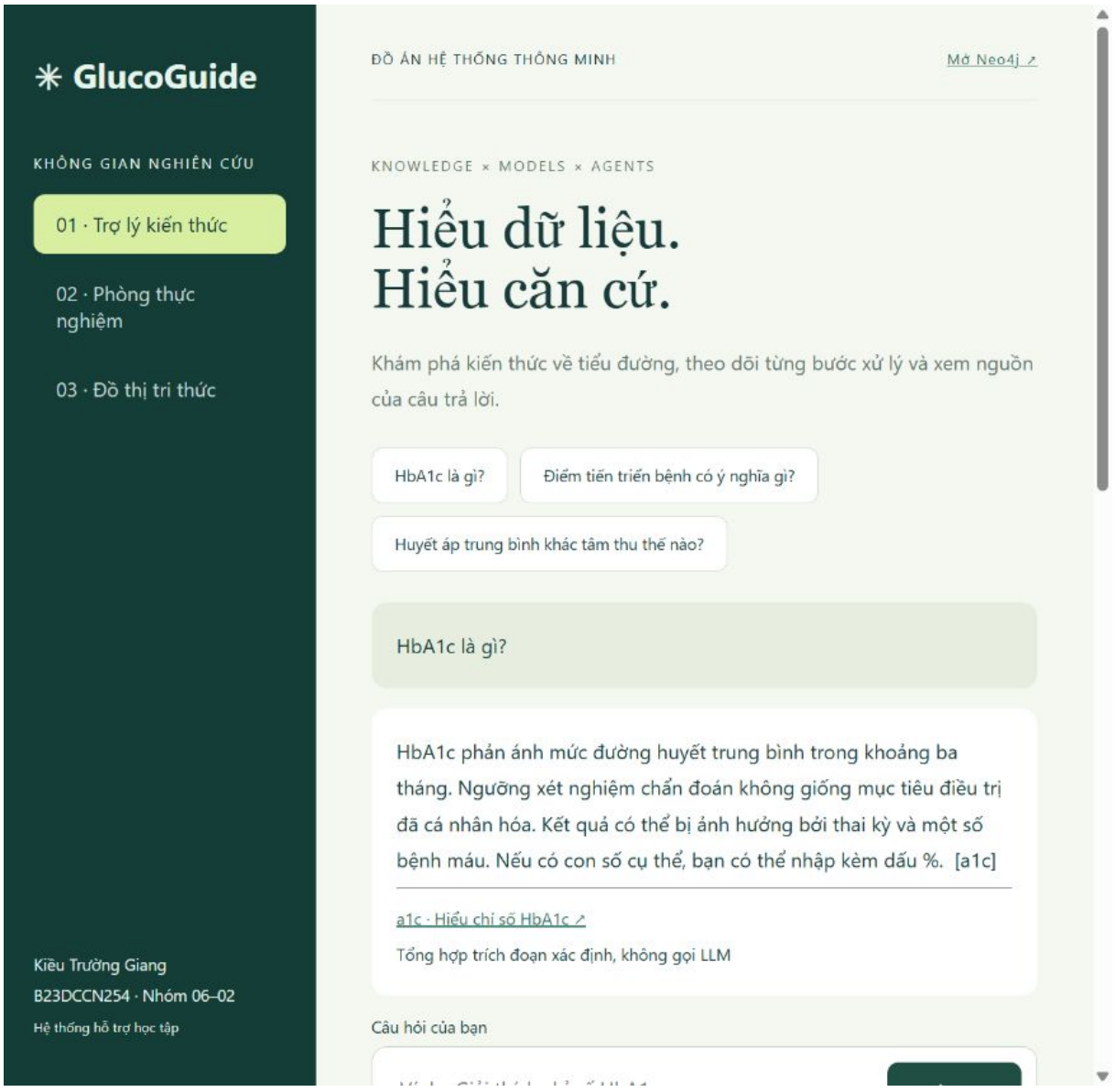


Hình 5.2: Đầu ra hồi quy là điểm tiến triển nghiên cứu.

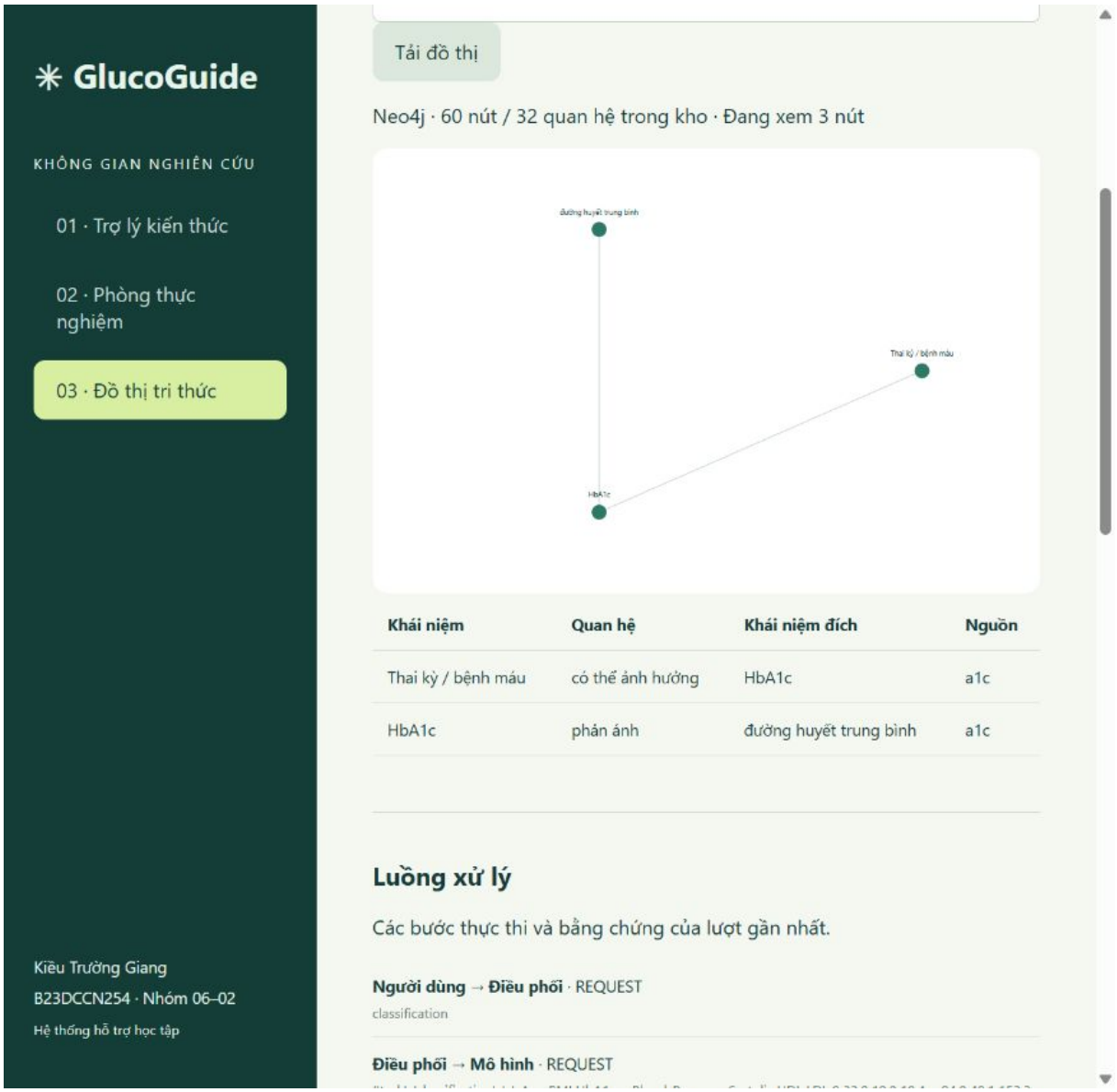


Hình 5.3: Neo4j Browser đã kết nối và trả về các quan hệ Concept.

5.4 Ảnh chụp sản phẩm đang chạy



Hình 5.4: Chatbot trả lời HbA1c kèm mã tài liệu và liên kết nguồn.



Hình 5.5: Lọc quan hệ theo tài liệu a1c, dữ liệu đọc trực tiếp từ Neo4j.

Tài liệu tham khảo

[1] Kiều Trường Giang, Tiểu luận 02, nhóm 06–02, tài liệu học phần do sinh viên cung cấp.

[2] Kiều Trường Giang, Tiểu luận 03, B23DCCN254, tài liệu và thực nghiệm học phần do sinh viên cung cấp.

[3] Phạm Mai Linh, Tiểu luận 01, nhóm 06–03, tài liệu tham khảo do sinh viên chỉ định.

[4] Nguyễn Tự Kiên, Tiểu luận final CT_07 và Project01 CT_07, hai PDF tham khảo về bố cục do sinh viên cung cấp.

[5] T. Kipf và M. Welling, Semi-Supervised Classification with Graph Convolutional Networks, ICLR 2017. https://arxiv.org/abs/1609.02907.

[6] W. Hamilton, R. Ying và J. Leskovec, Inductive Representation Learning on Large Graphs, 2017. https://arxiv.org/abs/1706.02216.

[7] P. Velickovic và cộng sự, Graph Attention Networks, ICLR 2018. https://arxiv.org/abs/1710.10903.

[8] M. Schlichtkrull và cộng sự, Modeling Relational Data with Graph Convolutional Networks, 2017. https://arxiv.org/abs/1703.06103.

[9] Scikit-learn, tài liệu load_diabetes. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html.

[10] Mobeen Fatimah, Diabetes Risk Prediction Dataset 50K Patients. https://www.kaggle.com/datasets/mobeenfatimah/diabetes-risk-prediction-dataset-50k-patients.

[11] NIDDK, Diabetes Tests and Diagnosis. https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis.

[12] Danh mục nguồn tri thức của project được lưu tại data/knowledge.json; mỗi đoạn văn có mã tài liệu, mã nguồn và các bộ ba có căn cứ.

Phụ lục A

Mã nguồn tái lập

A.1 scripts/ingest_graph.py

```
from pathlib import Path
import json,os
from neo4j import GraphDatabase
root=Path(__file__).resolve().parents[1]
env=dict(x.split('=',1) for x in (root/'.env').read_text().splitlines() if '=' in x)
kb=json.loads((root/'data/knowledge.json').read_text(encoding='utf-8'))
rows=[dict(source=t[0],label=t[1],target=t[2],document=d['id'],evidence=d['text']) for d in
 ↪  kb['documents'] for t in d.get('triples',[])]
with GraphDatabase.driver(env['NEO4J_URI'],auth=(env['NEO4J_USER'],env['NEO4J_PASSWORD']),connection_t
 ↪  imeout=5) as
 ↪  driver:
 driver.execute_query('CREATE CONSTRAINT concept_name IF NOT EXISTS FOR (c:Concept) REQUIRE c.name IS
  ↪  UNIQUE')
 driver.execute_query('UNWIND $rows AS row MERGE (a:Concept {name:row.source}) MERGE (b:Concept
  ↪  {name:row.target}) MERGE (a)-[r:RELATED_TO {label:row.label,document:row.document}]->(b) SET
  ↪  r.evidence=row.evidence',rows=rows)
 print(driver.execute_query('MATCH (n:Concept) RETURN count(n) AS nodes').records)
(root/'results/graph_import.json').write_text(json.dumps({'documents':len(kb['documents']),'relations'
 ↪  :len(rows),'storage':'Neo4j'}))
```

A.2 scripts/train_models.py

```
"""Train reproducible MLP classifier and regression baseline on distinct datasets."""
from pathlib import Path
import json, zipfile, warnings
import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier, MLPRegressor
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import accuracy_score, f1_score, mean_absolute_error, r2_score
import joblib
ROOT=Path(__file__).resolve().parents[1]
source=ROOT/'data/raw/diabetes.zip'
with zipfile.ZipFile(source) as z:
 df=pd.read_csv(z.open(z.namelist()[0]))
features=['Age','BMI','HbA1c','Fasting_Blood_Sugar','Blood_Glucose','Blood_Pressure_Systolic','HDL','L
 ↪  DL']
```

```
X=df[features];y=df.Diabetes_Risk
tr,te=train_test_split(np.arange(len(df)),test_size=.2,stratify=y,random_state=254)
rows=[]
for name,est in
 ↪  [('LogisticRegression',LogisticRegression(max_iter=1000,class_weight='balanced')),('MLPClassifier'
 ↪  ,MLPClassifier(hidden_layer_sizes=(64,32,16),early_stopping=True,max_iter=150,random_state=254))]:
 model=make_pipeline(SimpleImputer(strategy='median'),StandardScaler(),est)
 model.fit(X.iloc[tr],y.iloc[tr]); pred=model.predict(X.iloc[te])
 rows.append(dict(task='classification',model=name,n_train=len(tr),n_test=len(te),accuracy=accuracy_sc
  ↪  ore(y.iloc[te],pred),macro_f1=f1_score(y.iloc[te],pred,average='macro')))
 joblib.dump(model,ROOT/'models'/f'{name}.joblib')
 if name.startswith('MLP'): pd.DataFrame({'row':te,'actual':y.iloc[te].values,'predicted':pred}).to_cs
  ↪  v(ROOT/'results/classification_predictions.csv',index=False)
d=load_diabetes(scaled=False)
tr,te=train_test_split(np.arange(len(d.target)),test_size=.2,random_state=254)
for name,est in [('Ridge',Ridge(alpha=10)),('MLPRegressor',MLPRegressor(hidden_layer_sizes=(64,32,16),
 ↪  max_iter=1000,early_stopping=True,random_state=254))]:
 model=make_pipeline(StandardScaler(),est);model.fit(d.data[tr],d.target[tr]);pred=model.predict(d.dat
  ↪  a[te])
 rows.append(dict(task='regression',model=name,n_train=len(tr),n_test=len(te),mae=mean_absolute_error(
  ↪  d.target[te],pred),r2=r2_score(d.target[te],pred)))
 joblib.dump(model,ROOT/'models'/f'{name}.joblib')
 if name.startswith('MLP'): pd.DataFrame({'row':te,'actual':d.target[te],'predicted':pred}).to_csv(ROO
  ↪  T/'results/regression_predictions.csv',index=False)
(ROOT/'results/model_metrics.json').write_text(json.dumps(rows,indent=2))
(ROOT/'models/schema.json').write_text(json.dumps({'classification':features,'regression':list(d.featu
 ↪  re_names),'seed':254,'classification_scope':'synthetic dataset
 ↪  labels','regression_scope':'research progression score, not glucose or personal disease
 ↪  probability'},indent=2))
print(json.dumps(rows,indent=2))
```

A.3 scripts/train_policy.py

```
from pathlib import Path
import json,numpy as np
root=Path(__file__).resolve().parents[1]
rng=np.random.default_rng(254);q=np.zeros((4,3));history=[]
# Explicit simulated workflow: missing evidence -> clarify; evidence -> cite;
# prediction plus evidence -> explain model and cite. Not a treatment policy.
for ep in range(3000):
 s=int(rng.integers(4));a=int(rng.integers(3)) if rng.random()<.2 else int(q[s].argmax())
 target=0 if s%2==0 else (2 if s==3 else 1)
 reward=1. if a==target else -1.
 q[s,a]+=.15*(reward-q[s,a]);history.append(reward)
payload={'seed':254,'episodes':3000,'q':q.tolist(),'actions':['Hỏi làm rõ','Trả lời có nguồn','Giải
 ↪  thích mô hình và dẫn nguồn'],'environment':'One-step simulated workflow, not clinical
 ↪  outcomes','last_300_reward':float(np.mean(history[-300:]))}
(root/'models/workflow_policy.json').write_text(json.dumps(payload,ensure_ascii=False,indent=2),encodi
 ↪  ng='utf-8')
print(payload['last_300_reward'])
```

A.4 app/agents.py

```
from dataclasses import dataclass,field
from typing import Callable,Any
@dataclass
class Message:
 sender:str
 receiver:str
 type:str
 content:Any
@dataclass
class Agent:
 name:str
 handler:Callable
 memory:dict=field(default_factory=dict)
 def receive(self,message,bus):
  self.memory['calls']=self.memory.get('calls',0)+1
  self.memory['last_request']=message.type
  return self.handler(message.content,bus)
class MessageBus:
 def __init__(self):self.agents={};self.trace=[]
 def register(self,agent):self.agents[agent.name]=agent
 def request(self,sender,receiver,content,kind='REQUEST'):
  self.trace.append(vars(Message(sender,receiver,kind, str(content) if isinstance(content,dict) and
   ↪  'x' in content else content)))
  try:
   result=self.agents[receiver].receive(Message(sender,receiver,kind,content),self)
   self.trace.append(vars(Message(receiver,sender,'INFORM',result.tolist() if hasattr(result,'tolist')
    ↪  else result)))
   return result
  except Exception as exc:
   self.trace.append(vars(Message(receiver,sender,'FAILURE',type(exc).__name__)))
   raise
```

A.5 app/main.py

```
import json, os, math, re, unicodedata
from pathlib import Path
import numpy as np
import pandas as pd
import joblib
from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field
from neo4j import GraphDatabase
from app.agents import Agent, MessageBus
from sklearn.feature_extraction.text import TfidfVectorizer
ROOT=Path(__file__).resolve().parents[1]
for line in (ROOT/'.env').read_text().splitlines() if (ROOT/'.env').exists() else []:
 if '=' in line:
  k,v=line.split('=',1);os.environ.setdefault(k,v)
kb=json.loads((ROOT/'data/knowledge.json').read_text(encoding='utf-8'))
docs=kb['documents']
vec=TfidfVectorizer(strip_accents='unicode',ngram_range=(1,2))
mat=vec.fit_transform([d['title']+' '+' '.join(d['terms'])+' '+d['text'] for d in docs])
schema=json.loads((ROOT/'models/schema.json').read_text())
```

```
models={k:joblib.load(ROOT/'models'/f'{v}.joblib') for k,v in
 ↪  [('classification','MLPClassifier'),('regression','MLPRegressor')]}
app=FastAPI(title='GlucoGuide • Giang KT')
app.mount('/assets',StaticFiles(directory=ROOT/'web'),name='assets')
class Request(BaseModel):
 question:str=Field(default='',max_length=3000)
 task:str='knowledge'
 values:dict=Field(default_factory=dict)
def graph_driver():
 return GraphDatabase.driver(os.environ.get('NEO4J_URI','bolt://127.0.0.1:7687'),auth=(os.environ.get(
  ↪  'NEO4J_USER','neo4j'),os.environ.get('NEO4J_PASSWORD','')),connection_timeout=3,connection_acquis
  ↪  ition_timeout=4,max_transaction_retry_time=0)
@app.get('/')
def home():return FileResponse(ROOT/'web/index.html')
@app.get('/api/metrics')
def metrics():return json.loads((ROOT/'results/model_metrics.json').read_text())
@app.get('/api/schema')
def fields():return schema
@app.get('/api/examples')
def examples():return json.loads((ROOT/'data/examples.json').read_text(encoding='utf-8'))
@app.get('/api/graph')
def graph():
 try:
  with graph_driver() as driver:
   rows=driver.execute_query('MATCH (a:Concept)-[r:RELATED_TO]->(b:Concept) RETURN a.name AS
    ↪  source,b.name AS target,r.label AS label,r.document AS document LIMIT 500').records
   edges=[dict(r) for r in rows]
   return {'storage':'Neo4j','nodes':sorted(set(e[k] for e in edges for k in
    ↪  ['source','target'])),'edges':edges}
 except Exception:raise HTTPException(503,'Neo4j chưa kết nối được. Khởi động cơ sở dữ liệu và kiểm
  ↪  tra cấu hình .env.')
@app.post('/api/chat')
def chat(req:Request):
 bus=MessageBus();trace=bus.trace
 bus.register(Agent("Mô hình",lambda payload,b: models[payload["task"]].predict(payload["x"])[0]))
 bus.register(Agent("Truy xuất",lambda payload,b: (mat@vec.transform([payload]).T).toarray().ravel()))
 bus.register(Agent("Đối chiếu",lambda payload,b: {"scope":payload.get("scope","Kiến thức có
  ↪  nguồn"),"allow_diagnosis":False}))
 bus.register(Agent("Phản hồi",lambda payload,b: "\n\n".join(d["text"]+" ["+d["id"]+"]" for d in
  ↪  payload)))
 def msg(sender,receiver,kind,content):trace.append(dict(sender=sender,receiver=receiver,type=kind,con
  ↪  tent=content))
 msg('Người dùng','Điều phối','REQUEST',req.question or req.task)
 if req.task not in ['knowledge','classification','regression']:raise HTTPException(422,'Bài toán
  ↪  không hợp lệ')
 result=None
 if req.task!='knowledge':
  fields=schema[req.task];missing=[f for f in fields if f not in req.values or req.values[f] in
   ↪  ['',None]]
  if missing:
   msg('Kiểm tra','Phản hồi','REQUEST','Bổ sung: '+', '.join(missing))
   return {'answer':'Cần bổ sung các trường: '+',
    ↪  '.join(missing),'trace':trace,'sources':[],'result':None}
  try:
   values=[float(req.values[f]) for f in fields]
   if not all(math.isfinite(v) for v in values):raise ValueError()
   if any(abs(v)>10000 for v in values):raise ValueError()
  except (ValueError,TypeError):raise HTTPException(422,'Các giá trị phải là số hữu hạn trong miền
   ↪  nhập liệu.')
```

```
 model=models[req.task];x=pd.DataFrame([values],columns=fields) if req.task=='classification' else
  ↪ np.array([values])
 pred=bus.request('Điều phối','Mô hình',{'task':req.task,'x':x})
 if req.task=='classification':
  result={'label':str(pred),'scores':dict(zip(map(str,model.classes_),map(float,model.predict_proba(x
   ↪ )[0]))),'scope':'Nhãn dữ liệu tổng hợp; không phải chẩn đoán hay xác suất mắc
   ↪ bệnh.'}
 else:result={'value':float(pred),'scope':'Điểm tiến triển bệnh nghiên cứu; không phải glucose hoặc
  ↪ HbA1c.'}
 msg('Mô hình','Đối chiếu','INFORM',json.dumps(result,ensure_ascii=False))
q=req.question or ('phân loại nhãn tổng hợp classification High Moderate Low' if
 ↪ req.task=='classification' else 'điểm tiến triển diabetes')
scores=bus.request('Điều phối','Truy xuất',q)
selected=[docs[i] for i in np.argsort(-scores)[:3] if scores[i]>max(.04,float(scores.max())*.65)]
msg('Điều phối','Tri thức','REQUEST',q)
msg('Tri thức','Đối chiếu','INFORM','Đã truy xuất: '+', '.join(d['id'] for d in selected))
graph_evidence=[]
try:
 with graph_driver() as driver:
  graph_evidence=[dict(r) for r in driver.execute_query('MATCH
   ↪ (a:Concept)-[r:RELATED_TO]->(b:Concept) WHERE r.document IN $ids RETURN a.name AS
   ↪ source,r.label AS relation,b.name AS target LIMIT 12',ids=[d['id'] for d in selected]).records]
 msg('Neo4j','Tri thức','INFORM',str(len(graph_evidence))+' quan hệ có nguồn')
except Exception:msg('Neo4j','Tri thức','FAILURE','Không kết nối; câu trả lời chỉ dùng truy xuất văn
 ↪ bản.')
# Learned workflow policy; rules constrain output when no evidence exists.
policy=json.loads((ROOT/'models/workflow_policy.json').read_text(encoding='utf-8'))
state=int(result is not None)*2+int(bool(selected)); action=int(np.argmax(policy['q'][state]))
if not selected:action=0
msg('Chính sách RL','Phản
 ↪ hồi','INFORM',{'state':state,'action':policy['actions'][action],'rule':'Thiếu bằng chứng thì yêu
 ↪ cầu làm rõ'})
answer=bus.request('Điều phối','Phản hồi',selected) if action in (1,2) else 'Tôi chưa tìm thấy bằng
 ↪ chứng phù hợp trong kho tri thức. Bạn hãy nêu rõ chỉ số hoặc chủ đề cần tìm hiểu.'
if result:
 verdict=bus.request('Điều phối','Đối chiếu',result)
 answer=verdict['scope']+'\n\n'+answer
msg('Đối chiếu','Phản hồi','INFORM','Giữ đúng phạm vi mô hình, dẫn nguồn tài liệu; không suy ra chẩn
 ↪ đoán từ nhãn tổng hợp.')
msg('Phản hồi','Người dùng','INFORM','Hoàn thành')
return {'answer':answer,'result':result,'trace':trace,'sources':[{'id':d['id'],'title':d['title'],'so
 ↪ urce':kb.get('sources',{}).get(d.get('source'),{})} for d in
 ↪ selected],'graph_evidence':graph_evidence,'generator':'Tổng hợp trích đoạn xác định, không gọi
 ↪ LLM'}
```

A.6 scripts/test_scenarios.py

```
from pathlib import Path
import json
from fastapi.testclient import TestClient
from unittest.mock import patch
from app.main import app
r=Path(__file__).resolve().parents[1];c=TestClient(app);examples=json.loads((r/'data/examples.json').r
 ↪ ead_text())
```

```
cases=[('knowledge',{'question':'HbA1c là
 ↪  gì?'},200),('unknown',{'question':'zzzxxyy'},200),('classification',{'task':'classification','valu
 ↪  es':examples['classification']},200),('regression',{'task':'regression','values':examples['regress
 ↪  ion']},200),('missing',{'task':'classification','values':{}},200),('invalid',{'task':'classificati
 ↪  on','values':{**examples['classification'],'Age':'nan'}},422),('bad_task',{'task':'unknown'},422)]
results=[]
for name,payload,status in cases:
 response=c.post('/api/chat',json=payload);body=response.json();assert
  ↪  response.status_code==status,(name,body)
 if name=='unknown':assert body['sources']==[]
 if name=='knowledge':assert body['sources'][0]['id']=='a1c' and body['graph_evidence']
 if name=='classification':assert body['result']['label'] in ['High','Moderate','Low']
 if name=='regression':assert isinstance(body['result']['value'],float)
 if name=='missing':assert body['result'] is None
 results.append({'case':name,'status':status,'passed':True,'response':body})
with patch('app.main.graph_driver',side_effect=RuntimeError('controlled outage')):
 response=c.post('/api/chat',json={'question':'HbA1c'});body=response.json();assert
  ↪  response.status_code==200 and any(t['type']=='FAILURE' for t in body['trace'])
 assert c.get('/api/graph').status_code==503
 results.append({'case':'controlled_neo4j_outage','passed':True,'response':body})
g=c.get('/api/graph').json();assert g['storage']=='Neo4j' and len(g['nodes'])>0
(r/'results/api_scenarios.json').write_text(json.dumps(results,ensure_ascii=False,indent=2),encoding='
 ↪  utf-8')
print(json.dumps({'passed':len(results),'nodes':len(g['nodes']),'edges':len(g['edges'])}))
```